

KAMP 제조AI데이터셋 활용 권한

No. GB20250818-PU1755498594-243799



⚠ 제조AI데이터셋 활용 주의사항

연구/공식적으로 사용하실 때에는 꼭 아래와 같이 'KAMP 출처(reference)'를 남겨주시고, 인용 시 활용한 내용과 문서 등은 아래 이메일로 보내주시기를 바랍니다.
(E-mail : kamp@kaist.ac.kr)

국문 출처 표기 양식

중소벤처기업부, Korea AI Manufacturing Platform(KAMP), 의류 공정최적화 AI 데이터셋, 스마트제조혁신추진단(주)임팩스, 2023.08.18., www.kamp-ai.kr

영문 출처 표기 양식

Ministry of SMEs and Startups., and KAIST (Korea Advanced Institute of Science and Technology). (2020, December 14). Apparel process optimization AI dataset. Korea AI Manufacturing Platform(KAMP). <https://www.kamp-ai.kr/>

제조AI데이터셋 개요

제조AI데이터셋명 : 의류 공정최적화 AI 데이터셋

업종 : 의류 | **목적** : 생산공정 최적화 | **유형** : csv

알고리즘 : 1) 랜덤포레스트 회귀
2) 유전 알고리즘

사용조건 : 콘텐츠 변경허용

제공기관 : 스마트제조혁신추진단 (수행기관 : (주)임팩스)

등록일 : 2023-08-18

이용자 정보

회원 ID : 01soju

다운로드일 : 2025-08-18 17:28:20

활용목적 : 공부

인공지능 제조 플랫폼(KAMP, Korea AI Manufacturing Platform)에서 다운로드한 제조AI데이터셋의 저작권을 위하여 문서 고유 번호와 워터마크를 부여하여, 해당 문서의 표준을 준수하였음을 증명합니다.

운영기관 : KAIST 제조AI빅데이터센터

주소 : 대전광역시 유성구 문지로 193 KAIST 문지캠퍼스 행정동

전화번호 : 042-350-1331 | 이메일 : kamp@kaist.ac.kr

「의류 공정최적화 AI 데이터셋」 분석실습 가이드북



「의류 공정최적화 AI 데이터셋」 분석실습 가이드북





Contents

1 분석요약	04
2 분석 실습	
1. 분석 개요	05
1.1 분석 배경	05
1) 공정(설비) 개요	
2) 이슈사항(Pain point)	
1.2 분석 목표	08
1) 분석 목표	
2) 데이터 정의 및 소개	
3) 데이터 분석 기대효과	
4) 시사점(implication) 요약기술	
2. 분석실습	10
2.1 제조데이터 소개	10
1) 데이터 수집 방법	
2) 데이터 유형/구조	
3) 데이터 (품질) 전처리	
2.2 분석 모델 소개	18
1) 데이터 흐름 및 인공지능 모델 적용 흐름도	
2) AI 분석모델	
2.3 분석 체험	28
1) 필요 SW, 패키지 설치 방법 및 절차 가이드	
2) 분석 단계별 프로세스 - Flow Chart	
[단계 ①] 라이브러리 및 데이터 불러오기	
[단계 ②] 데이터 탐색 및 특성 파악	
[단계 ③] 데이터셋 전처리 및 병합	
[단계 ④] 데이터 시각화	
[단계 ⑤] 주요 변수 선택	
[단계 ⑥] 데이터셋 분리 및 스케일링	
[단계 ⑦] 모델 구축, 학습 및 평가	
[단계 ⑧] 유전 알고리즘(Genetic Algorithm) 적용 및 공정 최적화	
3. 유사 타 현장의 「의류 공정최적화 AI 데이터셋」 분석 적용	77
부 록	
분석환경 구축을 위한 설치 가이드	79
데이터 품질 전처리 [실습 코드]	92



「의류 공정최적화 AI 데이터셋」 분석실습 가이드북

- 필요 SW : Python, Anaconda - Jupyter Notebook
- 필요 패키지 : Pandas, Numpy, Matplotlib, seaborn, scikit-learn, Shap, geneticalgorithm
- 분석 환경 : [CPU] Inter i7 2.6 GHz, [RAM] 40GB
- 필요 데이터 : LOT 물량.xlsx, PRODUCTION_TREND.csv, CCM 측정값.xlsx

1 분석요약

No	구분		내용
1	분석 목적 (현장 이슈, 목적)		- 숙련공이 아니면 파악할 수 없는 자신만의 노하우(Know-How)로 생산 활동을 진행하고 있고, 컬러 검사는 CCM(Computer Color Matching) 장비와 사람의 육안으로 판별하는데 이는 비효율적이며 일관적이지가 않으며, 현재 CCM에서 측정하는 컬러 요소에 대한 오차값의 신뢰도가 낮아 최종 육안으로 판정하고 있다. - 염색가공 공정에서 품질문제와 공정 데이터들을 분석하여 데이터 간의 상관관계를 찾고 제품 컬러에 영향을 주는 인자를 분석하여 품질을 예측하고, 이를 기반으로 공정 최적화 모델을 만들고자 한다.
2	데이터셋 형태 및 수집방법		1) 분석에 사용된 변수 : '염색 가동 길이', 'TRD_TEMP_PV', 'TRD_SPEED1', '염색 색차 DE' 등 총 11개 변수 2) 데이터 수집 방법 : 염색설비 PLC, CCM 검사설비 PLC, 물량정보 PC key-in 등을 통해 설비/품질/물량 데이터 확보 3) 데이터셋 파일 확장자 : xlsx, csv
3	데이터 개수 데이터셋 총량		- 데이터 개수 : 총 31,813,751개 - 데이터셋 총량 : 157MB
4	분석적용 알고리즘	알고리즘	1) 랜덤포레스트 회귀 2) 유전 알고리즘
		알고리즘 간략소개	1) 랜덤포레스트 알고리즘은 결정 트리를 기반으로 하는 알고리즘으로, 여러 개의 학습 모델이 내린 예측 결과를 다수결로 통합하여 여러 결과로 합치는 앙상블 기법이다. 각각의 의사결정나무의 평균을 통해 값을 예측함에 따라 과적합을 방지할 수 있고, 높은 안정성을 보인다. 2) 유전 알고리즘은 전역 최적화(Global Optimization) 기법 중 하나로, 생물의 진화를 모방한 진화 연산 기법이다. 특정한 문제를 풀기 위한 알고리즘이라기보다는 최적화 문제를 풀기 위한 방법론에 가까우며, 데이터를 요구되는 형식에 맞게 바꿔주기만 하면 모든 문제에 적용이 가능한 매우 유용한 알고리즘이다. 자연 진화의 염색체 교차(Crossover)와 변이(Mutation) 등의 과정을 반복하며 최적의 해를 찾아간다.
5	분석결과 및 시사점		- 분석결과 : 랜덤포레스트 알고리즘을 활용하여 염색품질 예측 모델을 구축하고, 이를 다시 유전 알고리즘에 적용하여 공정 최적값을 도출한 결과, 목표 염색품질 값에 매우 근접하게 최적화가 가능함을 확인하였다. 다만, 제조회장에 적용을 위해서는 도메인 지식에 기반하여 변수 범위 및 제한조건에 대한 보다 정교한 설정이 필요하다. - 시사점 : 현장에서 수집한 제조회데이터에 머신러닝과 최적화 기법의 적용을 통해 제품품질에 대한 예측뿐만 아니라 목표 품질값 달성을 위한 공정조건 최적화가 가능함을 확인하였다. 이를 통해 품질향상과 비용절감의 효과를 기대할 수 있다.



2 분석 실습

1. 분석 개요

1.1 분석 배경

1) 공정(설비) 개요

• 섬유 염색가공 공정 특징

- 섬유 가공은 제직 공정, 세팅 공정, 염색공정, 가공공정, 검사공정으로 구성되어 있다.

• 제직공정

- 직물의 길이 방향과 평행되게 배열된 실들을 경사라 하고, 경사를 가로질러 경사와 직각으로 교차되는 실들을 위사라고 한다. 경사와 위사를 교차하여 직물을 만드는 공정이다.

• 세팅공정

- Stenter기로 원단 양 가장자리를 핀으로 고정하여 일정한 열을 가한다. 염색 전 원단을 일정한 폭과 길이로 형태 안정성을 부여하는 공정이다.

• 염색공정

- 원단을 염색하기 위하여 염료와 조제를 CCK(염색자동화시스템)를 활용하여 레시피의 표준량을 염색기로 이송한다.
- 이송된 염료와 조제에 표준량의 물과 원단을 염색기로 투입하여 온도, 시간 등의 환경을 염색 제어시스템에서 제어하여 원단에 컬러를 부여하는 공정이다.
- 염색 후 샘플을 절취하여 CCM(Computer Color Matching) 장치 또는 육안으로 컬러를 확인한다.
- 컬러의 수정이 필요할 경우 수정 레시피를 작성하여 염색수정 작업을 수행한다.

• 가공공정

- 세팅공정과 동일하게 Stenter기로 원단 양 가장자리를 핀으로 고정하여 일정한 열을 가한다. (온도 등의 작업조건은 다를 수 있다.)
- 염색 후 원단을 최종 제품의 폭과 길이로 형태 안정성을 부여하는 공정이다.

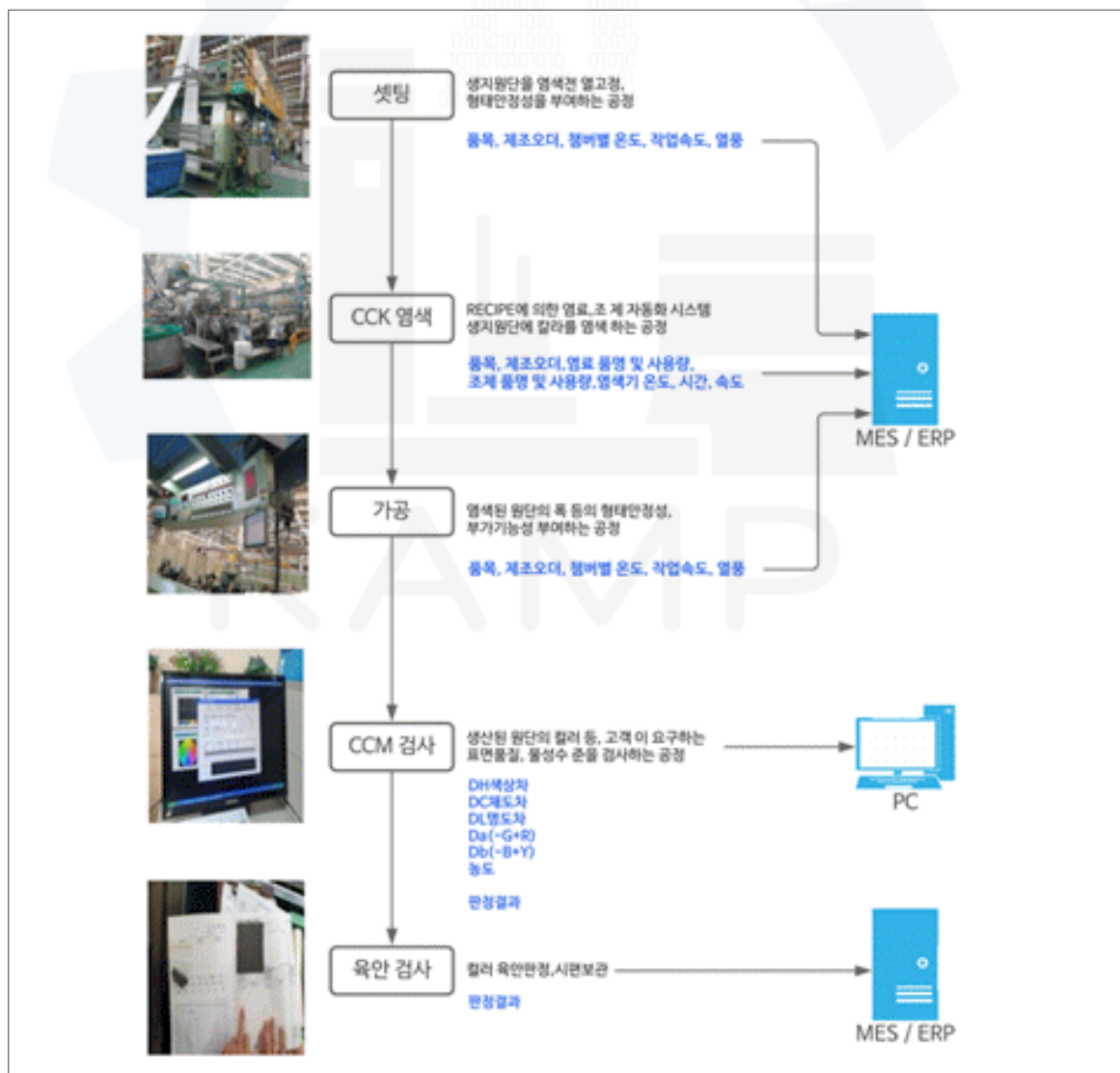


- 고객이 제품에 발수 기능 등의 별도 추가 기능을 원할 경우 기능성 조제를 원단에 부가한 후 Stenter기에 투입하여 가공작업을 수행한다.

• 검사공정

- 가공작업이 완료된 원단을 검단기에서 검사기준에 따라서 오염 등의 결점을 확인하고 일정한 수량으로 감는 공정이다.
- 컬러는 1차 검사자가 기존 컬러와 CCM 판별을 수행하고 판별이 어려울 경우 1차 부적합을 등록하여 품질관리자, 생산관리자, 영업담당자가 2차로 판별을 수행한다.
- 1차 판별 또는 2차 판별에서 합격한 제품은 최종 고객에게 컬러 판정을 받은 후 합격한 제품은 출고 진행을 수행한다.

• 염색가공 공정 및 자료수집 구성도



[그림 1] 염색가공 공정 및 자료수집 구성도



2) 이슈사항(Pain point)

• 공정(설비)상의 문제 현황

- 세팅/염색/가공 공정의 작업 표준은 정의되어 있으나 표준의 도출과정이 객관적이지 못하며, 구체적이지 못한 부분이 있다.
- 문제 발생 시에 설비의 세팅값 조정에 대한 객관화된 가이드가 없이, 숙련공의 노하우를 기반으로 변경하고 있으며, 불량 발생에 대한 원인 규명도 주관적인 판단으로 이루어지고 있다.
- 현장 관리자 과거의 경험과 지식을 통해 개선 활동을 하고 있으며, 숙련공이 아니면 파악할 수 없는 자신만의 노하우(Know-How)로 생산 활동을 진행하고 있다.
- 현장에서 오랫동안 생산을 해온 숙련공들의 생산 지식 및 노하우를 데이터화하여 개인의 능력이 아닌 기업의 경쟁력을 높여줄 생산기술 자산화가 필요하다.

• 문제해결 장애요인

- 기존 레시피에 따라 진행하였으나 원하는 컬러가 아닌 경우에도 불량으로 판정되어 재생산해야 한다.
- 현재 컬러 검사는 CCM과 사람의 육안으로 판별하는데 이는 비효율적이며 일관적이지가 않고, CCM에서 측정하는 컬러 요소에 대한 오차값의 신뢰도가 낮아 최종 육안으로 판정하고 있다.
- 염색공정에서 가공공정으로 넘어가기 전 컬러판별을 통해 불량 컬러 발생 시 수정 레시피를 작성하여 수정작업을 진행하며 품질의 적합수준까지 재작업하는 경우도 발생하고 있다.

• 극복 방안

- 이미 수집된 염색공정의 설비데이터 및 품질데이터를 활용하여 AI 모델을 통한 염색 품질을 예측하고, 목표 염색품질값 달성을 위한 공정 최적화 모델을 활용하여 해결하고자 한다.



1.2 분석 목표

1) 분석목표

- 제직/세팅/염색/가공 공정에서 품질문제와 공정 데이터들을 분석하여 데이터 간의 상관관계를 찾고 제품 컬러에 영향을 주는 인자를 분석하여 품질을 예측하고, 이를 기반으로 공정 최적화 모델을 만들고자 한다.

2) 데이터 정의 및 소개

• LOT 물량 데이터

- 제품번호, 공정코드, 원단 투입 중량, 염색기 투입 액량, 원단 투입 길이 등 제품 LOT 별 생산 물량 관련 데이터로 이루어진다.

• 설비 데이터

- 제품번호, 공정코드, 작업횟수, 공정진행시간, 염색설비 목표온도, 지시온도(제어 목표온도), 진행온도, 포속1(1번 튜브 포지 회전속도), 포속2(2번 튜브 포지 회전속도), 포속3(3번 튜브 포지 회전속도), 포속4(4번 튜브 포지 회전속도) 등 염색공정 운영 관련 데이터로 이루어진다.

• 품질 측정 데이터(CCM 측정 결과)

- 제품번호, 품질검사 검사차수, DHue(색조차), DC(채도차), DL(명도차), Da/Db(색상 좌표값), De(염색 색차값) 등의 CCM 측정 결과 데이터로 이루어진다. 이 중 본 분석의 종속변수인 De(DeltaE 또는 dE로 표기)는 색차(color difference)를 나타내는 값으로, 두 색상 간의 거리로 정의할 수 있다. De 값은 CCM 시스템을 통해 측정한 다른 검사항목(색조차, 채도차, 명도차 등)을 조합한 계산식을 통해 산출한다. 참고로 De 1.0이 인간의 눈으로 차이를 구별할 수 있는 정도이다.

3) 데이터 분석 기대효과

- 기존의 단순 수집에 목적을 둔 데이터 축적 방식에서 벗어나 AI 데이터 분석을 위한 데이터셋 구성 환경 구축으로 전환할 수 있을 것으로 판단된다.
- 설비 운영변수를 포함한 모든 공정변수와 품질과의 상관관계 분석을 통해 변수 간의 관계성을 파악하여 AI 공정최적화 모델을 통해 제품 생산 공정에 넓게 활용할 수 있을 것으로 판단된다.



4) 시사점(implication) 요약기술

- 기존에는 육안판별과 CCM 검사결과를 바탕으로 작업자의 주관적인 판단으로 작업완료 여부가 결정되었기 때문에 작업자의 경험과 노하우에 의존할 수밖에 없었으나, AI 기반의 컬러판별 시스템의 기틀을 마련하였기 때문에 작업자의 숙련도에 의존하지 않아도 양품을 생산할 수 있는 최적의 공정 조건을 도출하였다.
- 공정 데이터와 최종 품질검사 데이터를 수집하고 데이터 가공/전처리, AI 모델 개발과 제조공정의 적용 및 검증을 통해 열악한 중소기업에 빅데이터 및 AI 기술을 적용하여 실질적인 품질향상 및 비용절감에 기여한다는 점에서 시사하는 바가 크다.





2. 분석실습

2.1 제조데이터 소개

1) 데이터 수집 방법

- 제조분야 : 섬유 및 인공피혁
- 제조 공정명 : 염색공정
- 수집 장비 : ① 'LOT 물량' 데이터 - MES, PC key-in
② 'PRODUCTION_TREND' 데이터 - PLC
③ 'CCM 측정값' 데이터 - PLC
- 수집 기간 : 2022년 1월 ~ 2022년 10월 (약 10개월)
- 수집 주기 : ① 'LOT 물량' 데이터 - 불규칙(LOT 종료시 수집)
② 'PRODUCTION_TREND' 데이터 - 사이클타임 1분
③ 'CCM 측정값' 데이터 - 불규칙(LOT 종료시 수집)

EXT1_QTY(투입중량 (KG))	EXT2_QTY (액량 (LITER))	PRODT_ORDER_NO	JOB_CD	PROCESS_CD	단위중량	염색 가동 길이
245.6	4000	F2111300008	F12-1203	F12-1203	590	120
71.4	600	F2111300012	F12-1203	F12-1203	445	123
98.3	4000	F2111300020	F12-1203	F12-1203	0	460
97	4000	F2111300021	F12-1203	F12-1203	0	452
40.6	600	F2111300027	F12-1203	F12-1203	445	86
163.6	2000	F2111300031	F12-1203	F12-1203	445	218
190.1	4000	F2111300041	F12-1203	F12-1203	146	1600
191	4000	F2111300042	F12-1203	F12-1203	146	1600
193.6	4000	F2111300043	F12-1203	F12-1203	146	1600
193.6	4000	F2111300043	F12-1203	F12-1203	146	1600
188.2	4000	F2111300044	F12-1203	F12-1203	146	1600
190.4	4000	F2111300045	F12-1203	F12-1203	146	1600
182.6	2000	F2111300072	F12-1203	F12-1203	400	115
142.4	2000	F2111300104	F12-1203	F12-1203	400	62
49.6	1000	F2111300139	F12-1203	F12-1203	294	27
250.8	4000	F2112010011	F12-1203	F12-1203	590	410
156	4000	F2112010012	F12-1203	F12-1203	590	340
260	4000	F2112010013	F12-1203	F12-1203	590	480
117.3	2000	F2112010015	F12-1203	F12-1203	300	14
260	4000	F2112010016	F12-1203	F12-1203	590	419
52	600	F2112010055	F12-1203	F12-1203	400	3
81.4	1000	F2112010057	F12-1203	F12-1203	475	2
85.2	4000	F2112020007	F12-1203	F12-1203	213	358
138.4	4000	F2112020008	F12-1203	F12-1203	213	54
133.8	4000	F2112020011	F12-1203	F12-1203	213	483
26.6	600	F2112020013	F12-1203	F12-1203	213	82
174.4	2000	F2112020017	F12-1203	F12-1203	380	442
111.2	4000	F2112020037	F12-1203	F12-1203	190	581
128.8	4000	F2112020038	F12-1203	F12-1203	190	581

[그림 2] 데이터셋#1 'LOT 물량' 데이터 스크린샷

LOT_NO	WC_CD	WC_CNT	SEQ_NO	PGM_ID	RESOURCE_CD	CR_TEMP	TRD_TEMP_SP	TRD_TEMP_PV	TRD_SPEED1	TRD_SPEED2	TRD_SPEED3	TRD_SPEED4
F2112280C F12-1203		1	1	B9700	FCM04	30	30	14.5	0	0	0	0
F2112280C F12-1203		1	2	B9700	FCM04	30	30	16.1	0	0	0	0
F2112280C F12-1203		1	3	B9700	FCM04	30	30	19.6	111	1	112	1
F2112280C F12-1203		1	4	B9700	FCM04	30	30	20	111	1	112	1
F2112280C F12-1203		1	5	B9700	FCM04	30	30	19.9	111	1	112	1
F2112280C F12-1203		1	6	B9700	FCM04	30	30	19.9	111	1	112	1
F2112280C F12-1203		1	7	B9700	FCM04	50	30	19.8	111	1	112	1
F2112280C F12-1203		1	8	B9700	FCM04	50	30	19.8	111	1	112	1
F2112280C F12-1203		1	9	B9700	FCM04	60	31.1	18	111	1	112	1
F2112280C F12-1203		1	10	B9700	FCM04	60	31.1	23	87	2	86	2
F2112280C F12-1203		1	11	B9700	FCM04	60	31.1	27.6	87	2	86	2
F2112280C F12-1203		1	12	B9700	FCM04	100	34.2	31.2	91	3	90	3
F2112280C F12-1203		1	13	B9700	FCM04	120	38.4	35.4	92	4	87	4
F2112280C F12-1203		1	14	B9700	FCM04	120	42.3	39.3	92	4	87	4
F2112280C F12-1203		1	15	B9700	FCM04	135	46.4	43.4	91	5	87	5
F2112280C F12-1203		1	16	B9700	FCM04	135	50	47.3	90	6	85	6
F2112280C F12-1203		1	17	B9700	FCM04	80	50	47.7	90	6	85	6
F2112280C F12-1203		1	18	B9700	FCM04	70	50	45.2	88	7	84	7
F2112280C F12-1203		1	19	B9700	FCM04	70	50	45.3	88	7	85	8
F2112280C F12-1203		1	20	B9700	FCM04	30	50.5	46.4	89	8	85	8
F2112280C F12-1203		1	21	B9700	FCM04	30	53.3	51.9	90	9	88	9
F2112280C F12-1203		1	22	B9700	FCM04	30	56.7	55.2	90	9	87	10
F2112280C F12-1203		1	23	B9700	FCM04	30	60	58.8	90	9	87	10
F2112280C F12-1203		1	24	B9700	FCM04	30	60	56.3	90	9	85	11
F2112280C F12-1203		1	25	B9700	FCM04	30	60	55.5	269	10	87	12
F2112280C F12-1203		1	26	B9700	FCM04	30	60	55.4	269	10	87	12
F2112280C F12-1203		1	27	B9700	FCM04	0	60	55.7	94	11	90	13
F2112280C F12-1203		1	28	B9700	FCM04	0	60.4	57.9	94	11	90	14
F2112280C F12-1203		1	29	B9700	FCM04	0	61.9	61.6	93	12	90	14
F2112280C F12-1203		1	30	B9700	FCM04	0	63.6	63.4	93	13	91	15
F2112280C F12-1203		1	31	B9700	FCM04	0	65	64.2	93	13	90	16
F2112280C F12-1203		1	32	B9700	FCM04	0	66.7	66.1	93	13	90	16

[그림 3] 데이터셋#2 'PRODUCTION_TREND' 데이터 스크린샷

lot_no	seq	oper_id	염색 색자 DL	염색 색자 DA	염색 색자 DB	염색 색자 DC	염색 색자 DH	염색 색자 DE
F2002240067	0	염색	1.402431	-0.07987022	0.2555132	0.06365299	-0.2600278	1.427754
F2003120058	0	염색	-0.04318619	-0.02196431	0.6147623	0.03267336	-0.6142862	0.6166686
F2006160123	0	원단검사	12.72453	-1.963228	-1.810259	1.940723	-1.834361	13.00173
F2009030093	0	원단검사	0.2431526	0.2516061	-3.862172	3.830362	-0.5549878	3.87799
F2009030093	0	원단검사	0.6737862	0.1993477	-3.894216	3.854515	-0.5893819	3.9571
F2009050181	0	염색	1.032806	-0.03953278	0.04003143	-0.03947449	-0.04008822	1.034338
F2009140009	0	염색	1.150017	0.5714595	-2.682459	-2.485372	1.159776	2.974003
F2010020008	0	원단검사	12.73621	-2.11367	-1.712829	1.902705	-1.944506	13.02354
F2010080143	0	원단검사	1.630646	2.445013	-0.2168775	2.448905	0.1672998	2.946885
F2011040339	0	원단검사	0.8417549	0.2781898	-3.523368	3.499851	-0.4924991	3.633189
F2011040630	0	원단검사	0.6532822	0.9754601	0.2052665	0.9351263	0.3452468	1.19182
F2012021018	0	원단검사	-0.1163635	0.2553167	-0.06710291	0.2629223	-0.0236918	0.2884959
F2012110046	0	염색	2.462723	0.1294911	-1.441013	-1.444911	0.07429474	2.856272
F2012110046	QC	원단검사	-0.2369766	0.09942055	-0.9436607	-0.9483385	0.0321575	0.9780276
F2012110055	0	염색	0.02278137	0.1087189	0.546217	0.5342646	-0.1572709	0.5573973
F2012230062	QC	원단검사	-4.120605	0.2439618	-1.815939	-1.832237	0.007628052	4.509605
F2012280071	0	염색	1.643995	0.4790723	0.3181696	-0.2652912	0.5102575	1.741684
F2012280071	QC	원단검사	1.264555	0.4053116	0.7778225	-0.7306013	0.4852907	1.538956
F2101050023	0	염색	0.182457	-0.7344484	4.590677	-4.61268	0.5804402	4.652636
F2103020135	0	원단검사	0.3569012	-1.82353	0.1485767	0.6425209	-1.713039	1.864058
F2103050146	0	염색	1.124537	-0.149861	-0.06158257	0.06445026	-0.14865	1.136148
F2104220009	0	원단검사	-0.6880875	0.3035069	-3.523767	3.504424	-0.4775577	3.603126
F2104220009	0	원단검사	-0.626873	0.3232956	-3.521222	3.505261	-0.4654745	3.591169
F2104270080	0	염색	0.6583977	-0.9627941	1.774334	-1.893011	-0.7012436	2.123375
F2104270080	1	염색	0.6397305	-0.9571316	1.699339	-1.817919	-0.706417	2.052586
F2104280005	0	염색	-0.6626053	-0.3237724	0.272572	0.3907809	0.1625249	0.7862378
F2104280102	0	염색	-0.9150009	-0.4164577	-1.046157	-1.094034	-0.2664028	1.450899
F2105060046	0	염색	1.490326	-0.6565452	0.7301807	0.64012	0.7446218	1.784737
F2105070069	0	염색	2.212891	-0.2588034	-0.3130436	-0.2944622	0.2797636	2.249858
F2105070069	1	염색	-0.5632477	-0.01007318	-0.1492376	-0.1483593	0.01904854	0.5827703
F2105070082	1	염색	-1.009823	0.1493096	-0.086689	0.06154251	0.1613096	1.024476
F2105070082	1	염색	0.1570053	-0.3888607	0.2861252	-0.2085991	-0.4353926	0.5076721

[그림 4] 데이터셋#3 'CCM 측정값' 데이터 스크린샷



2) 데이터 유형/구조

- 데이터셋 구조 : 테이블 형식(Tabular)
- 데이터 개수

구분	파일명	Column 개수	Row 개수	데이터 개수
데이터셋#1	LOT 물량.xlsx	7	2,150	15,050
데이터셋#2	PRODUCTION_TREND.csv	15	2,119,545	31,793,175
데이터셋#3	CCM 측정값.xlsx	9	614	5,526
합계		31	2,122,309	31,813,751

[표 1] 데이터 개수

• 제조AI데이터셋 주요 변수 정의

- 데이터셋에는 LOT번호, 설비번호, 투입중량, 다양한 염색공정 설비운영정보 및 CCM 검사결과 정보 등이 포함되며, 주요 변수는 아래 표와 같다. LOT 물량 변수와 염색공정 설비운영 변수는 독립변수로, CCM 검사결과 변수는 종속변수로 활용된다.

No	속성(column)	설명	비고
1	EXT1_QTY(투입중량 (KG))	원단 투입중량(kg)	데이터형: float
2	EXT2_QTY (액량 (LITER))	염색 투입액량(L)	데이터형: int
3	PRODT_ORDER_NO	제품번호, 생산 LOT번호	데이터형: object
4	JOB_CD	공정코드	데이터형: object
5	PROCESS_CD	공정코드	데이터형: object
6	단위중량	염색 단위중량(kg)	데이터형: int
7	염색 가동 길이	원단 투입길이(m)	데이터형: float

[표 2] 데이터셋#1 'LOT 물량' 데이터의 변수 속성 정의

No	속성(column)	설명	비고
1	LOT_NO	제품번호, 생산 LOT번호	데이터형: object
2	WC_CD	공정코드	데이터형: object
3	WC_CNT	작업횟수	데이터형: int
4	SEQ_NO	공정진행시간	데이터형: int
5	PGM_ID	염색설비 프로그램 코드	데이터형: object
6	RESOURCE_CD	염색설비 번호	데이터형: object
7	CR_TEMP	염색설비 가동 목표온도(℃)	데이터형: int
8	TRD_TEMP_SP	염색설비 가동 지시온도(℃)	데이터형: float
9	TRD_TEMP_PV	염색설비 가동 실제온도(℃)	데이터형: float
10	TRD_SPEED1	염색설비 튜브1 회전속도	데이터형: int
11	TRD_SPEED2	염색설비 튜브2 회전속도	데이터형: int
12	TRD_SPEED3	염색설비 튜브3 회전속도	데이터형: int
13	TRD_SPEED4	염색설비 튜브4 회전속도	데이터형: int
14	INSRT_DT	공정일시	데이터형: object
15	PRODUCTION_RESULT_iD	공정결과코드	데이터형: object

[표 3] 데이터셋#2 'PRODUCTION_TREND' 데이터의 변수 속성 정의

No	속성(column)	설명	비고
1	lot_no	제품번호, 생산 LOT번호	데이터형: object
2	seq	염색품질검사 검사차수	데이터형: object
3	oper_id	작업구분: '원단검사' 또는 '염색'	데이터형: object
4	염색 색차 DL	CCM 색상검사항목: DL, 명도차	데이터형: float
5	염색 색차 DA	CCM 색상검사항목: Da, 색상 좌표값(-G, +R)	데이터형: float
6	염색 색차 DB	CCM 색상검사항목: Db, 색상 좌표값(-B, +Y)	데이터형: float
7	염색 색차 DC	CCM 색상검사항목: DC, 채도차	데이터형: float
8	염색 색차 DH	CCM 색상검사항목: DHue, 색조차	데이터형: float
9	염색 색차 DE	CCM 색상검사항목: 계산식으로 산출된 종합색차	데이터형: float

[표 4] 데이터셋#3 'CCM 측정값' 데이터의 변수 속성

※ object : 문자, int : 정수, float : 실수



• 주요 변수 기술 통계

변수	개수	평균	표준편차	최솟값	중앙값	최댓값
EXT1_QTY(투입중량 (KG))	2150	146.88	85.31	0	150.6	190.75
EXT2_QTY(액량 (LITER))	2150	2460.37	1338.93	600	2000	4000
단위중량	2150	390.44	194.84	0	400	560
염색 가동 길이	2150	432.93	522.60	1	262	500

[표 5] 데이터셋#1 'LOT 물량' 데이터의 수치형 변수 기술통계

변수	개수	평균	표준편차	최솟값	중앙값	최댓값
WC_CNT	2119545	1.20	15.42	1	1	1203
SEQ_NO	2119545	164.24	124.06	1	147	1381
CR_TEMP	2119545	5.69	18.50	0	0	138
TRD_TEMP_SP	2119545	68.01	36.08	0	70	138
TRD_TEMP_PV	2119545	69.48	34.12	0	69.5	152.5
TRD_SPEED1	2119545	101.30	411.97	0	65	9998
TRD_SPEED2	2119545	115.96	207.61	0	72	9027
TRD_SPEED3	2119545	91.25	388.37	0	61	9997
TRD_SPEED4	2119545	89.94	247.84	0	46	9058

[표 6] 데이터셋#2 'PRODUCTION_TREND' 데이터의 수치형 변수 기술통계

변수	개수	평균	표준편차	최솟값	중앙값	최댓값
염색 색차 DL	614	-0.56	5.45	-20.13	0.00	40.77
염색 색차 DA	614	-1.01	5.49	-14.90	-0.10	71.51
염색 색차 DB	614	1.92	8.14	-22.79	0.11	44.79
염색 색차 DC	614	-1.84	9.75	-31.88	-0.12	81.71
염색 색차 DH	614	0.03	1.60	-7.68	-0.02	20.61
염색 색차 DE	614	4.89	10.35	0.02	1.39	93.48

[표 7] 데이터셋#3 'CCM 측정값' 데이터의 수치형 변수 기술통계



• 독립변수/종속변수 정의

- 독립변수는 다른 변수의 변화와 관계없이 독립적으로 변화하는 변수, 즉, 다른 변수에 영향을 받지 않는 변수로, 입력값이나 원인을 나타낸다. 종속변수는 독립변수의 변화에 따라 영향을 받는 변수로, 결과물이나 효과를 나타낸다.
- 섬유 및 인공피혁 염색공정의 공정 최적화 모델링을 위해 전처리 완료된 데이터셋에서 최종 선별된 독립변수 및 종속변수는 하기 표와 같다.

구분	변수	변수명 변경 전 명칭
독립변수	단위중량(kg)	단위중량
	투입중량(kg)	EXT1_QTY(투입중량 (KG))
	염색길이(m)	염색 가동 길이
	투입중량/길이	*파생변수
	투입중량/액량	*파생변수
	공정진행시간(%)	*파생변수
	진행온도	TRD_TEMP_PV
	포속1	TRD_SPEED1
	포속3	TRD_SPEED3
	포속4	TRD_SPEED4
종속변수	염색색차 DE	염색 색차 DE

[표 8] 독립변수와 종속변수 정의

※ 파생변수: 데이터 전처리 과정에서 기존 변수들의 조합을 통해 새로이 생성된 변수

3) 데이터 품질 전처리

• 데이터 품질 전처리 목적

- 실제 공정에서 발생하는 데이터는 의미 없는 값이거나, 누락 및 오타가 발생하여 품질이 좋지 못하다. 품질이 낮은 데이터를 분석에 이용하면 좋은 결과를 얻기 힘들기 때문에 데이터 전처리는 데이터 분석에 있어서 필수적인 단계이다.
- 데이터 품질 전처리를 위해 아래 6가지의 데이터 품질지수를 파악하고, 데이터의 품질지수를 향상시킨다.

• 데이터 품질지수

- 제조현장에서 수집되는 데이터를 가공 및 AI 분석모델을 적용하기 전에 품질 확인이 선행되어야 한다. 다량의 데이터를 확보하는 것도 중요하지만, 데이터 사이언스 분야에서 쓰이는 표현인 'garbage in, garbage out'이 의미하는 것처럼 결함이 있는 입력

데이터를 이용하면 좋지 않은 결과를 도출할 수 있다. 그러므로 데이터의 품질이 확보되었을 때 AI 분석모델의 우수한 성능을 기대할 수 있다.

- 본 가이드북에서 가공 및 분석을 진행할 섬유 염색공정 데이터의 경우 결측치 제거, 이상치 제거 등의 데이터 전처리를 통해 데이터의 품질을 제고할 수 있다. 데이터 품질 확인을 위해 Gartner에서 제시한 아래의 6종의 데이터 품질 평가지표를 고려하였다.

완전성(Completeness)	필수항목에 누락이 없어야 한다.
유일성(Uniqueness)	데이터 항목은 유일해야 하며 중복되어서는 안 된다.
유효성(Validity)	데이터 항목은 정해진 유효범위 및 도메인을 충족해야 한다.
일관성(Consistency)	데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고, 서로 일치해야 한다.
정확성(Accuracy)	실제 존재하는 객체의 표현 값이 정확하게 반영이 되어야 한다.
무결성(Integrity)	데이터베이스 자료의 오류 없이 변화에 영향을 받지 않고 데이터의 유일성, 유효성, 일관성이 보호되어야 한다.

[표 9] 6가지 데이터 품질지수

• 데이터 품질지수 세부 설명

- 완전성 품질지수

$$\text{완전성 품질지수} = \left(1 - \frac{\text{결측데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 결측 데이터의 값이 30% 이상인 데이터들은 데이터의 완전성을 떨어뜨리기 때문에 각 열(column)에 대한 결측 데이터값의 비율을 확인하여 삭제한다.
- ② 총 결측 데이터의 개수를 확인하기 위해 'isnull()' 함수를 사용하여 각 열(column)에서의 결측 데이터 개수를 구한 뒤 'sum()' 함수를 이용하여 합산한다.
- ③ 앞서 구한 결측 데이터의 개수로부터 위 식에 따라 완전성 품질지수를 구한다.

- 유일성 품질지수

$$\text{유일성 품질지수} = \left(\frac{\text{유일한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터에서 유일한 값을 갖는 열(column)을 찾는다. (EX) 시계열 데이터 : 시간
- ② 유일한 값을 갖는 열(column)에 대해 'groupby()' 함수를 이용하여, 각 열(column)에서의 유일한 데이터 값을 확인하고, 'size()' 함수를 이용하여 그 개수를 구한다.
- ③ 앞서 구한 유일한 데이터의 개수로부터 위 식에 따라 유일성 품질지수를 구한다.



- 유효성 품질지수

$$\text{유효성 품질지수} = \left(\frac{\text{유효성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 'max()', 'min()' 함수를 이용하여 데이터가 유효범위를 벗어나지 않는지를 확인한다.
- ② Pandas의 'info()' 함수로 데이터의 타입이 정해진 형식에 맞는지 확인한다.
- ③ 그 외에 수집된 날짜 안에 들어가 있는지 등을 검증하여 유효성을 만족한 데이터의 개수를 구해 위 식에 따라 유효성 품질지수를 구한다.

- 일관성 품질지수

$$\text{일관성 품질지수} = \left(\frac{\text{일관성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터 테이블 간 종속관계를 확인한다.
- ② Pandas의 'info()' 함수로 데이터의 타입이 정해진 형식에 맞는지 확인한다.
- ③ 그 외에 수집된 날짜 안에 들어가 있는지 등을 검증하여 일관성을 만족한 데이터의 개수를 구해 위 식에 따라 일관성 품질지수를 구한다.

- 정확성 품질지수

$$\text{정확성 품질지수} = 1 - \left(\frac{\text{정확성에 위배되는 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터의 '평균값'이 수집된 데이터에 한하여, 다음과 같이 정확성 품질지수를 확인할 수 있다.
- ② 누락된 데이터가 있는지 확인하기 위해, 'mean()' 함수를 이용하여 데이터들의 평균을 구한다.
- ③ 'mean()' 함수로 구한 데이터의 평균에서 데이터의 '평균값'과의 차이를 구한다.
- ④ 이 차이가 0이 아니면 데이터를 위배하는 데이터가 존재하게 된다.

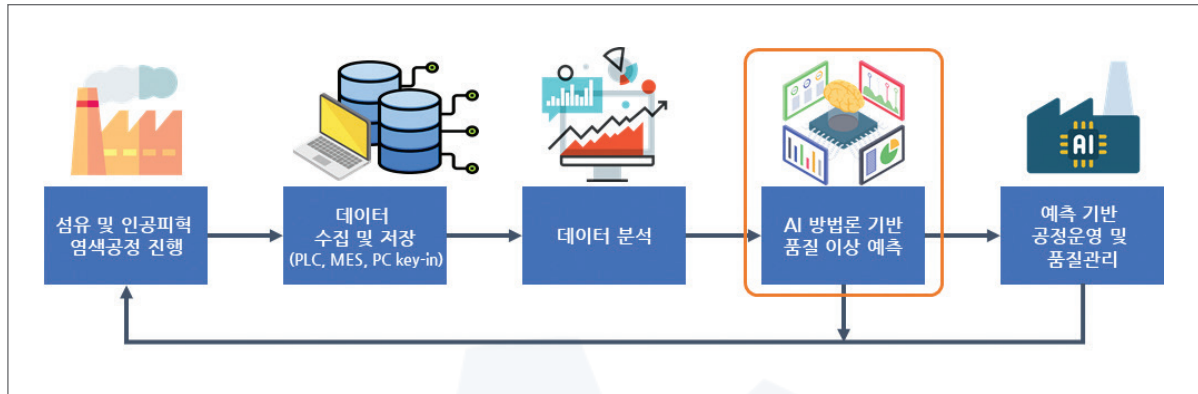
- 무결성 품질지수

$$\text{무결성 품질지수} = \left\{ 1 - \left(\frac{\text{유일성, 유효성, 일관성 중 100%가 아닌 지수}}{3} \right) \right\} \times 100$$

- 무결성 품질지수는 데이터가 '유일성', '유효성', '일관성' 지수를 만족하는지 확인하는 지표이다. 일반적으로 세 가지 지수 중 100% 품질을 담당하는 지수의 비율을 무결성 품질지수로 정의한다.

2.2 분석 모델 소개

1) 데이터 흐름 및 인공지능 모델 적용 흐름도



[그림 5] 염색공정 데이터를 활용한 AI 분석모델 적용 흐름도(프레임워크)

2) AI 분석모델

• 해당 AI 방법론(알고리즘) 선정 이유

- 공정 최적화란?

- 일반적으로 공정 최적화(process optimization)란 제조공정에 필요한 자원을 효과적으로 관리하고, 더 적은 자원을 통해 더 좋은 공정 결과를 만들어 내기 위해 제조상의 문제를 해결하는 것을 의미한다.
- 본 분석에서는 섬유 염색가공 공정의 목표 품질값(염색색차) 달성을 위해 필요한 최적의 염색물량 조건 및 염색설비 가동조건을 도출하고자 한다. AI 모델링을 통해 도출한 최적값을 현장에 적용하여 염색공정 차수를 줄이고, 이에 따라 원재료, 시간, 비용 감소의 효과를 기대할 수 있다.

- 염색공정 품질예측 분석 방법: 랜덤포레스트 회귀모델

- 본 실습에 사용되는 염색공정 제조데이터는 염색물량 데이터("LOT 물량.xlsx"), 염색설비 운영 데이터("PRODUCTION_TREND.csv"), 염색품질 데이터("CCM 측정값.xlsx")로 이뤄져 있다. 염색품질 데이터는 양품/불량 등으로 구분되는 이산형(범주형)의 데이터가 아닌 연속형(수치형) 데이터이며, 해당 값을 기반으로 공정 최적화 분석을 진행할 예정이므로, 회귀모델 구축이 적합해 보인다.
- 회귀모델은 주어진 독립변수를 통해 알고 싶은 종속변수의 값을 예측하는 방법이다. 본 실습에서는 염색물량 데이터와 염색설비 운영데이터가 독립변수이며, 이를 통해 알고자 하는 종속변수가 염색품질 값이다.
- 회귀모델은 크게 선형모델(Linear Regression, Ridge, Lasso 등), 트리 기반 모델(Random Forest Regression, XGBoost Regression 등), 신경망(Neural Network) 모델 등, 복잡도가 낮은 알고리즘부터 높은 알고리즘까지 다양하게 있다. 이 중 본 실습에서는 높은 안정성을 보이며, 과적합도 방지할 수 있는 랜덤포레스트 모델을 선정하였다.



- 랜덤포레스트는 여러 개의 학습모델이 내린 예측 결과를 다수결로 통합하여, 여러 결과로 합치는 앙상블 기법으로 각각의 의사결정나무의 평균을 통해 값을 예측한다.

- 공정 최적화 분석 방법: 유전 알고리즘

- 최적화를 위한 알고리즘은 목표로 하는 최적화 값(일반적으로 최댓값 혹은 최솟값)을 각 탐색 범위 내에서 조절함으로써 주어진 비용함수(손실함수) 값을 최소화 또는 최대화하는 해를 찾아내는 방법을 지칭한다.
- 최적화 알고리즘은 찾고자 하는 해의 요구 수준에 따라 크게 전역 최적화 기법(global optimization method)과 지역 최적화 기법(local optimization method)으로 나눌 수 있다. 전역 최적화 기법은 다소 시간이 걸리더라도 전체 탐색영역에서 가장 좋은 해를 찾는 것을 목표로 하며, 지역 최적화 기법은 단시간에 일부 탐색영역 내에서 가장 좋은 해를 찾는 것을 목표로 한다. 유전 알고리즘은 전역 최적화 기법 중 하나이다.
- 유전 알고리즘은 풀고자 하는 문제에 대한 가능한 해들을 정해진 형태의 자료구조로 표현한 다음 이들을 점차 변형함으로써 점점 더 좋은 해들을 만들어 내는 과정을 거치게 된다. 전체 과정은 크게 Initialize(초기 염색체 생성), Fitness(적합도 계산), Evaluation(평가), Crossover(교차), Mutation(변이), Selection(선별), Termination(종료) 등의 단계로 구성되며, 각 과정은 생물의 진화 과정을 닮았다.
- 유전 알고리즘은 이론적으로 전역 최적점을 찾을 수 있고, 수학적으로 명확하게 정의되지 않은 문제에도 적용할 수 있을 뿐만 아니라, 적합도 평가에 기울기 정보를 필요로 하지 않으므로 함수의 연속성이나 미분가능성 등의 제약을 받지 않는다는 점도 큰 장점으로 작용한다.

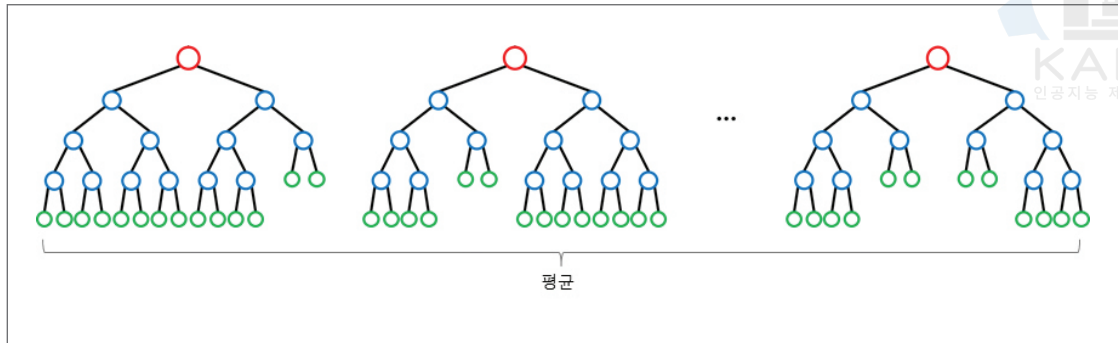
• 적용하고자 하는 AI 분석 방법론(알고리즘)의 구체적 소개

- 랜덤포레스트 회귀(Random Forest Regressor)

- 랜덤포레스트는 결정 트리를 기반으로 하는 알고리즘으로, 여러 개의 결정 트리 분류기가 배깅(bagging)을 기반으로 각자의 데이터를 여러 번 샘플링하여 학습을 수행한 후에 최종적으로 보팅(voting)을 통해 예측 결정을 한다. 여러 개의 학습 모델이 내린 예측 결과를 다수결로 통합하여 여러 결과로 합치는 앙상블 기법이다.

- 랜덤포레스트의 장단점

- 랜덤포레스트는 결정 트리의 쉽고 직관적인 장점을 그대로 가지며, 앙상블 알고리즘 중 비교적 빠른 수행 속도를 가진다는 장점이 있다. 아래 그림과 같이 각각의 의사결정나무의 평균을 통해 값을 예측하기 때문에 의사결정나무로부터 발생하는 과적합을 방지할 수 있다. 또한, 높은 안정성을 보이며, 특성 중요도(feature importance) 분석 기능도 제공한다. 데이터 스케일과 다중 공선성에 대한 영향이 적다는 장점도 있다.



[그림 6] 랜덤포레스트 모델

- 하지만 차원이 높고 희소한 데이터에는 취약하다는 단점이 있으며, 하이퍼파라미터가 많아 튜닝을 위한 시간이 많이 소요된다. 소요되는 시간에 비해 파라미터 최적화를 통한 큰 성능개선을 기대하기는 어렵다.

- 랜덤포레스트 하이퍼파라미터

- 랜덤포레스트는 트리기반의 하이퍼파라미터에 배깅, 부스팅, 학습, 정규화 등을 위한 하이퍼파라미터까지 추가되어 튜닝할 파라미터가 많다.

파라미터 명	설명
n_estimators	- 결정 트리의 개수를 지정 - 계속 증가시킨다고 해서 성능이 무조건 향상되지는 않으며, 연산시간만 증가하므로, 적절히 높은 값이 필요함 - default=10
max_features	- 최적의 분할을 위해 고려할 최대 피쳐(변수) 개수 - 대체로 기본값 사용이 가장 좋음 - default=auto (전체 피쳐 중 $\sqrt{\text{피쳐개수}}$ 만큼 선정)
max_depth	- 트리의 최대 깊이 - 깊이가 깊어지면 과적합 될 수 있으므로 적절히 제어 필요 - default=None (완벽하게 클래스 값이 결정될 때까지 또는 데이터 개수가 min_samples_split보다 작아질 때까지 분할)
min_samples_leaf	- 리프노드가 되기 위해 필요한 최소한의 샘플 데이터수 - min_sample_split과 함께 과적합 제어 용도로 사용됨 - 불균형 데이터의 경우 특정 클래스의 데이터가 극도로 작을 수 있으므로 작게 설정 필요
min_samples_split	- 노드를 분할하기 위한 최소한의 샘플 데이터수 - 과적합 개선을 위해 사용됨 - default=2 (작게 설정할수록 분할 노드가 많아져 과적합 가능성이 증가하므로, 값을 늘려가며 과적합을 해소함)
max_leaf_nodes	- 리프노드의 최대 개수 - default=None

[표 10] 랜덤포레스트 주요 파라미터



- 랜덤포레스트 모델 평가

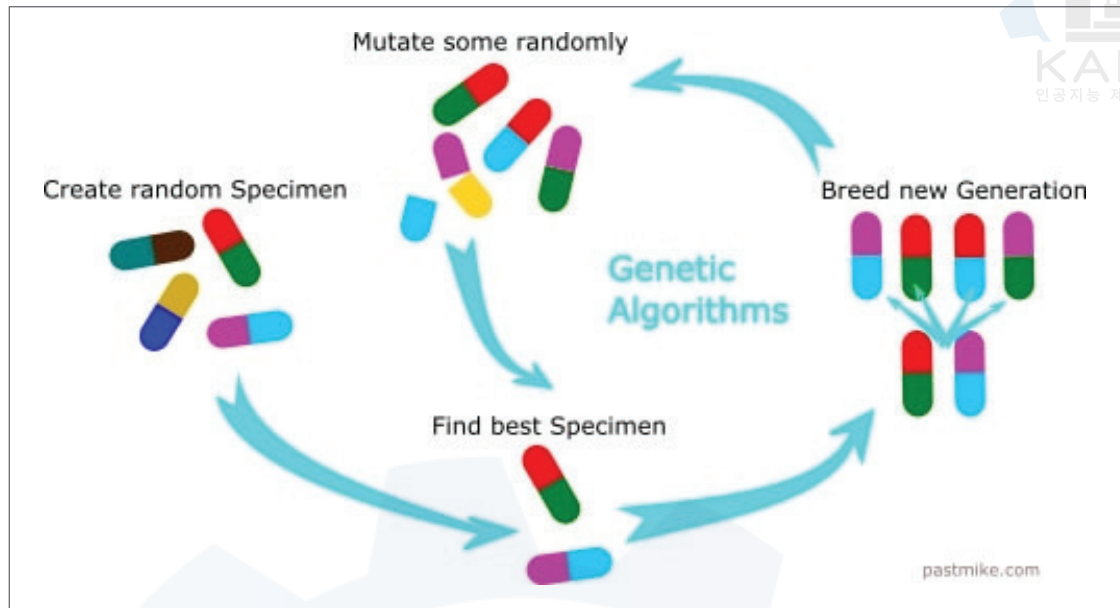
- 머신러닝 모델의 성능은 다양한 방법으로 평가할 수 있다. 회귀모델에 대한 검증은 실제값과 모델의 예측값의 차이에 기반하여 평가한다. 예측값이 실제값과 100% 일치하는 것은 현실적으로 힘들기 때문에 오차를 구해 오차의 허용 범위를 결정하게 된다.
- 다음은 회귀모델에 대한 성능 평가지표들이다. 이 중 본 실습에서는 수정 결정계수(Adjusted R^2), RMSE, MAE를 통해 모델을 평가한다.

평가 지표	설명
MSE	Mean Squared Error : 평균 제곱 오차 예측값과 실제값 간 오차들의 제곱 평균. outlier에 민감함 $MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$
RMSE	Root Mean Squared Error : 평균 제곱근 오차 MSE에 루트를 씌운 값. 오류지표를 실제값과 유사한 단위로 다시 변환하기 때문에 해석이 용이함 $RMSE = \sqrt{\frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{n}}$
MAE	Mean Absolute Error : 평균 절대 오차 실제값과 예측값 간 오차들의 절댓값 평균. 변동성이 큰 지표와 낮은 지표를 같이 예측할 때 유용함. MSE보다 특이치에 덜 민감함. $MAE = \frac{1}{N} \sum_{i=1}^N y_i - \hat{y}_i $
R^2	Coefficient of Determination : 결정계수 피어슨 상관계수의 제곱. 회귀모델의 설명력을 표현하는 지표로 1에 가까울수록 높은 성능을 나타냄 $R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$
Adjusted R^2	Adjusted Coefficient of Determination : 수정 결정계수 결정계수에 표본의 크기와 독립변수의 수를 고려하여 계산한 지표. 다중회귀분석을 수행하는 경우에 활용 $Adjusted R^2 = 1 - \frac{(1 - R^2)(N - 1)}{N - P - 1}$

[표 11] 회귀모델 주요 성능 평가지표

- 유전 알고리즘(Genetic Algorithm)

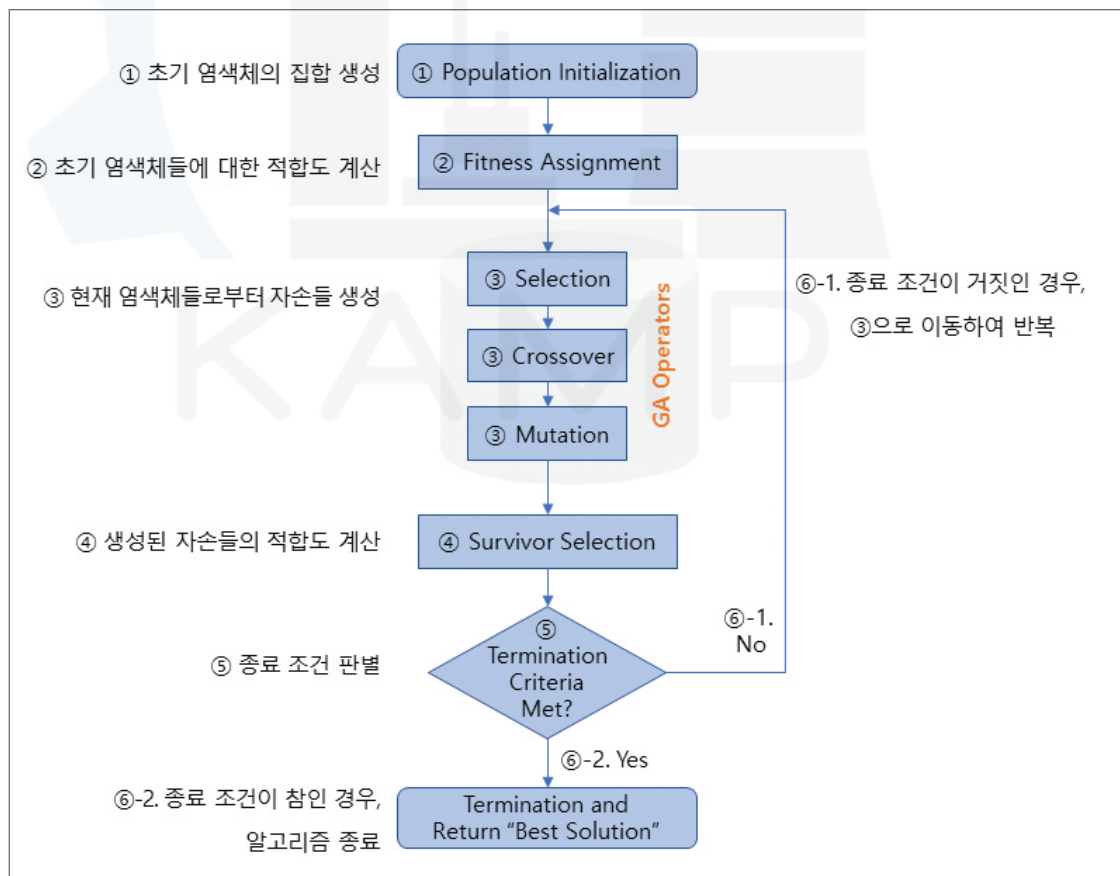
- 유전 알고리즘(GA)은 자연계의 생물 유전학에 기본 이론을 두고, 생물의 진화를 모방한 전역 최적화 진화 연산 기법이다. 특정한 문제를 풀기 위한 알고리즘이라기보다는 최적화 문제를 풀기 위한 방법론에 가까우며, GA에서 사용할 수 있는 형식으로 데이터를 바꿔주지만 모든 문제에 대해서 적용할 수 있다.



[그림 7] 유전 알고리즘 기본 개념도

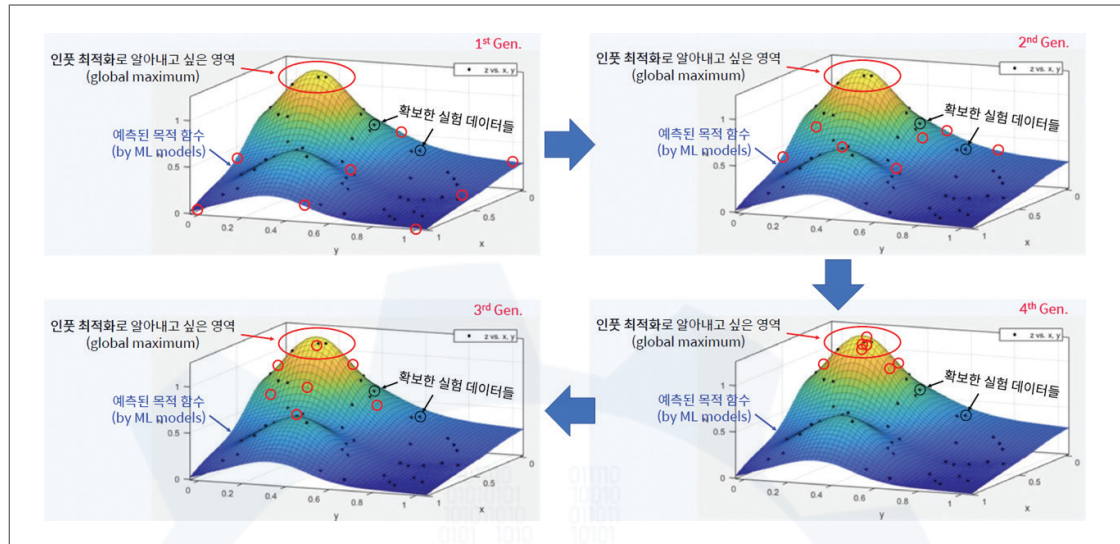
*출처: <https://pastmike.com/what-is-a-genetic-algorithm/>

- 유전 알고리즘은 random한 초기 염색체를 생성하고, 이 중 자연 선택된 염색체들끼리 교차 (Crossover) 및 변이(Mutation) 연산을 하여 자식세대를 생성한다. 이 과정을 반복하면서 점차 최적의 해를 찾아가는 방식으로 작동한다.



[그림 8] 유전 알고리즘 작동 원리

- 교차연산을 통해 더 나은 해 집합을 생성하고, 세대가 진행될수록 해 집합은 전역해에 다가가게 된다. 아래 그림에서 1세대에서 4세대로 진행하면서 해 집합(빨간색 작은 원)이 전역해(빨간색 큰 원)에 가까워짐을 볼 수 있다. 전역해에 근접한 실험 데이터(검은색 점)가 많을수록 목표값에 가깝게 예측이 가능하다.



[그림 9] 유전 알고리즘 세대 진행 도식화

- 유전 알고리즘을 통한 최적화는 데이터 분포가 미치는 영향이 적다는 장점이 있다. 또한, 목적함수의 연속성, 미분 가능성과도 무관하여 수학적으로 명확하게 정의되지 않은 문제에도 적용할 수 있어, 매우 유용하게 활용이 가능하다.
- 유전 알고리즘 하이퍼파라미터
- 유전 알고리즘 파라미터를 통해 진화횟수, 초기 세대 개체수, 세대 변이율, 세대 교차율 등을 조정할 수 있다.



파라미터 명	설명
max_num_iteration	- 진화(연산) 횟수. 즉, 최대 세대수
population_size	- 초기 세대(generation)의 개체수(population) - 많을수록 결과는 좋으나 연산로드가 있음
mutation_probability	- 다음 세대의 몇 %를 변이로 채울 것인지 결정
elit_ratio	- 이전 세대의 가장 뛰어난 해의 몇 %를 가져올 것인지 결정
crossover_probability	- 가져온 이전 세대와 현 세대를 몇 % 교차할 것인지 결정
parents_portion	- 자식 세대가 부모세대로부터 얼마의 %만큼 가져오는지 결정
crossover_type	- 교차변이의 종류 • uniform : 균일 교차 변이. 부모의 변수가 랜덤하고 균일하게 자손 변수들에게 유전됨 • single_point : 한 점(single point) 교차 변이. 교차지점을 임의로 설정한 뒤, 교차지점 앞쪽에서는 부모1의 유전자를, 뒤쪽에서는 부모2의 유전자를 가져옴 • two_points: 두 점(two points) 교차 변이. 두 개의 교차지점에서 부모1과 부모2의 유전자가 교차됨
max_iteration_without_improv	- 지정한 횟수만큼 연산이 없으면 정지 • None : 정지하지 않음

[표 12] 유전 알고리즘 주요 파라미터

• AI 분석 방법론(알고리즘) 구축 절차 설명

- 데이터 전처리

- 염색공정 제조데이터를 활용하여 AI 분석모델을 구축하기 위해서는 분석에 적합한 형태로 데이터셋을 변환하는 것이 필요하다. 염색공정에서 수집된 3개의 데이터셋에 대해 정제 및 전처리를 진행하며, 이 과정에서 결측치 제거, 중복치 제거, 파생변수 생성 등이 수행된다.
- 세 개의 데이터셋 중, 염색공정 설비데이터('PRODUCTION_TREND.csv')는 시계열 데이터인 반면, 품질데이터('CCM 측정값.xlsx') 및 물량데이터('LOT 물량.xlsx')는 LOT번호별로 구분된 횡단면 데이터이다. 따라서 3개의 데이터를 연결하기 위해서 데이터를 2차 가공해야 한다. 각기 다른 차원의 데이터를 연결하는 방법은 <그림 10>과 같이 시계열 데이터를 압축하거나, 횡단면 데이터를 확장하는 방법이 있다. 본 실습에서는 데이터양을 보존하기 위해 횡단면 데이터를 시계열에 맞춰 확장하는 방식으로 진행한다.

[방법①] 시계열 데이터를 횡단면 데이터에 맞춰서 압축

SET2: 시계열 설비데이터

LOT번호	공정시간	설비변수1	설비변수2
F2208110087	1	14.5	0
F2208110087	2	16.1	0
F2208110087	3	19.6	111
F2208110087	4	20	111
F2208110087	5	19.9	111
F2208110087	6	19.9	111
F2208110087	7	19.8	111
F2208110087	8	19.8	111
F2208110087	9	18	111
F2208110087	10	23	87

압축

SET2: 통계압축 설비데이터 (예시)

LOT번호	설비변수1 평균	설비변수1 표준편차	설비변수1 Q1	설비변수1 Q3
F2208110087				

SET1: LOT 물량

LOT번호	물량변수1	물량변수2
F2208110087	245.6	4000

SET3: CCM 검사결과

LOT번호	품질변수1	품질변수2
F2208110087	-2.08697	-0.3912598

[방법②] 횡단면 데이터를 시계열 데이터에 맞춰서 확장

SET2: 시계열 설비데이터

LOT번호	공정시간	설비변수1	설비변수2
F2208110087	1	14.5	0
F2208110087	2	16.1	0
F2208110087	3	19.6	111
F2208110087	4	20	111
F2208110087	5	19.9	111
F2208110087	6	19.9	111
F2208110087	7	19.8	111
F2208110087	8	19.8	111
F2208110087	9	18	111
F2208110087	10	23	87

SET1,3: LOT물량, CCM 검사결과

LOT번호	물량변수	품질변수
F2208110087	245.6	-0.3912598
F2208110087	245.6	-0.3912598
F2208110087	245.6	-0.3912598
F2208110087	245.6	-0.3912598
F2208110087	245.6	-0.3912598
F2208110087	245.6	-0.3912598
F2208110087	245.6	-0.3912598
F2208110087	245.6	-0.3912598
F2208110087	245.6	-0.3912598
F2208110087	245.6	-0.3912598

확장

SET1: LOT 물량

LOT번호	물량변수1	물량변수2
F2208110087	245.6	4000

SET3: CCM 검사결과

LOT번호	품질변수1	품질변수2
F2208110087	-2.08697	-0.3912598

[그림 10] 시계열 데이터와 비시계열 데이터 병합 방법 모식도 - ①

- 다만 후자의 방식을 채택하는 경우, “시계열”로 진행되는 설비데이터에 “최종” 품질데이터가 매칭되는 것이기 때문에 이 둘을 논리적으로 연결해줄 수 있는 다리 역할의 변수가 필요하게 된다. 예를 들어, 1분부터 10분까지 1분 단위로 수집되는 설비데이터에 10분 쯤에 측정한 품질검사 값을 매칭하게 되는 것인데, 10분 쯤의 데이터는 해석에 문제가 없으나, 1분~9분 사이의 설비데이터에 10분 쯤의 품질데이터를 매칭하는 것이 문제가 된다. 10분 쯤의 검사결과 값은 1분~10분 동안 진행된 공정의 영향이 축적된 결과이기 때문에 동일하게 매칭 할 수는 없다.
- 이를 해결하기 위해 본 분석에서는 가장 간단한 방법으로 '총 공정시간 대비 진행시간(%)'이란 변수를 추가하여 두 데이터의 연결에 설명 논리를 부여하고자 한다. 위의 예로 다시 돌아가 보면, 공정진행시간 10%부터 100%까지의 설비데이터에 품질데이터를 매칭하게 되며, 10%에 매칭된 최종 품질데이터에 대해서는, ‘남은 공정시간 90%가 지나면 매칭된 최종 품질검사 값이 나올 것이다’라고 해석이 가능해진다. 완벽한 해결책은 아니겠지만 비교적 단순한 파생변수 하나만 생성하면 되며, 상대적으로 이해하기도 쉽다.



[방법②] 횡단면 데이터를 시계열 데이터에 맞춰서 확장 + 파생변수로 연결

SET2: 시계열 설비데이터				파생변수		SET1,3: LOT물량, CCM 검사결과		
LOT번호	공정시간	설비변수1	설비변수2	공정시간(%)		LOT번호	물량변수	품질변수
F2208110087	1	14.5	0	10%	✓	F2208110087	245.6	-0.3912598
F2208110087	2	16.1	0	20%	✓	F2208110087	245.6	-0.3912598
F2208110087	3	19.6	111	30%	✓	F2208110087	245.6	-0.3912598
F2208110087	4	20	111	40%	✓	F2208110087	245.6	-0.3912598
F2208110087	5	19.9	111	50%	✓	F2208110087	245.6	-0.3912598
F2208110087	6	19.9	111	60%	✓	F2208110087	245.6	-0.3912598
F2208110087	7	19.8	111	70%	✓	F2208110087	245.6	-0.3912598
F2208110087	8	19.8	111	80%	✓	F2208110087	245.6	-0.3912598
F2208110087	9	18	111	90%	✓	F2208110087	245.6	-0.3912598
F2208110087	10	23	87	100%	✓	F2208110087	245.6	-0.3912598

[그림 11] 시계열 데이터와 비시계열 데이터 병합 방법 모식도 - ②

- 참고로, 앞서 언급된 시계열 설비데이터를 횡단면으로 압축하는 방법으로는 통계량 압축방법이 있다. 이 방법은 공정진행시간 동안의 공정의 영향을 압축적으로 담아내기에는 공정시간 파생변수를 생성하는 방법보다는 더 적합하겠으나, 기본적으로 하나의 변수당 다량의 변수가 만들어지기 때문에(예: 평균값, 표준편차값, Q1, Q3 등) 본 실습과 같이 데이터양에 비해 변수의 개수가 많은 데이터에 대한 분석으로는 적합하지 않은 것으로 판단된다. (실제로 통계 압축 데이터로 사전 분석을 진행한 결과, 모델 성능이 매우 나빴다.)
- 따라서 <그림 11>의 [방법 ②]와 같이 횡단면 데이터를 시계열 데이터에 맞춰 확장하는 방식으로 3개의 데이터셋을 하나로 결합한 뒤, 이상치 제거, 불필요한 열 제거 등의 2차 가공과 데이터 시각화를 진행한다.

- 변수 선별

- 병합 및 최종 전처리가 완료된 데이터셋에서 분석에 사용될 최종 변수를 선별한다. 유일값이 전체 데이터양에 비해 너무 적은 변수는 제거하고, 독립변수 간의 상관관계가 깊은 변수는 다중공선성 이슈로 제외한다. 이 외에는 전체적인 변수 중요도를 살펴보기 위해 모두 모델에 포함하여 진행한다.

- 데이터셋 분리 및 스케일링

- 최종 데이터셋에서 독립변수와 종속변수 데이터를 각각 추출한 후, 학습 및 평가 데이터셋으로 분리한다. 학습 데이터는 GridSearchCV를 통한 모델 학습 과정에서 학습 및 검증 데이터로 다시 분리된다. 독립변수가 정규분포(평균 0, 분산 1)를 따르도록 데이터 표준화를 진행한다.



- 랜덤포레스트 회귀 모델 구축 및 최적화

- GridSearchCV를 통해 랜덤포레스트 모델의 최적 파라미터를 찾는 하이퍼파라미터 튜닝 작업을 수행한다. 최적의 파라미터를 통해 모델 최적화를 진행한다.

- 모델 평가 및 변수 중요도 분석

- 수정 결정계수, RMSE, MAE 등의 평가지표를 활용하여 모델의 성능을 평가한다. SHAP 분석을 통해 모델에 대한 각 변수의 영향력(크기 및 방향)을 살펴본다.

- 유전 알고리즘 적용 및 공정 최적값 도출

- 유전 알고리즘을 품질예측 모델에 적용하여 목표 품질값(염색색차)을 달성하기 위한 각 공정변수의 최적값을 도출한다.

2.3 분석 체험

1) 필요 SW, 패키지 설치 방법 및 절차 가이드

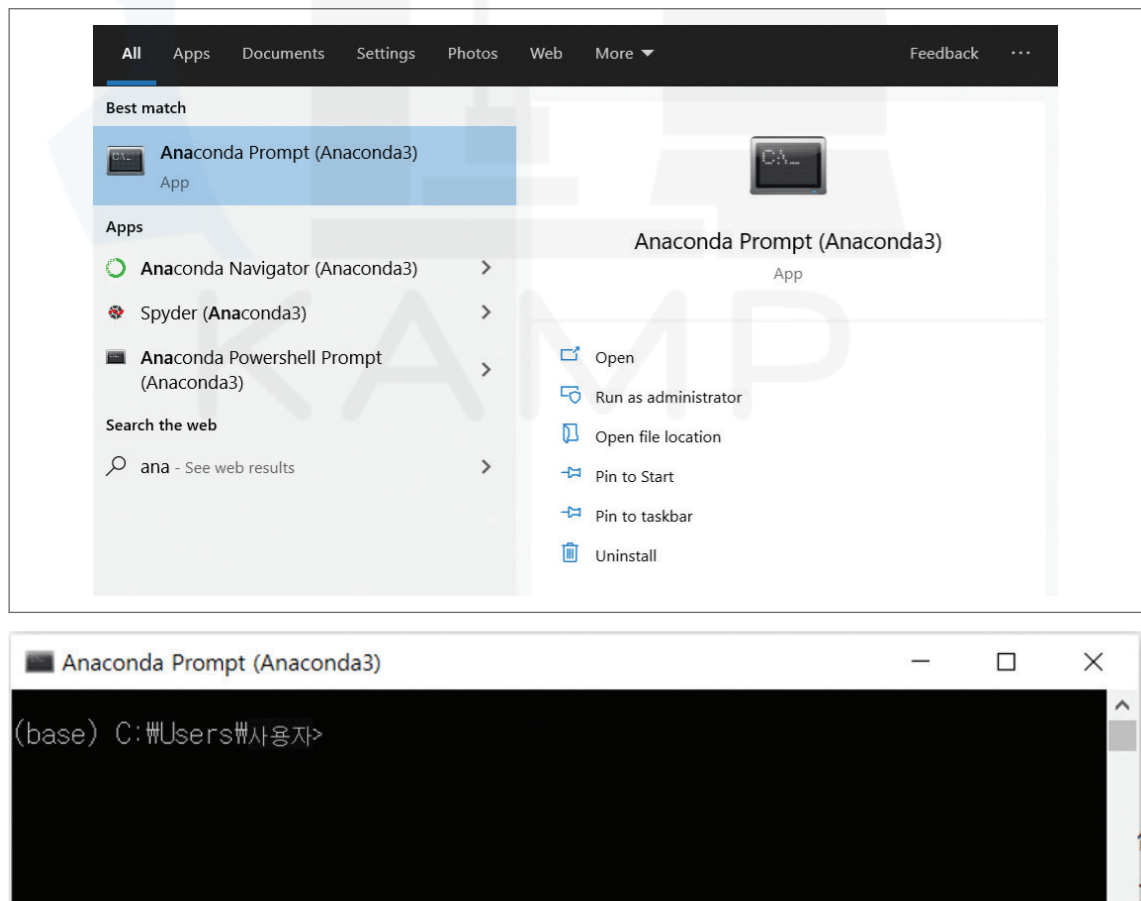
• 필요 SW 설치

- 프로그래밍 언어인 파이썬(Python)을 사용하며 개발 환경으로는 주피터 노트북(Jupyter Notebook)을 활용한다. 관련 SW 설치는 부록 3 [분석 환경 구축을 위한 설치 가이드]를 참고하면 된다.

• 가상환경 구축 가이드

- 부록 3 [분석환경 구축을 위한 설치 가이드]에서 설치한 아나콘다(Anaconda)에서 파이썬의 독립적인 가상의 작업 환경을 구축한다. 파이썬의 모듈은 다양한 버전이 존재하기 때문에 충돌이 일어나기 쉬우며, 이러한 충돌을 방지하기 위해 실습마다 독립적인 가상환경 구축을 권장한다. 가상환경을 사용하기 위해서는 환경을 구축하고 실행(활성화)해야 하고, 실습이 끝난 이후에는 환경을 비활성화 시켜야 한다. 주피터 노트북에서 가상환경을 실행하는 구체적인 순서 및 실행 방법은 아래와 같다.

- 아나콘다 프롬프트 실행



- 가상환경 생성 : 파이썬 버전 정보와 가상환경 이름을 입력하여 아래와 같이 가상환경 생성 작업을 시작한다. 파이썬 버전 정보는 “python --version” 코드를 통해 확인할 수 있다.

```
python --version
```

```
# conda create -n 가상환경이름 python=버전
conda create -n KAMP python=3.8.8
```

```

Anaconda Prompt (Anaconda3)
(base) C:\Users\사용자>conda create -n KAMP python=3.8.8
Collecting package metadata (current_repodata.json): done
Solving environment: failed with repodata from current_repodata.json, will retry with next repodata source.
Collecting package metadata (repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 4.10.1
  latest version: 4.10.3

Please update conda by running

  $ conda update -n base -c defaults conda
  
```

```
...
Proceed ([y]/n)? y # 해당 문구가 뜨면 'y' 를 입력한다.
```

```

...
done
##
## To activate this environment, use
##
##     $ conda activate KAMP
##
## To deactivate an active environment, use
##
##     $ conda deactivate
##
(base) C:\Users\사용자>
  
```

- 가상환경 실행 : 가상 환경이 활성화되면 아래 그림처럼 명령 프롬프트 앞에 가상 환경 이름이 표시된다.

```
# conda activate 가상환경이름
conda activate KAMP
```

```
(base) C:\Users\사용자>conda activate KAMP
(KAMP) C:\Users\사용자>
```

- 분석이 끝난 후에는 가상환경을 종료 한다.

```
conda deactivate
```

```
(KAMP) C:\Users\사용자>conda deactivate
(base) C:\Users\사용자>
```

- 주피터 노트북에서 가상환경 실행 : 주피터 노트북에서 가상 환경을 실행하기 위해 가상 환경 비활성화(종료) 상태에서 아래의 코드를 작성한다. 주피터 노트북 실행창에서 빨간 박스처럼 생성한 가상 환경이 있는 것을 확인할 수 있다. 해당 가상 환경을 선택 후 실습을 진행한다.

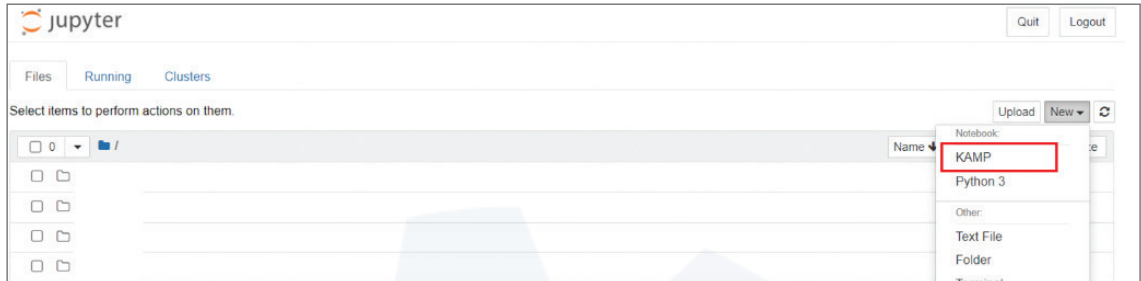


```

pip install ipykernel

# python -m ipykernel install --user --name 가상환경이름 --display-name "가상환경이름"
python -m ipykernel install --user --name KAMP --display-name "KAMP"

# jupyter notebook 실행
jupyter notebook
    
```



• 패키지 설치 가이드

- 패키지 설치 전 파이썬 관련 용어 정리

모듈 (module)	- 함수나 변수 또는 클래스를 모아 놓은 파일 - 파이썬 확장자(.py) 파일에 특정 함수, 변수, 클래스 등이 저장된 코드이다.
패키지 (package)	- 여러 모듈의 집합 - 하나의 디렉토리(경로)에 놓인 모듈들의 집합을 의미하며, 일반적으로 __init__.py라는 패키지 초기화 파일이 존재한다. - 예) NumPy : 수치해석 패키지, pandas : 데이터 분석 패키지
라이브러리 (library)	- 여러 패키지의 집합 - 파이썬을 설치할 때 기본적으로 설치되는 라이브러리를 표준라이브러리라고 한다. 파이썬 공식이 아닌 외부에서 개발한 모듈과 패키지를 묶어 외부 라이브러리라고 한다. - 예) matplotlib : 데이터 시각화를 생성하기 위한 표준 라이브러리
프레임워크 (framework)	- 라이브러리의 집합 - 목적에 맞게 코드를 작성할 수 있는 기본 아키텍처이며, 필수 구성 요소를 제공한다.

[표 13] 파이썬 용어 정리

• 패키지 설치 방법

- 분석환경인 주피터 노트북 내에서 데이터 분석 및 AI 분석모델 적용을 위한 패키지를 설치한다. pip install <패키지 이름>을 입력하여 원하는 라이브러리(혹은 패키지)를 설치할 수 있다.

```

!pip install pandas
!pip install numpy matplotlib scikit-learn tensorflow seaborn
    
```


- 이미 설치되어 있는 패키지 외에 신규 설치가 필요한 패키지들이 자동으로 설치된다.

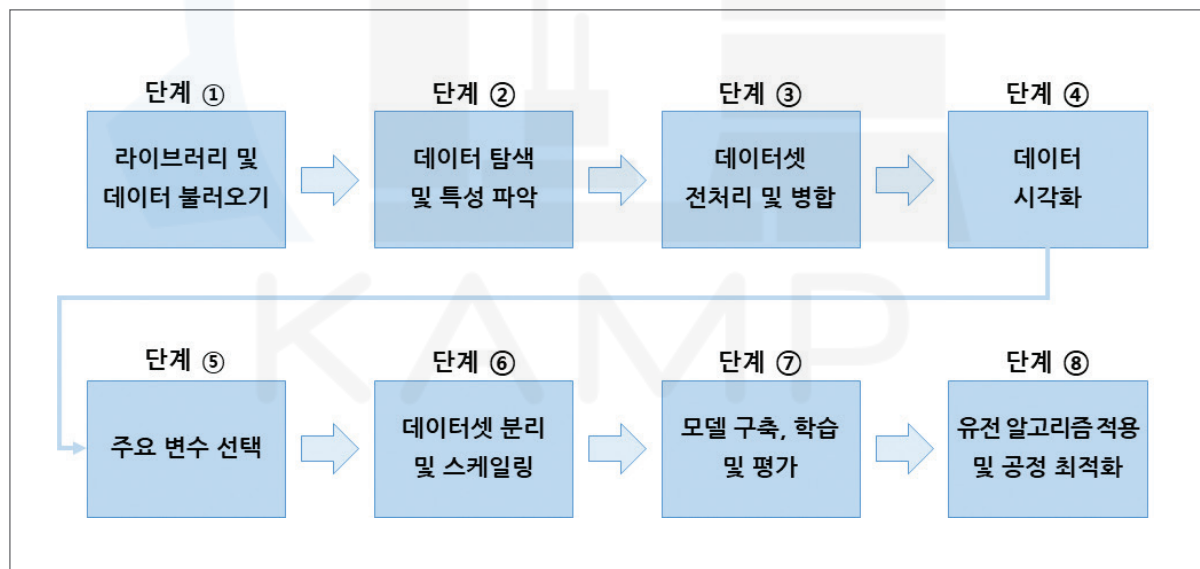
```
Requirement already satisfied: pandas in /home/ubuntu/anaconda3/lib/python3.6/site-packages (0.23.0)
Requirement already satisfied: python-dateutil>=2.5.0 in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from pandas) (2.7.3)
Requirement already satisfied: pytz>=2011k in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from pandas) (2018.4)
Requirement already satisfied: numpy>=1.9.0 in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from pandas) (1.19.5)
Requirement already satisfied: six>=1.5 in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from python-dateutil>=2.5.0->pandas) (1.15.0)
```

- 아래 코드를 통해 설치된 패키지들의 전체 목록을 확인할 수 있다.

```
!pip list
```

Package	Version
abs1-py	0.14.1
alabaster	0.7.10
alembic	1.4.1
anaconda-client	1.6.14
anaconda-navigator	1.8.7
anaconda-project	0.8.2

2) 분석 단계별 프로세스 - Flow Chart



[그림 12] AI 분석 단계별 흐름도



[단계 ①] 라이브러리 및 데이터 불러오기

①-1. 패키지 불러오기

- 'import <패키지 이름>', 혹은 'from <패키지 이름> import <하위 모듈 이름>'을 사용하여 전체 패키지를 불러오거나 특정 패키지 안의 모듈만 불러오기가 가능하다. 또한 'import <패키지 이름> as <약자>'를 사용하여 추후 패키지 사용 시 약자(alias)로 이름을 대신할 수 있다.

```
import pandas as pd
import numpy as np
```

[코드 1] 필요 패키지 불러오기

①-2. 데이터 불러오기

- 분석에 사용할 데이터를 분석 코드와 같은 폴더 내에 저장 후 사용한다. 본 분석에서는 'LOT 물량.xlsx', 'PRODUCTION_TREND.csv', 'CCM 측정값.xlsx' 등 3개의 데이터를 사용한다.
- 'pandas' 패키지 내의 'read_excel()' 함수를 활용하여 EXCEL(.xls 또는 .xlsx) 형식의 데이터를, 'read_csv()' 함수를 활용하여 CSV(.csv) 형식의 데이터를 불러올 수 있다. CSV 파일은 한글 변수로 인해 utf-8 인코딩 에러가 발생할 수 있으니, 파라미터에 encoding='CP949' 옵션을 추가하여 문제를 해결한다.

```
# SET1: LOT 물량 정보
workload = pd.read_excel('LOT 물량.xlsx')
# SET2: 시계열 설비데이터
operation = pd.read_csv('PRODUCTION_TREND.csv', encoding='CP949')
# SET3: CCM 검사결과
ccm = pd.read_excel('CCM 측정값.xlsx')
```

[코드 2] 데이터 불러오기

①-3. 참고사항 - 경고메시지 미 노출

- 분석을 진행하다 보면 다양한 경고메시지를 만나게 된다. 아마 위 [코드 2]를 통해 데이터를 불러올 때도 경고 메시지가 화면에 출력되었을 것이다. 경고 메시지는 결과 도출에는 영향을 미치지 않으나, 편의를 위해 경고 메시지가 노출되지 않도록 설정하는 것을 추천한다.
- 'warnings' 패키지의 'filterwarnings()' 함수를 활용하고, 파라미터 'action'을 'ignore'로 설정하면 경고메시지를 노출되지 않게 할 수 있다. 혹 경고메시지를 다시 활성화하고 싶으면 'ignore' 대신 'default'를 사용하면 된다.

```
import warnings
warnings.filterwarnings(action='ignore')
```

[코드 3] 경고메시지 숨기기



①-4. 데이터 확인

- 'head()' 함수를 활용하여 불러온 데이터의 상단 5개 행의 내용을 확인할 수 있다.

SET1: LOT 물량 데이터 확인
workload.head()

	EXT1_QTY(투입중량 (KG))	EXT2_QTY(액량 (LITER))	PRODT_ORDER_NO	JOB_CD	PROCESS_CD	단위중량	염색 가동 길이
0	245.6	4000	F2111300008	F12-1203	F12-1203	590	120.0
1	71.4	600	F2111300012	F12-1203	F12-1203	445	123.0
2	98.3	4000	F2111300020	F12-1203	F12-1203	0	460.0
3	97.0	4000	F2111300021	F12-1203	F12-1203	0	452.0
4	40.6	600	F2111300027	F12-1203	F12-1203	445	86.0

SET2: 염색공정 시계열 설비데이터 확인
operation.head()

	LOT_NO	WC_CD	WC_CNT	SEQ_NO	PGM_ID	RESOURCE_CD	CR_TEMP	TRD_TEMP_SP	TRD_TEMP_PV	TRD_SPEED1	TRD_SPEED2	TRD_SPEED3
0	F2112280005	F12-1203	1	1	B9700	FCM04	30	30.0	14.5	0	0	
1	F2112280005	F12-1203	1	2	B9700	FCM04	30	30.0	16.1	0	0	
2	F2112280005	F12-1203	1	3	B9700	FCM04	30	30.0	19.6	111	1	11
3	F2112280005	F12-1203	1	4	B9700	FCM04	30	30.0	20.0	111	1	11
4	F2112280005	F12-1203	1	5	B9700	FCM04	30	30.0	19.9	111	1	11

SET3: CCM 검사결과 확인
ccm.head()

	lot_no	seq	oper_id	염색 색차 DL	염색 색차 DA	염색 색차 DB	염색 색차 DC	염색 색차 DH	염색 색차 DE
0	F2002240067	0	염색	1.402431	-0.079870	0.255513	0.063653	-0.260028	1.427754
1	F2003120058	0	염색	-0.043186	-0.021964	0.614762	0.032673	-0.614286	0.616669
2	F2006160123	0	원단검사	12.724530	-1.963228	-1.810259	1.940723	-1.834361	13.001730
3	F2009030093	0	원단검사	0.243153	0.251606	-3.862172	3.830362	-0.554988	3.877990
4	F2009030093	0	원단검사	0.673786	0.199348	-3.894216	3.854515	-0.589382	3.957100

[코드 4] 불러온 데이터의 상단 5행 확인



[단계 ②] 데이터 탐색 및 특성 파악

②-1. 데이터 기본정보 확인

- 분석데이터의 크기, 변수명 등의 기초 정보를 확인한다. 'info()' 함수를 사용하여 데이터의 개수 및 용량, 컬럼정보, 데이터 타입, 결측치 개수 등의 다양한 정보를 확인할 수 있다.

```
workload.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2150 entries, 0 to 2149
Data columns (total 7 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                -
0   EXT1_QTY(투입중량 (KG))              2150 non-null   float64
1   EXT2_QTY (액량 (LITER))              2150 non-null   int64
2   PRODT_ORDER_NO                      2150 non-null   object
3   JOB_CD                              2150 non-null   object
4   PROCESS_CD                          2150 non-null   object
5   단위중량                            2150 non-null   int64
6   염색 가동 길이                      2150 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 117.7+ KB
```

[코드 5] 데이터셋#1 데이터의 기본정보 확인

- 확인한 정보를 바탕으로, 데이터 탐색을 더 진행하기 전에 분석에 필요한 변수를 먼저 추출한다. '[[]]' 안에 추출하고자 하는 컬럼명을 나열하면, 해당 열만 따로 추출할 수 있다. 이때 원하는 순서대로 나열하여 컬럼 순서 변경도 동시에 진행할 수 있다.

```
workload = workload[['PRODT_ORDER_NO', 'JOB_CD', 'EXT1_QTY(투입중량 (KG))', 'EXT2_QTY (액량 (LITER))', '단위중량', '염색 가동 길이']]
```

[코드 6] 데이터셋#1 필요 변수만 추출 및 변수 순서 변경

- 컬럼명 변경은 [코드 7]과 같이 데이터의 컬럼명 재정의를 통해 진행할 수 있으며, 변경된 내용은 'head()' 함수를 사용하여 확인할 수 있다. 괄호 안에 확인하고자 하는 상단 행의 개수를 지정할 수 있다.

```
workload.columns = ['LOT번호', '공정코드', '투입중량(kg)', '투입액량(L)', '단위중량(kg)', '염색길이(m)']
```

[코드 7] 데이터셋#1 컬럼명 변경



```
workload.head(1)
```

	LOT번호	공정코드	투입중량(kg)	투입액량(L)	단위중량(kg)	염색길이(m)
0	F2111300008	F12-1203	245.6	4000	590	120.0

[코드 8] 데이터셋#1 변경 내용 확인

- 데이터셋#2 시계열 설비데이터와 데이터셋#3 CCM 데이터도 위와 같은 방식으로 데이터 기본정보를 확인하고, 변수 추출 및 변수명 변경을 진행한다.

```
# 데이터 기본정보 확인
operation.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2119545 entries, 0 to 2119544
Data columns (total 15 columns):
#   Column                Dtype
---  -
0   LOT_NO                object
1   WC_CD                 object
2   WC_CNT                int64
3   SEQ_NO                int64
4   PGM_ID                object
5   RESOURCE_CD           object
6   CR_TEMP               int64
7   TRD_TEMP_SP           float64
8   TRD_TEMP_PV           float64
9   TRD_SPEED1            int64
10  TRD_SPEED2             int64
11  TRD_SPEED3             int64
12  TRD_SPEED4             int64
13  INSRT_DT               object
14  PRODUCTION_RESULT_ID  object
dtypes: float64(2), int64(7), object(6)
memory usage: 242.6+ MB
```

```
# 데이터 탐색의 편의를 위해 필요한 변수만 추출 및 변수 순서 변경
```

```
operation = operation[['LOT_NO', 'WC_CD', 'RESOURCE_CD', 'INSRT_DT', 'SEQ_NO', 'CR_TEMP', 'TRD_TEMP_SP', 'TRD_TEMP_PV', 'TRD_SPEED1', 'TRD_SPEED2', 'TRD_SPEED3', 'TRD_SPEED4']]
```

```
# 변수명 변경
```

```
operation.columns = ['LOT번호', '공정코드', '설비번호', '공정일시', '공정진행시간', '목표온도', '지시온도', '진행온도', '포속1', '포속2', '포속3', '포속4']
```

```
# 데이터 변경 내용 확인
```

```
operation.head(1)
```

	LOT번호	공정코드	설비번호	공정일시	공정진행시간	목표온도	지시온도	진행온도	포속1	포속2	포속3	포속4
0	F2112280005	F12-1203	FCM04	2022-01-03	1	30	30.0	14.5	0	0	0	0

[코드 9] 데이터셋#2 기본정보 확인, 필요 변수 추출 및 변수명 변경

```
# 데이터 기본정보 확인
ccm.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   lot_no      614 non-null    object
1   seq         614 non-null    object
2   oper_id     614 non-null    object
3   염색 색차 DL 614 non-null    float64
4   염색 색차 DA 614 non-null    float64
5   염색 색차 DB 614 non-null    float64
6   염색 색차 DC 614 non-null    float64
7   염색 색차 DH 614 non-null    float64
8   염색 색차 DE 614 non-null    float64
dtypes: float64(6), object(3)
memory usage: 43.3+ KB
```

```
# 필요한 변수만 추출 및 변수 순서 변경
ccm = ccm[['lot_no', 'seq', 'oper_id', '염색 색차 DE']]
```

```
# 변수명 변경
ccm.columns = ['LOT번호', '검사차수', '작업명', '염색색차 DE']
```

```
# 데이터 변경 내용 확인
ccm.head(1)
```

	LOT번호	검사차수	작업명	염색색차 DE
0	F2002240067	0	염색	1.427754

[코드 10] 데이터셋#3 기본정보 확인, 필요 변수 추출 및 변수명 변경

No	데이터셋	변경전 컬럼명	변경후 컬럼명
1	데이터셋#1 'LOT 물량'	EXT1_QTY(투입중량 (KG))	투입중량(kg)
2		EXT2_QTY (액량 (LITER))	투입액량(L)
3		PRODT_ORDER_NO	LOT번호
4		JOB_CD	공정코드
5		PROCESS_CD	*삭제
6		단위중량	단위중량(kg)
7		염색 가동 길이	염색길이(m)
1	데이터셋#2 'PRODUCTION_TREND'	LOT_NO	LOT번호
2		WC_CD	공정코드
3		WC_CNT	*삭제
4		SEQ_NO	공정진행시간
5		PGM_ID	*삭제
6		RESOURCE_CD	설비번호
7		CR_TEMP	목표온도
8		TRD_TEMP_SP	지시온도
9		TRD_TEMP_PV	진행온도
10		TRD_SPEED1	포속1
11		TRD_SPEED2	포속2
12		TRD_SPEED3	포속3
13		TRD_SPEED4	포속4
14		INSRT_DT	공정일시
15		PRODUCTION_RESULT_iD	*삭제
1	데이터셋#3 'CCM 측정값'	lot_no	LOT번호
2		seq	검사차수
3		oper_id	작업명
4		염색 색차 DL	*삭제
5		염색 색차 DA	*삭제
6		염색 색차 DB	*삭제
7		염색 색차 DC	*삭제
8		염색 색차 DH	*삭제
9		염색 색차 DE	염색색차 DE

[표 14] 데이터셋#1~#3 컬럼명 변경 및 삭제 내역



②-2. 데이터 타입 확인

- 'dtypes()' 함수를 사용하여 염색공정 데이터의 데이터 타입을 확인한다.

# SET1	# SET2	# SET3
workload.dtypes	operation.dtypes	ccm.dtypes
LOT번호 object 공정코드 object 투입중량(kg) float64 투입액량(L) int64 단위중량(kg) int64 염색길이(m) float64 dtype: object	LOT번호 object 공정코드 object 설비번호 object 공정일시 object 공정진행시간 int64 목표온도 int64 지시온도 float64 진행온도 float64 포속1 int64 포속2 int64 포속3 int64 포속4 int64 dtype: object	LOT번호 object 검사차수 object 작업명 object 염색색차 DE float64 dtype: object

[코드 11] 데이터 타입 확인

- 본 분석데이터는 타입 변경이 필요 없으나, 만약 정수(int64)를 실수(float64) 형식으로 변경하고자 한다면 아래 코드 예시와 같이 astype() 함수를 사용한다. 분석에는 영향을 미치지 않으므로 타입 변경 없이 그대로 진행해도 무방하다.

```
# [예시] 데이터 타입 변환: 정수->실수
workload = workload.astype({'투입액량(L)': 'float', '단위중량(kg)': 'float'})

# 데이터 타입 재확인
workload.dtypes
```

LOT번호	object
공정코드	object
투입중량(kg)	float64
투입액량(L)	float64
단위중량(kg)	float64
염색길이(m)	float64
dtype:	object

[코드 12] 데이터 타입 변경

②-3. 기초 통계 확인

- 염색공정 데이터의 기초 통계량을 확인한다. 'pandas' 패키지의 'describe()' 함수를 사용하면 변수별 행 개수(count), 평균(mean), 표준편차(std), 1사분위수(25%), 3사분위수(75%) 등의 통계량을 확인할 수 있다.
- 참고로, 'describe()' 함수는 자동으로 수치형 변수의 통계량을 보여주기 때문에 기초 통계 확인 전에 먼저 데이터 타입 확인을 통해 각 변수에 적합한 타입으로 변경해주는 게 좋다.



```
workload.describe()
```

	투입중량(kg)	투입액량(L)	단위중량(kg)	염색길이(m)
count	2150.000000	2150.000000	2150.000000	2150.000000
mean	146.882963	2460.372093	390.440000	432.928567
std	85.305814	1338.933356	194.841062	522.604399
min	0.000000	600.000000	0.000000	1.000000
25%	75.850000	1000.000000	234.000000	60.000000
50%	150.600000	2000.000000	400.000000	262.000000
75%	190.750000	4000.000000	560.000000	500.000000
max	504.200000	4000.000000	1150.000000	1600.000000

[코드 13] 데이터셋#1 수치형 데이터의 요약 통계량 확인

- describe() 함수의 파라미터에 'include=np.object' 옵션을 추가하면 범주형(object 타입) 변수의 행 개수, unique value 개수, 빈도가 가장 높은 값과 그 값의 빈도수 등의 정보를 확인할 수 있다.

```
workload.describe(include=np.object)
```

	LOT번호	공정코드
count	2150	2150
unique	2127	1
top	F2204120145	F12-1203
freq	4	2150

[코드 14] 데이터셋#1 범주형 데이터의 요약 통계량 확인

- 데이터셋#2도 위와 같은 방식으로 요약 통계량을 산출한다.



```
# 수치형 데이터의 요약통계
operation.describe()
```

	공정진행시간	목표온도	지시온도	진행온도	포속1	포속2	포속3	포속4
count	2119545.000000	2119545.000000	2119545.000000	2119545.000000	2119545.000000	2119545.000000	2119545.000000	2119545.000000
mean	164.241272	5.694867	68.007681	69.482272	101.302876	115.957423	91.250039	89.939476
std	124.064699	18.498098	36.079763	34.115828	411.971363	207.606980	388.370409	247.838779
min	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	73.000000	0.000000	30.000000	37.200000	23.000000	11.000000	0.000000	0.000000
50%	147.000000	0.000000	70.000000	69.500000	65.000000	72.000000	61.000000	46.000000
75%	227.000000	0.000000	97.600000	97.300000	99.000000	156.000000	99.000000	122.000000
max	1381.000000	138.000000	138.000000	152.500000	9998.000000	9027.000000	9997.000000	9058.000000

```
# 범주형 데이터의 요약통계
operation.describe(include=np.object)
```

	LOT번호	공정코드	설비번호	공정일시
count	2119545	2119545	2119545	2119545
unique	7003	13	28	231
top	F2205260013	F12-1203	FCM04	2022-01-28
freq	1803	1155603	138758	16996

[코드 15] 데이터셋#2 수치형 및 범주형 데이터의 요약 통계량 확인

- 데이터를 특정 열(변수)을 기준으로 그룹화하여 그룹별 요약 통계량을 살펴볼 수도 있다. 데이터 그룹화에는 'groupby()' 함수를 사용한다. 아래 코드와 같이 데이터셋#3을 그룹화하여 살펴본 결과, '작업명' '염색' 데이터와 '원단검사' 데이터가 평균, 표준편차, 최댓값 등에서 상당한 차이를 보임을 알 수 있다. 이처럼 데이터를 다양한 기준으로 구분하여 살펴보면 새로운 인사이트를 얻을 수도 있다.

```
# 수치형 데이터의 요약통계를 지정한 변수를 기준으로 그룹핑하여 확인
ccm.groupby('작업명').describe()
```

	count	mean	std	min	25%	50%	75%	max
작업명								
염색	258.0	7.370115	12.421090	0.015057	0.713638	1.477504	4.035216	35.60790
원단검사	356.0	3.100112	8.095812	0.088128	0.744514	1.349430	2.690986	93.47859

[코드 16] 데이터셋#3 수치형 데이터의 그룹별 요약 통계량 확인

```
# 범주형 데이터의 요약통계
ccm.describe(include=np.object)
```

	LOT번호	검사차수	작업명
count	614	614	614
unique	434	4	2
top	F2111110047	0	원단검사
freq	13	428	356

[코드 17] 데이터셋#3 범주형 데이터의 요약 통계량 확인

②-4. 결측치 확인

- 데이터 내부에 'NaN'으로 표기된 결측치 여부를 확인한다. 'isnull()' 함수와 'sum()' 함수를 통해 컬럼별(변수별) 결측치 개수를 확인할 수 있다. [부록 4] 품질 전처리 과정을 통해 발견한 결측치는 ②-1의 필요 변수만 추출하는 과정에서 이미 제외되어 현재는 모든 데이터에 결측치가 없는 것으로 확인된다.

# SET1 workload.isnull().sum()	# SET2 operation.isnull().sum()	# SET3 ccm.isnull().sum()
LOT번호 0 공정코드 0 투입중량(kg) 0 투입액량(L) 0 단위중량(kg) 0 염색길이(m) 0 dtype: int64	LOT번호 0 공정코드 0 설비번호 0 공정일시 0 공정진행시간 0 목표온도 0 지시온도 0 진행온도 0 포속1 0 포속2 0 포속3 0 포속4 0 dtype: int64	LOT번호 0 검사차수 0 작업명 0 염색색차 DE 0 dtype: int64

[코드 18] 변수별 결측치 개수 확인

②-5. 참고사항 - 전체 열(column) 또는 행(row)이 다 보이게 설정

- 데이터의 열 또는 행이 너무 많은 경우, 주피터 노트북에서는 이를 모두 보여주지 않고 중간 부분을 생략한다. 만약 생략된 부분 없이 모두 확인하고 싶다면, 'pandas' 패키지의 'set_option()' 함수를 활용하고, 파라미터를 아래 코드와 같이 설정하면 된다. 설정을 취소하고 싶으면 'reset_option()' 함수를 활용한다.

```
# 전체 열(column) 다 보이게 설정
pd.set_option('display.max_columns', None)
# 전체 행(row) 다 보이게 설정
pd.set_option('display.max_rows', None)

# 설정 리셋
pd.reset_option('display.max_columns')
pd.reset_option('display.max_rows')
```

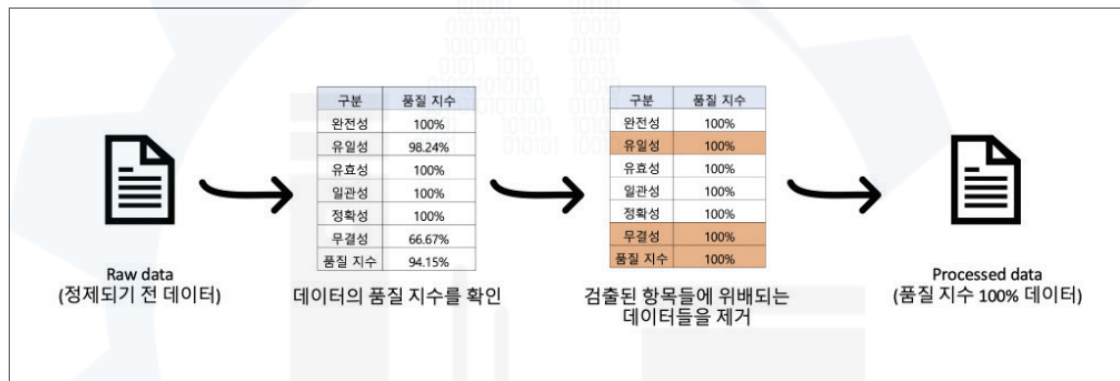
[코드 19] 전체 열(column) 및 행(row)이 다 보이게 설정

[단계 ③] 데이터셋 전처리 및 병합

③-1. 데이터 전처리

- 데이터 전처리 방법

- 개발 언어 : Python (파이썬)
- 개발 환경 : Jupyter Notebook (주피터 노트북)



[그림 13] 데이터 전처리 과정 도식화

③-2. 데이터 형식 통일

- 데이터의 'LOT번호'가 대문자로 시작하는지 확인하고, 그렇지 않을 경우 수정해준다. 'LOT번호' 옆에 'str.islower()' 함수를 적용하여 확인한 결과, 데이터셋#3에 일부 형식에 맞지 않는 데이터가 있음을 확인하였다. 'str.capitalize()' 함수를 사용하여 첫 번째 글자를 대문자로 변경해준다.



```
# LOT번호가 소문자로 시작하는 데이터 개수 확인
ccm[ccm['LOT번호'].str.islower().sum()]
```

37

```
# LOT번호가 소문자로 시작하는 데이터 확인
ccm[ccm['LOT번호'].str.islower()]
```

	LOT번호	검사차수	작업명	염색색차 DE
125	f2109240189	0	원단검사	0.654607
127	f2109240190	0	원단검사	0.277524
274	f2202160031	0	원단검사	0.649170
275	f2202160033	0	원단검사	1.815297
284	f2202210075	0	원단검사	0.838446
287	f2202210107	0	원단검사	5.252130

[코드 20] 데이터가 알파벳 소문자로 시작하는지 확인

```
# LOT번호의 첫번째 글자를 대문자로 변경
ccm['LOT번호'] = ccm['LOT번호'].str.capitalize()
```

```
# 재확인 - 소문자로 시작하는 데이터 개수 확인
ccm[ccm['LOT번호'].str.islower().sum()]
```

0

[코드 21] 데이터를 알파벳 대문자로 변경 및 수정 결과 확인

③-3. 중복데이터 처리

- 'duplicated()' 함수를 활용하여 각 분석 데이터셋에 중복데이터가 존재하는지 확인하고 처리한다. 'duplicated()' 함수의 'keep' 파라미터는 첫 번째(keep='first') 또는 마지막(keep='last') 중복값을 카운트에 포함할지, 또는 모든(keep=False) 중복값을 포함할지 여부를 결정한다.



```
# SET1 중복데이터 개수 확인
workload.duplicated(keep=False).sum()
#keep=False -> 모든 중복데이터 포함, keep='first' -> 첫번째 중복데이터는 제외
```

38

```
# SET1 중복데이터 내용 확인
workload[workload.duplicated(keep=False)]
```

	LOT번호	공정코드	투입중량(kg)	투입액량(L)	단위중량(kg)	염색길이(m)
8	F2111300043	F12-1203	193.6	4000	146	1600.0
9	F2111300043	F12-1203	193.6	4000	146	1600.0
251	F2112240047	F12-1203	102.6	1000	380	300.0
252	F2112240047	F12-1203	102.6	1000	380	300.0
467	F2201180073	F12-1203	102.1	4000	190	500.0

[코드 22] 데이터셋#1 중복데이터 개수 및 내용 확인

- 중복데이터의 개수와 내용을 먼저 확인한 후, 첫 번째 값만 남기고 모두 삭제한다. 아래 코드의 'workload.duplicated(keep='first')' 부분은 첫 번째 중복값을 제외한 나머지 중복데이터를 의미한다. 이 앞에 물결표(~)를 붙여주면 뒤에 정의한 내용의 여집합을 의미한다. 즉, '~workload.duplicated(keep='first')'는 비중복 데이터와 첫 번째 중복데이터를 의미하게 된다.

```
# 중복데이터 한 개씩만 남기고 모두 삭제
workload2 = workload[~workload.duplicated(keep='first')]
```

[코드 23] 데이터셋#1 중복데이터 삭제

- 행을 삭제하는 과정에서 인덱스번호에 빈칸이 생기므로 'reset_index()' 함수를 활용하여 인덱스번호를 재설정한다. 함수의 'drop' 파라미터는 기존의 인덱스번호를 살려서(False) 새로운 컬럼으로 추가할지, 또는 삭제할지(True)를 결정한다. 'inplace' 파라미터를 'True'로 설정하면 함수가 해당 데이터프레임에 적용 및 저장되고, 'False'로 설정하면 적용된 결과가 화면에 출력만 되고 저장되지는 않는다.

```
# 인덱스번호 리셋
workload2.reset_index(drop=True, inplace=True)
```

```
# 중복데이터 개수 재확인
workload2.duplicated().sum()
```

[코드 24] 데이터셋#1 인덱스번호 리셋 및 중복데이터 삭제 결과 확인

- 데이터셋#2 설비데이터와 데이터셋#3 CCM 데이터도 데이터셋#1과 동일한 방식으로 중복데이터를 확인하고 삭제한다.

```
# 중복데이터 개수 확인
```

```
operation.duplicated(keep=False).sum()
```

```
21600
```

```
# LOT번호+공정진행시간 순으로 데이터 재정렬
```

```
operation.sort_values(by=['LOT번호', '공정진행시간'], ascending=True, inplace=True, ignore_index=True)
```

```
# 중복데이터 내용 확인
```

```
operation[operation.duplicated(keep=False)]
```

	LOT번호	공정코드	설비번호	공정일시	공정진행시간	목표온도	지시온도	진행온도	포속 1	포속 2	포속 3	포속 4
9165	F2112100004	F12-1203	FCM11	2022-01-03	1	30	30.0	28.2	0	0	0	0
9166	F2112100004	F12-1203	FCM11	2022-01-03	1	30	30.0	28.2	0	0	0	0
9167	F2112100004	F12-1203	FCM11	2022-01-03	2	30	30.0	25.5	0	0	0	0
9168	F2112100004	F12-1203	FCM11	2022-01-03	2	30	30.0	25.5	0	0	0	0
9169	F2112100004	F12-1203	FCM11	2022-01-03	3	30	30.0	23.0	37	1	54	1
...	...	...	...	...	...	...	...	...	...	...	...	...
1820212	L2206270021	L12-1203	LCM53	2022-07-01	44	20	0.0	0.0	0	0	0	0
1820214	L2206270021	L12-1203	LCM53	2022-07-01	45	20	0.0	0.0	0	0	0	0
1820215	L2206270021	L12-1203	LCM53	2022-07-01	45	20	0.0	0.0	0	0	0	0
2113220	L2210120013	L12-1203	LCM55	2022-10-19	15	20	20.0	27.5	0	0	0	0
2113221	L2210120013	L12-1203	LCM55	2022-10-19	15	20	20.0	27.5	0	0	0	0

```
# 중복데이터 한 개씩만 남기고 모두 삭제
```

```
operation2 = operation[~operation.duplicated(keep='first')]
```

```
# 인덱스번호 리셋
```

```
operation2.reset_index(drop=True, inplace=True)
```

```
# 중복데이터 개수 재확인
```

```
operation2.duplicated().sum()
```

```
0
```

[코드 25] 데이터셋#2 중복데이터 개수 확인 및 삭제



```
# 중복데이터 개수 확인
ccm.duplicated(keep=False).sum()
```

44

```
# 중복데이터 내용 확인
ccm[ccm.duplicated(keep=False)]
```

	LOT번호	검사차수	작업명	염색색차 DE
170	F2111010033	0	염색	34.27313
171	F2111010033	0	염색	34.27313
172	F2111010033	1	염색	34.30603
173	F2111010033	1	염색	34.30603
174	F2111010033	1	염색	34.10254
175	F2111010033	1	염색	34.10254
184	F2111100017	0	염색	35.04726
185	F2111100017	0	염색	35.04726
187	F2111100017	1	염색	34.90097
188	F2111100017	1	염색	34.90097
189	F2111100017	1	염색	34.64586
190	F2111100017	1	염색	34.64586
203	F2111110047	0	염색	35.60790
204	F2111110047	0	염색	35.60790
206	F2111110047	1	염색	35.41956

```
# 중복데이터 한 개씩만 남기고 모두 삭제
ccm2 = ccm[~ccm.duplicated(keep='first')]

# 인덱스번호 리셋
ccm2.reset_index(drop=True, inplace=True)

# 중복데이터 개수 재확인
ccm2.duplicated().sum()
```

0

[코드 26] 데이터셋#3 중복데이터 확인 및 삭제



③-4. 비유일성 데이터 확인 및 처리

- 각 데이터셋에 비유일성을 가진 데이터가 있는지 확인하고, 처리한다. 앞서 [부록 4] 품질 전처리에서 살펴본 바와 같이 각 데이터셋은 유일한 값을 가지는 key가 있다. 데이터셋#1은 'LOT번호'가 유일한 값을 가지는 key이며, 데이터셋#2는 'LOT번호'+ '공정코드'+ '공정진행시간' 결합 key가, 데이터셋#3은 'LOT번호'+ '검사차수' 결합 key가 유일한 값을 가져야 한다.
- 먼저 'sort_values()' 함수를 사용하여 상기 key값을 기준으로 데이터를 재정렬한다. 'by' 파라미터를 통해 정렬 기준 열을 설정하고, 오름차순('ascending=True') 여부, 재정렬 후 인덱스번호를 재설정할 것인지('ignore_index=True'), 적용 결과를 해당 데이터프레임에 바로 저장할 것인지('inplace=True') 등을 파라미터 옵션을 통해 설정한다. 데이터 재정렬은 비유일성 처리결과에는 영향을 주진 않지만, 비유일성 내용 확인 시 중복내용을 더 알아보기 쉽게 해준다.

```
# LOT번호순으로 데이터 재정렬
workload2.sort_values(by='LOT번호', ascending=True, ignore_index=True, inplace=True)
```

[코드 27] 데이터셋#1 데이터 재정렬

- 데이터 재정렬 후, 주어진 기준변수에 따라 데이터를 그룹화하는 'groupby()' 함수와 'size()' 함수를 활용하여 데이터 유일성을 벗어나는 데이터의 개수를 먼저 확인하고, 중복데이터 확인 시 사용하였던 'duplicated()' 함수를 사용하여 비유일성 데이터의 내용을 확인한다.
- 'duplicated()' 함수의 'subset' 파라미터의 옵션으로 유일성의 key값이 되는 변수 또는 변수목록을 설정하면, 해당 변수를 기준으로 한 중복데이터를 확인할 수 있다.

```
# 데이터 유일성을 벗어나는 데이터 개수 확인
(workload2.groupby('LOT번호').size() > 1).sum()
```

2

```
# 비유일성 데이터 내용 확인
workload2[workload2.duplicated(subset='LOT번호', keep=False)]
```

	LOT번호	공정코드	투입중량(kg)	투입액량(L)	단위중량(kg)	염색길이(m)
1646	F2207120068	F12-1203	279.0	3000	686	500.0
1647	F2207120068	F12-1203	279.0	3000	686	1.0
1938	F2209010042	F12-1203	191.8	4000	146	1600.0
1939	F2209010042	F12-1203	191.8	4000	146	1.0

[코드 28] 데이터셋#1 비유일성 데이터 확인

- 데이터셋#1의 비유일성 데이터 확인 결과, 염색길이가 1m인 데이터가 수집오류로 보이므로, 'drop()' 함수를 활용하여 해당 데이터를 제거한다. 'drop()' 함수의 'axis' 파라미터는 삭제가 행('axis=0') 또는 열('axis=1') 기준으로 수행되는지를 결정한다. 아래 코드와 같이 행 기준으로 삭제할 경우는 함수의 파라미터로 삭제할 데이터의 인덱스번호를 넣어주면 된다. 데이터 삭제 후에는 인덱스번호를 리셋해준다.

```
# 비유일성 데이터 삭제
workload3 = workload2.drop([1647, 1939], axis=0) #index=1647, 1939 위치의 행(row) 삭제

# 인덱스번호 리셋
workload3.reset_index(drop=True, inplace=True)

# LOT번호 기준으로 데이터 유일성 재확인
(workload3.groupby('LOT번호').size() > 1).sum()
```

0

[코드 29] 데이터셋#1 비유일성 데이터 삭제

- 데이터셋#2와 데이터셋#3도 위와 동일한 방식으로 비유일성 데이터를 확인한다.

```
# LOT번호+공정진행시간 순으로 데이터 재정렬
operation2.sort_values(by=['LOT번호', '공정진행시간'], ascending=True, inplace=True, ignore_index=True)

# 데이터 유일성을 벗어나는 데이터 개수 확인
(operation2.groupby(['LOT번호', '공정코드', '공정진행시간']).size() > 1).sum()
```

8841

```
# 비유일성 데이터 내용 확인
operation2[operation2.duplicated(subset=['LOT번호', '공정코드', '공정진행시간'], keep=False)].head(10)
```

	LOT번호	공정코드	설비번호	공정일시	공정진행시간	목표온도	지시온도	진행온도	포속1	포속2	포속3	포속4
43597	F2112270335추가	1	FCM06	2022-01-13	1	30	30.0	65.6	0	0	0	0
43598	F2112270335추가	1	FCM06	2022-01-13	1	30	30.0	66.8	0	0	0	0
43599	F2112270335추가	1	FCM06	2022-01-13	2	30	0.0	0.0	0	0	0	0
43600	F2112270335추가	1	FCM06	2022-01-13	2	30	50.0	64.4	0	0	0	0
43601	F2112270335추가	1	FCM06	2022-01-13	3	70	0.0	0.0	0	0	0	0
43602	F2112270335추가	1	FCM06	2022-01-13	3	30	50.0	62.9	0	0	0	0
43603	F2112270335추가	1	FCM06	2022-01-13	4	70	0.0	0.0	0	0	0	0
43604	F2112270335추가	1	FCM06	2022-01-13	4	30	50.0	35.6	0	1	0	0
43605	F2112270335추가	1	FCM06	2022-01-13	5	70	0.0	0.0	0	0	0	0
43606	F2112270335추가	1	FCM06	2022-01-13	5	30	50.2	40.4	15	2	15	1

[코드 30] 데이터셋#2 비유일성 데이터 확인

- 데이터셋#2는 동일한 'LOT번호'-'공정코드'-'공정진행시간' 값에 해당하는 복수의 데이터가 있는 것으로 보아, 데이터 수집오류로 추정된다. 비유일성 데이터의 삭제는 중복데이터 삭제와 동일한 방식으로 진행한다.



```
# 중복데이터 한 개씩만 남기고 모두 삭제
operation3 = operation2[~operation2.duplicated(subset=['LOT번호', '공정코드', '공정진행시간'], keep='last')]

# 인덱스번호 리셋
operation3.reset_index(drop=True, inplace=True)

# 데이터 유일성 재확인
(operation3.groupby(['LOT번호', '공정코드', '공정진행시간']).size() > 1).sum()
```

0

[코드 31] 데이터셋#2 비유일성 데이터 삭제

- 데이터셋#3 CCM데이터는 'LOT번호-검사차수' 하나당 여러 검사결과 값이 있음을 다음 코드 결과에서 확인할 수 있다. 검사차수가 잘못 입력됐거나, 동일한 차수에 여러 번 검사를 진행하였으나 따로 구분하여 데이터를 수집하지 않은 것으로 추측된다. 앞서 중복데이터를 처리할 때처럼 첫 번째 값만 남기고 나머지 중복데이터는 다 삭제하거나, 중복되는 검사차수를 하나씩 올리는 방법 등의 데이터 처리 방안이 있을 것이다.

```
# LOT번호+검사차수 순으로 데이터 재정렬
ccm2.sort_values(by=['LOT번호', '검사차수'], ascending=True, ignore_index=True, inplace=True)

# 데이터 유일성을 벗어나는 데이터 개수 확인
(ccm2.groupby(['LOT번호', '검사차수']).size() > 1).sum()
```

52

```
# 비유일성 데이터 내용 확인
ccm2[ccm2.duplicated(subset=['LOT번호', '검사차수'], keep=False)].head(20)
```

	LOT번호	검사차수	작업명	염색색차 DE
3	F2009030093	0	원단검사	3.877990
4	F2009030093	0	원단검사	3.957100
21	F2104220009	0	원단검사	3.603126
22	F2104220009	0	원단검사	3.591169
30	F2105070082	1	염색	1.024476
31	F2105070082	1	염색	0.507672
94	F2108310449	1	염색	3.246939

[코드 32] 데이터셋#3 비유일성 데이터 개수 및 내용 확인

- 그런데 이 데이터셋의 비유일성 문제를 해결하기 전에, 먼저 전체 데이터 분석 방향을 생각해볼 필요가 있다. 현재 종속변수인 CCM 데이터(데이터셋#3)는 'LOT번호'-'검사차수'별로 구분이 되어있지만, 주요 독립변수인 시계열 설비데이터(데이터셋#2)는 'LOT번호'로만 구분되어 있다. 이 두 데이터를 결합하기 위해서는 설비데이터를 CCM데이터와 같이 '검사차수' 기준으로 구분할 수 있어야 하는데, 주어진 데이터로는 설비데이터를 구분할 수 있는 검사차수 시점에 대한 정보가 부족하다. 즉, 설비데이터와 CCM데이터는 '검사차수' 기준으로 매칭 할 수가 없다.
- 따라서 설비데이터는 'LOT번호' 기준으로만 사용할 수밖에 없으므로, 데이터셋 결합을 위해서는 CCM데이터도 'LOT번호' 기준으로만 사용할 수밖에 없다. 결과적으로 '검사차수' 정보는 더 이상 CCM데이터에 대한 기준값으로 사용할 수 없게 된다. 이에 따라, 'groupby()' 함수와 'last()' 함수를 통해 CCM데이터는 각 'LOT번호'의 마지막 검사차수 값만 추출해서 사용하고자 한다.

```
# LOT번호 기준으로 가장 마지막 값(=마지막 검사차수값)만 추출
ccm3 = ccm2.groupby(['LOT번호']).last()

# groupby를 통해 인덱스가 된 LOT번호를 다시 컬럼명으로 변경
ccm3.reset_index(drop=False, inplace=True)

# 추출 데이터 확인
ccm3
```

	LOT번호	검사차수	작업명	염색색차 DE
0	F2002240067	0	염색	1.427754
1	F2003120058	0	염색	0.616669
2	F2006160123	0	원단검사	13.001730
3	F2009030093	0	원단검사	3.957100
4	F2009050181	0	염색	1.034338
...	...	...	...	...
422	F2209080008	1	염색	0.323110
423	F2209130036	0	원단검사	4.567289
424	F2209130097	0	원단검사	0.784885
425	F2209130098	0	원단검사	0.323461
426	F2209200039	0	원단검사	1.952230

```
# 마지막 검사차수 추출 전후 데이터 개수 비교
print(ccm.shape)
print(ccm3.shape)
```

```
(614, 4)
(427, 4)
```

```
# 데이터 유일성 재확인
ccm3.duplicated(subset=['LOT번호', '검사차수']).sum()
```

```
0
```

[코드 33] 데이터셋#3 비유일성 데이터 처리



③-5. 파생변수 생성

- 기존 변수들의 조합 또는 변형을 통해 분석에 도움이 될만한 파생변수를 생성한다. 예로, 대상자들의 건강상태를 예측하기 위한 데이터셋에 ‘키’와 ‘몸무게’ 변수가 있을 경우, ‘키’와 ‘몸무게’ 변수를 그대로 사용하는 것보다 ‘BMI 지수’나, ‘키 대비 몸무게’ 변수를 생성하여 분석에 사용하는 것이 더 유의미한 분석이 될 것이다. 이와 같은 이유로 본 가이드북에서는 기존 변수들을 활용한 파생변수를 몇 생성하고, [단계 ⑦]에서는 해당 파생변수들의 특성 중요도를 함께 살펴보고자 한다.

③-5-1. 염색량 관련 파생변수 생성

- LOT 물량 데이터(데이터셋#1)의 ‘투입중량’, ‘투입액량’, ‘염색길이’, 3개 변수를 활용하여 ‘염색길이 대비 투입중량’, ‘투입액량 대비 투입중량’ 변수를 생성한다. 생성 방식은 간단히 해당 열(컬럼)들의 산술적 조합인 나눗셈(‘/’)을 활용한다.

```
# 데이터프레임 복사
workload4 = workload3.copy()
# 염색길이 대비 투입중량 파생변수 생성
workload4['투입중량/길이'] = workload4['투입중량(kg)'] / workload4['염색길이(m)']
# 투입액량 대비 투입중량 파생변수 생성
workload4['투입중량/액량'] = workload4['투입중량(kg)'] / workload4['투입액량(L)']
```

```
# 데이터 확인
workload4.head()
```

	LOT번호	공정코드	투입중량(kg)	투입액량(L)	단위중량(kg)	염색길이(m)	투입중량/길이	투입중량/액량
0	F2111300008	F12-1203	245.6	4000.0	590.0	120.0	2.046667	0.061400
1	F2111300012	F12-1203	71.4	600.0	445.0	123.0	0.580488	0.119000
2	F2111300020	F12-1203	98.3	4000.0	0.0	460.0	0.213696	0.024575
3	F2111300021	F12-1203	97.0	4000.0	0.0	452.0	0.214602	0.024250
4	F2111300027	F12-1203	40.6	600.0	445.0	86.0	0.472093	0.067667

[코드 34] 데이터셋#1 염색량 관련 파생변수 생성

③-5-2. 공정진행시간(%) 파생변수 생성

- 앞서 2.2장 “AI 분석 방법론(알고리즘) 구축 절차 설명”에서 기술한 바와 같이 본 분석은 비시계열성의 데이터셋#1, 데이터셋#3과 시계열성의 데이터셋#2를 결합하여야 한다. 이처럼 차원이 다른 데이터셋 연결방식에는 다양한 방법론이 있겠지만, 본 가이드북에서는 비시계열 데이터의 차원을 확장하여 시계열 데이터에 맞춰 매칭하고자 한다.
- 이 경우 LOT번호별, 시간대별로 진행되는 설비데이터에 LOT번호별 물량 및 품질데이터가 매칭되는 것이기 때문에, 성격이 다른 이들을 논리적으로 연결해줄 수 있는 변수가 필요하게 된다. 본 분석에서는 ‘총 공정시간 대비 진행시간(%)’ 변수를 추가하여



이중 데이터의 확장연결에 논리를 부여하고자 한다. 이에 대한 더 자세한 설명은 2.2장의 데이터 분석 방법론 부분을 참고하길 바란다.

- 데이터를 그룹화할 수 있는 'groupby()' 함수와 for문 반복문을 활용하여 각 'LOT번호' 그룹의 '공정진행시간(%)' 변수를 생성한다. for문은 기본적으로 다음의 형태를 가지며, 반복횟수만큼 카운터변수의 값이 변동하면서 실행명령의 내용을 반복한다.

for 카운터변수 *in* 반복횟수:
반복해서 실행할 명령

- [코드 35]의 카운터변수는 각 그룹의 이름('name')과 각 그룹의 객체('group')이며, 'operation3.groupby('LOT번호')'로 정의된 그룹의 개수만큼 for문이 반복될 때마다 'name'과 'group'의 값이 자동적으로 변한다.

```
# 결과를 저장할 새 데이터프레임 생성
operation4 = pd.DataFrame()

# 각 LOT번호 그룹을 루핑하면서 공정진행시간% 계산
for name, group in operation3.groupby('LOT번호'):
    group.sort_values(by=['공정진행시간'], ascending=True, inplace=True, ignore_index=True)

    # 전체 공정시간 대비 진행시간(%)
    group['공정진행시간(%)'] = round(group['공정진행시간'] / group['공정진행시간'].max() * 100, 2)

    # 결과 저장
    operation4 = operation4.append(group)

# 인덱스 리셋
operation4.reset_index(drop=True, inplace=True)
```

[코드 35] 데이터셋#2 '공정진행시간(%)' 파생변수 생성

- for문 안에서 'group'은 하나의 'LOT번호'에 대한 데이터만을 담고 있다. 데이터가 시간순으로 정렬되어 있지 않을 수도 있기 때문에 먼저 'sort_values()' 함수를 사용하여 재정렬해준다. 그리고 간단한 산술적 조합을 통해 총 공정시간 대비 현재 공정시간을 계산하고, 새로이 정의된 '공정진행시간(%)' 열(컬럼)에 저장해준다. 'max()' 함수는 가장 큰 값, 이 경우 '공정진행시간'의 가장 마지막값(=종료시점)을 추출한다. 나눗셈을 통해 계산한 값은 'round()' 함수를 통해 소수점 몇 자리까지 표시할지를 조정한다.
- for문 안에서 'group'의 파생변수를 생성한 후 다음 'group'으로 넘어가기 전에 결과물을 저장해준다. 결과물은 for문 수행 이전에 'pandas' 패키지의 'DataFrame()' 함수를 사용하여 생성한 데이터프레임 객체에 추가('append')하는 형식으로 저장한다. 본 분석과 같이 for문의 결과물을 새로운 객체에 저장하고자 하는 경우, 해당 객체는 반드시 for문 이전에 정의되어야 하고, 결과물의 저장은 for문 안에서 수행되어야 한

다. 최종 결과물은 그룹별로 인덱스번호가 새로 시작하므로 for문이 끝난 후 인덱스 번호를 재설정해준다.

```
# 결과 확인
operation4
```

	LOT번호	공정코드	설비번호	공정일시	공정진행시간	목표온도	지시온도	진행온도	포속1	포속2	포속3	포속4	공정진행시간(%)
0	F2004270020	F12-1203	FCM12	2022-05-18	1	30	30.0	31.9	0	0	0	0	0.22
1	F2004270020	F12-1203	FCM12	2022-05-18	2	30	30.0	30.7	45	1	0	0	0.44
2	F2004270020	F12-1203	FCM12	2022-05-18	3	30	30.0	30.2	32	2	0	0	0.66
3	F2004270020	F12-1203	FCM12	2022-05-18	4	30	30.0	29.8	32	4	0	0	0.88
4	F2004270020	F12-1203	FCM12	2022-05-18	5	30	30.0	29.4	32	4	0	0	1.10
...	...	...	...	...	...	...	...	...	...	...	...	...	...
2099356	L2210180007	L12-1203	LCM55	2022-10-19	277	0	20.0	28.0	0	0	0	0	98.58
2099357	L2210180007	L12-1203	LCM55	2022-10-19	278	0	20.0	28.0	0	0	0	0	98.93
2099358	L2210180007	L12-1203	LCM55	2022-10-19	279	0	20.0	26.5	0	0	0	0	99.29
2099359	L2210180007	L12-1203	LCM55	2022-10-19	280	0	20.0	26.7	0	0	0	0	99.64

[코드 36] 데이터셋#2 '공정진행시간(%)' 파생변수 생성결과 확인

- 기존의 '공정진행시간' 변수를 제외한 필요 변수들만 추출하고, 변수들의 순서를 변경해준다.

```
operation4 = operation4[['LOT번호', '공정코드', '설비번호', '공정진행시간(%)',
                        '목표온도', '지시온도', '진행온도', '포속1', '포속2', '포속3', '포속4']]
```

[코드 37] 데이터셋#2 필요 변수 추출 및 변수 순서 변경

③-6. 데이터 결합

- 개별 전처리 작업이 완료된 데이터셋을 'merge()' 함수를 사용하여 결합한다. 함수의 'on' 파라미터를 통해 결합기준이 될 열(컬럼)을 설정할 수 있고, 'how' 파라미터를 통해 결합방식을 설정할 수 있다. [코드 38]과 같이 inner-join 방식을 선택할 경우, 양쪽의 데이터에 동시에 존재하는 key값만 남게 된다.
- LOT 물량 데이터와 CCM 데이터는 'LOT번호'를 기준으로 결합한다. 여기에 다시 설비데이터를 'LOT번호'와 '공정코드'를 기준으로 동일하게 inner-join 방식으로 결합한다. 코드 결과에서 3개 데이터셋의 모든 컬럼이 병합되어 있는 것을 확인할 수 있다.



```
# LOT 물량 데이터 + CCM 데이터 결합 - 'LOT번호' 기준
df = ccm3.merge(workload4, how='inner', on='LOT번호')
#inner: 양쪽에 동시에 존재하는 LOT번호 기준으로 결합

# 설비데이터(시계열) + 그 외 데이터(배치성) 결합 - 'LOT번호', '공정코드' 기준
df2 = df.merge(operation4, how='inner', on=['LOT번호', '공정코드'])

# 결합 결과 확인
df2.head()
```

LOT번호	검사 차수	작업 명	염색색차 DE	공정 코드	투입중 량(kg)	투입액 량(L)	단위중 량(kg)	염색 길이 (m)	투입중량/ 길이	투입중 량/액량	설비번 호	공정진 행시간 (%)	목 표 온 도	지시 온도	진행 온도	포 속 1	포 속 2	포 속 3	포 속 4
0 F2201280039	0	원단검 사	0.753096	F12- 1203	44.6	1000.0	324.0	41.0	1.087805	0.0446	FCM08	0.19	30	30.0	10.9	0	0	0	0
1 F2201280039	0	원단검 사	0.753096	F12- 1203	44.6	1000.0	324.0	41.0	1.087805	0.0446	FCM08	0.39	30	30.0	10.3	16	1	16	1
2 F2201280039	0	원단검 사	0.753096	F12- 1203	44.6	1000.0	324.0	41.0	1.087805	0.0446	FCM08	0.58	30	30.0	10.3	68	2	16	1
3 F2201280039	0	원단검 사	0.753096	F12- 1203	44.6	1000.0	324.0	41.0	1.087805	0.0446	FCM08	0.77	30	30.0	10.2	69	3	138	2
4 F2201280039	0	원단검 사	0.753096	F12- 1203	44.6	1000.0	324.0	41.0	1.087805	0.0446	FCM08	0.97	30	30.0	10.8	69	3	138	2

[코드 38] 데이터셋 결합

- 시계열 데이터와 비시계열 데이터를 결합하였기 때문에 하나의 'LOT번호'에 대해 공정시간 값과 설비운영변수(온도, 포속 등) 값만 변동하고, 나머지 물량변수(투입중량, 투입액량 등) 및 품질변수(검사차수, 염색색차 DE 등) 값은 고정되어 있는 것을 볼 수 있다. 결합 전후의 데이터셋 크기 변화를 확인해보았다.

```
# 데이터 크기 변화 확인
print('SET1 데이터 크기: {}'.format(workload4.shape))
print('SET2 데이터 크기: {}'.format(operation4.shape))
print('SET3 데이터 크기: {}'.format(ccm3.shape))
print('\nSET1+SET3 데이터 크기: {}'.format(df.shape))
print('SET1+SET2+SET3 데이터 크기: {}'.format(df2.shape))
```

```
SET1 데이터 크기: (2127, 8)
SET2 데이터 크기: (2099361, 11)
SET3 데이터 크기: (427, 4)

SET1+SET3 데이터 크기: (127, 11)
SET1+SET2+SET3 데이터 크기: (34410, 20)
```

[코드 39] 데이터셋 결합 전후 데이터 크기 비교

- 다음 단계로 진행하기 전에 변수 순서를 독립변수-종속변수 순으로 변경해준다.



```
# 변수 순서 변경
df2 = df2[['LOT번호', '검사차수', '작업명', '공정코드', '설비번호',
            '단위중량(kg)', '투입중량(kg)', '투입액량(L)', '염색길이(m)', '투입중량/길이', '투입중량/액량',
            '공정진행시간(%)', '목표온도', '지시온도', '진행온도', '포속1', '포속2', '포속3', '포속4',
            '염색색차 DE']]
```

```
# 데이터 확인
df2.head(3)
```

	LOT번호	검사차수	작업명	공정코드	설비번호	단위중량(kg)	투입중량(kg)	투입액량(L)	염색길이(m)	투입중량/길이	투입중량/액량	공정진행시간(%)	목표온도	지시온도	진행온도	포속1	포속2	포속3	포속4	염색색차 DE
0	F2201280039	0	원단검사	F12-1203	FCM08	324.0	44.6	1000.0	41.0	1.087805	0.0446	0.19	30	30.0	10.9	0	0	0	0	0.753096
1	F2201280039	0	원단검사	F12-1203	FCM08	324.0	44.6	1000.0	41.0	1.087805	0.0446	0.39	30	30.0	10.3	16	1	16	1	0.753096
2	F2201280039	0	원단검사	F12-1203	FCM08	324.0	44.6	1000.0	41.0	1.087805	0.0446	0.58	30	30.0	10.3	68	2	16	1	0.753096

[코드 40] 결합 데이터셋 변수 순서 변경 및 변경 내용 확인

③-7. 이상치 제거

- 데이터에 이상치가 있는지 확인하고 처리한다. 이상치를 확인하는 방법에는 여러 가지가 있겠지만 본문에서는 ‘describe()’ 함수를 사용하여 요약 통계량을 도출하고, 각 변수값의 최소~최대 범위를 살펴보고자 한다.

```
df2.describe()
```

	단위중량(kg)	투입중량(kg)	투입액량(L)	염색길이(m)	투입중량/길이	투입중량/액량	공정진행시간(%)	목표온도	지시온도	진행온도
count	34410.000000	34410.000000	34410.000000	34410.000000	34410.000000	34410.000000	34410.000000	34410.000000	34410.000000	34410.000000
mean	448.063383	136.295086	2027.538506	234.081255	10.643335	0.067530	49.761110	7.159547	85.044897	84.815065
std	163.029422	86.847169	1196.773449	373.009418	32.734842	0.018442	28.803812	22.507373	31.310743	31.227978
min	0.000000	19.800000	600.000000	1.000000	0.117125	0.021400	0.170000	0.000000	0.000000	0.000000
25%	324.000000	60.800000	1000.000000	16.000000	0.511000	0.053300	24.802500	0.000000	70.000000	67.200000
50%	460.000000	115.100000	2000.000000	83.000000	1.116667	0.066200	49.610000	0.000000	83.300000	83.400000
75%	560.000000	189.100000	2000.000000	300.000000	4.715152	0.079300	74.580000	0.000000	112.800000	112.400000
max	1150.000000	368.000000	4000.000000	1600.000000	201.400000	0.110150	100.000000	135.000000	135.000000	137.400000

[코드 41] 결합 데이터셋 요약 통계량

- 각 변수의 생산관리 기준범위에 대한 정보가 있다면 이상치에 대한 판단이 더 수월하겠으나, 이에 대한 정보 없이도 상식적인 선에서 어느 정도는 유추가 가능하다. 예를 들어 상기 코드 결과에서 ‘단위중량=0’인 데이터는 이상치가 확실해 보이고, ‘염색길이=1’의 데이터도 의심이 간다.



③-7-1. 단위중량 이상치 처리

- 먼저 '단위중량' 변수를 살펴보자. '단위중량=0'인 데이터는 총 254개이며, 이에 해당하는 'LOT번호'의 개수는 1개이다.

```
# 이상치 개수 확인
len(df2[df2['단위중량(kg)']==0])
```

```
254
```

```
# 해당 이상치에 속하는 LOT번호 확인
df2[df2['단위중량(kg)']==0]['LOT번호'].unique()
```

```
array(['F2203230052'], dtype=object)
```

[코드 42] 결합 데이터셋 '단위중량' 이상치 확인

- 이상치 삭제는 전체 데이터셋에서 '단위중량'이 0 이상인 데이터를 추출하는 방식으로 진행한다. 데이터 행 삭제 후에는 항상 인덱스번호 재설정을 진행해주는 게 좋다.

```
# 단위중량 이상치에 해당하는 데이터 모두 제거
df3 = df2[df2['단위중량(kg)'] > 0]
df3.reset_index(drop=True, inplace=True)
```

```
# 데이터 크기 변화 확인
print('이상치 제거 전 데이터 크기: {}'.format(df2.shape))
print('이상치 제거 후 데이터 크기: {}'.format(df3.shape))
```

```
이상치 제거 전 데이터 크기: (34410, 20)
이상치 제거 후 데이터 크기: (34156, 20)
```

[코드 43] 결합 데이터셋 '단위중량' 이상치 제거

③-7-2. 염색길이 이상치 처리

- 위와 동일한 방식으로 '염색길이' 이상치를 살펴본다. '염색길이=1'인 데이터는 총 1,548개이며, 이에 해당하는 'LOT번호'의 개수는 5개이다.

```
# 이상치 개수 확인
len(df3[df3['염색길이(m)']==1])
```

```
1548
```

```
# 해당 이상치에 속하는 LOT번호 확인
df3[df3['염색길이(m)']==1]['LOT번호'].unique()
```

```
array(['F2202180111', 'F2203100080', 'F2205300077', 'F2208040015',
       'F2208040155'], dtype=object)
```

[코드 44] 결합 데이터셋 '염색길이' 이상치 확인

- '염색길이' 이상치는 데이터 수집 오류가 아닌 샘플테스트와 같은 특이 상황일 가능성도 있으나, 본 실습에서는 '단위중량' 이상치와 동일한 방식으로 제거를 진행한다.



```
# 염색길이 이상치에 해당하는 데이터 모두 제거
df3 = df3[df3['염색길이(m)'] > 1]
df3.reset_index(drop=True, inplace=True)

# 데이터 크기 변화 확인
print('이상치 제거 후 데이터 크기: {}'.format(df3.shape))
```

이상치 제거 후 데이터 크기: (32608, 20)

[코드 45] 결합 데이터셋 '염색길이' 이상치 제거

③-8. 불필요한 열 제거

- 변수별 유일값(unique value) 개수를 기준으로 분석에 불필요한 변수를 선별할 수 있다. 유일값이 적은 변수는 변동이 거의 없는 변수이기 때문에 종속변수에 상대적으로 영향을 덜 미친다. 'value_counts()' 함수와 for문을 사용하여 변수의 유일값 개수를 살펴본다.

```
# 각 수치형 변수의 unique 값 개수 확인
for col in df3.describe().columns:
    print(col, ': ', len(df3[col].value_counts()))
```

```
단위중량(kg) : 25
투입중량(kg) : 115
투입액량(L) : 4
염색길이(m) : 84
투입중량/길이 : 120
투입중량/액량 : 111
공정진행시간(%) : 9295
목표온도 : 15
지시온도 : 1073
진행온도 : 1220
포속1 : 380
포속2 : 988
포속3 : 1016
포속4 : 857
염색색차 DE : 120
```

[코드 46] 변수별 유일값(unique value) 개수 확인

- 유일값 확인 결과, '투입액량'과 '목표온도'가 불필요한 변수로 판단된다. 변수 삭제는 'drop()' 함수를 사용하며, 'axis' 파라미터는 삭제가 행('axis=0') 또는 열('axis=1') 기준으로 수행되는지를 결정한다.

```
# 불필요한 열 제거
df4 = df3.drop(['투입액량(L)', '목표온도'], axis=1)
```

[코드 47] 불필요한 열(컬럼) 제거



[단계 ④] 데이터 시각화

- 시각화에 앞서, 시각화를 도와줄 ‘matplotlib’ 패키지를 불러오고, 한글 변수로 인해 발생하는 오류를 막을 수 있는 옵션을 설정한다. 폰트는 맑은 고딕으로 지정하였다.

```
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
```

[코드 48] 데이터 시각화를 위한 패키지 불러오기

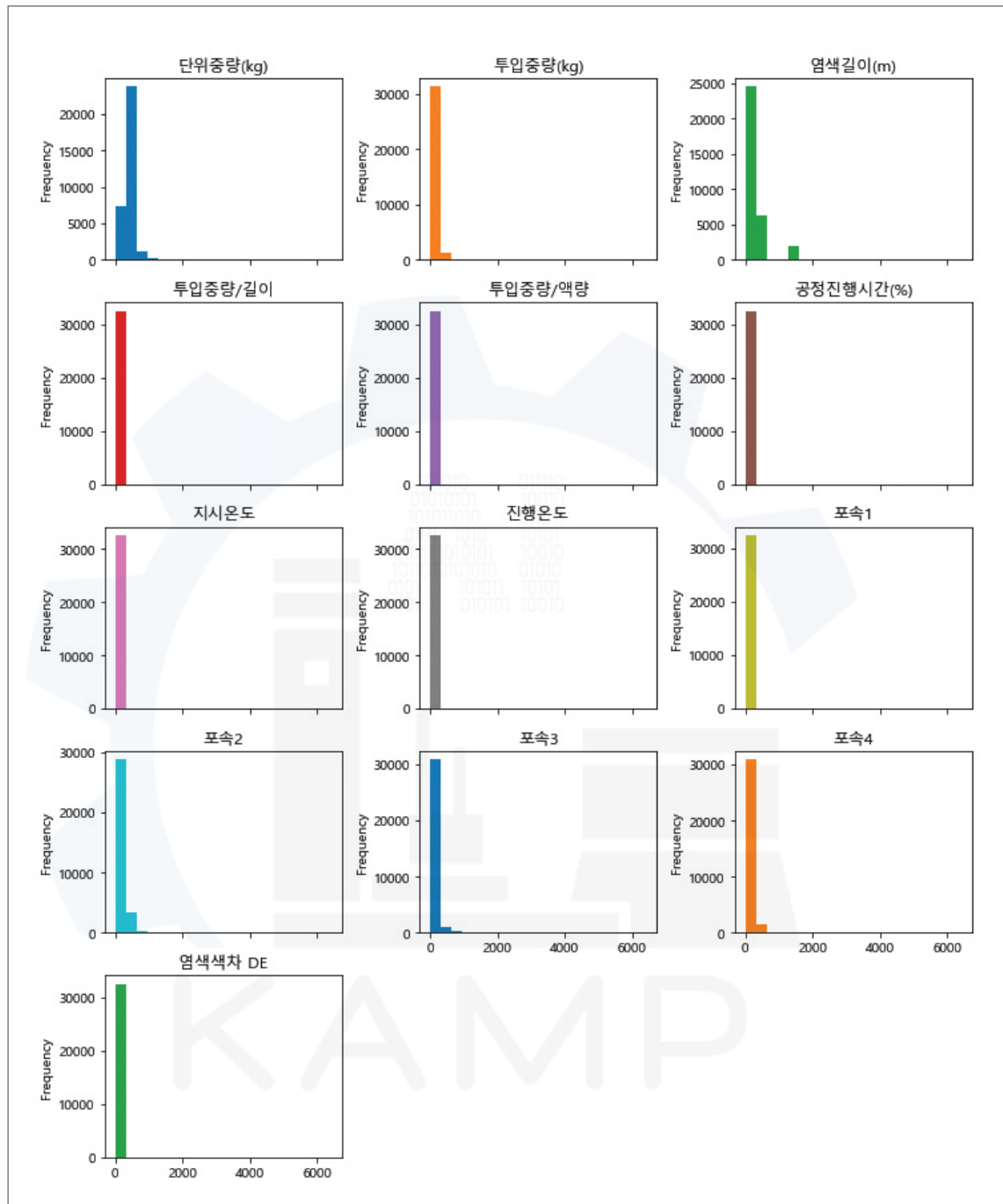
```
# 한글폰트 사용을 위한 환경 설정. '맑은 고딕'으로 폰트 설정
path = 'C:\\WINDOWS\\Fonts\\malgunbd.ttf'
font_name = fm.FontProperties(fname = path, size = 10).get_name()
plt.rc('font', family = font_name)
```

[코드 49] 한글폰트 사용을 위한 환경 설정

④-1. 히스토그램

- ‘pandas’ 패키지의 ‘plot()’ 함수에 kind='hist' 옵션을 사용하여 각 변수의 데이터 분포와 빈도를 확인할 수 있는 히스토그램을 그려본다. 따로 변수를 선별하지 않아도 ‘plot()’ 함수가 자동적으로 수치형 변수만 선별해서 시각화한다.
- 함수의 ‘bins’ 파라미터는 막대개수를 설정하는 옵션(히스토그램에만 사용 가능, 기본 값=10)이고, ‘figsize’는 그래프의 크기(넓이, 높이)를 조절한다. ‘subplots=True’는 컬럼별로 서브플롯을 그리는 옵션이며, ‘layout’은 서브플롯의 배열방식(행, 열 개수)을 조정한다. 범례는 ‘legend’, 제목은 ‘title’ 파라미터를 통해 조정할 수 있다.
- ‘pyplot’의 ‘tight_layout()’ 함수는 서브플롯 간의 간격이 너무 가깝지 않도록 조정해 준다.

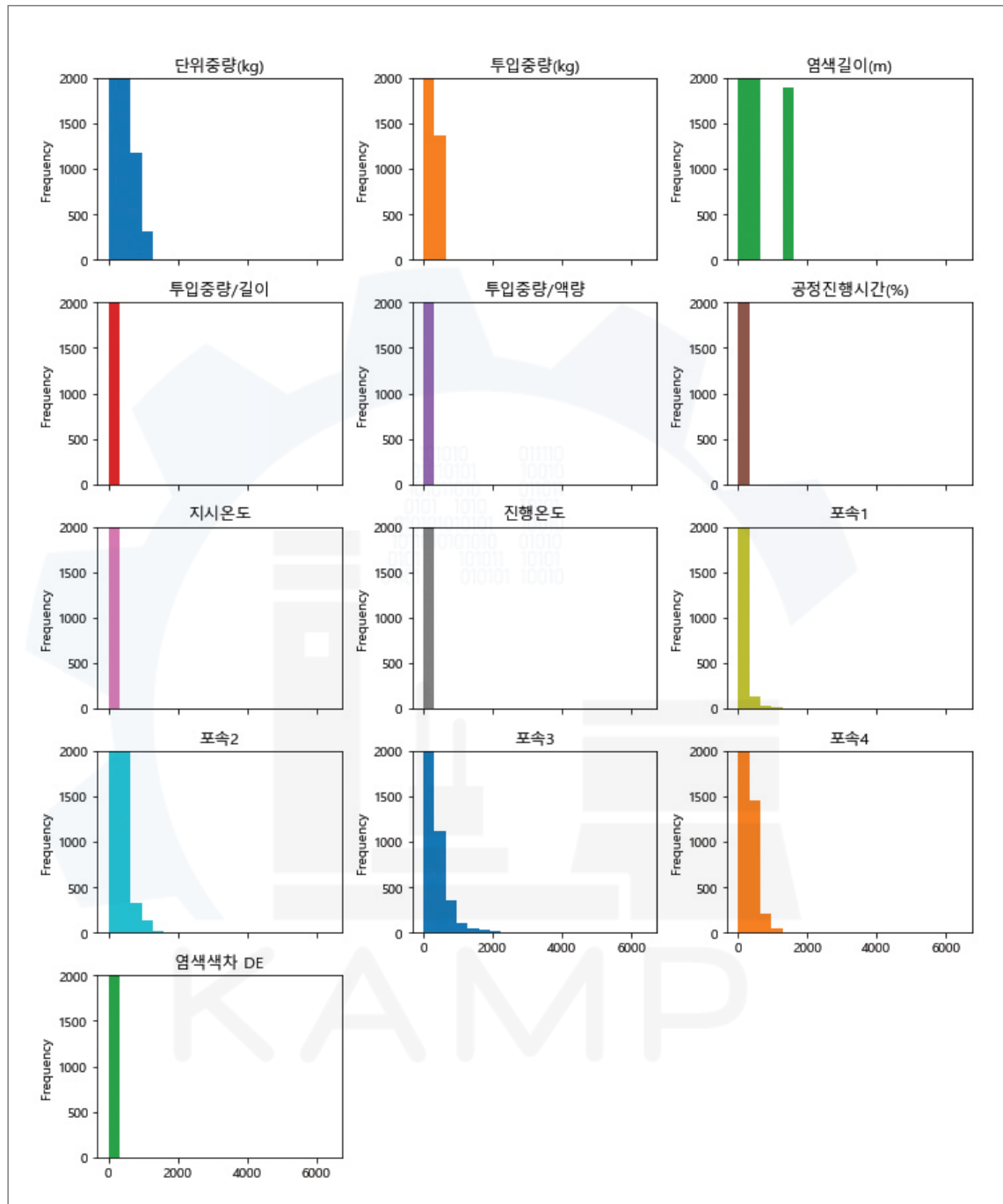
```
df4.plot(kind='hist', bins=20, figsize=(10,12), subplots=True, layout=(5,3),
         legend=False, title=list(df4.describe().columns))
plt.tight_layout()
```



[코드 50] 데이터 시각화 - 히스토그램

- 위 [코드 50]의 결과와 같이 히스토그램의 데이터 분포가 너무 한쪽에 치우쳐져 있어 저빈도 값을 보기 어려운 경우, y축의 범위를 조절하고 싶을 수도 있다. 이때는 'ylim' 파라미터를 사용하여 y축의 범위를 지정해준다.

```
# y축 조절 예시
df4.plot(kind='hist', bins=20, ylim=(0,2000), figsize=(10,12), subplots=True, layout=(5,3),
         legend=False, title=list(df4.describe().columns))
plt.tight_layout()
```

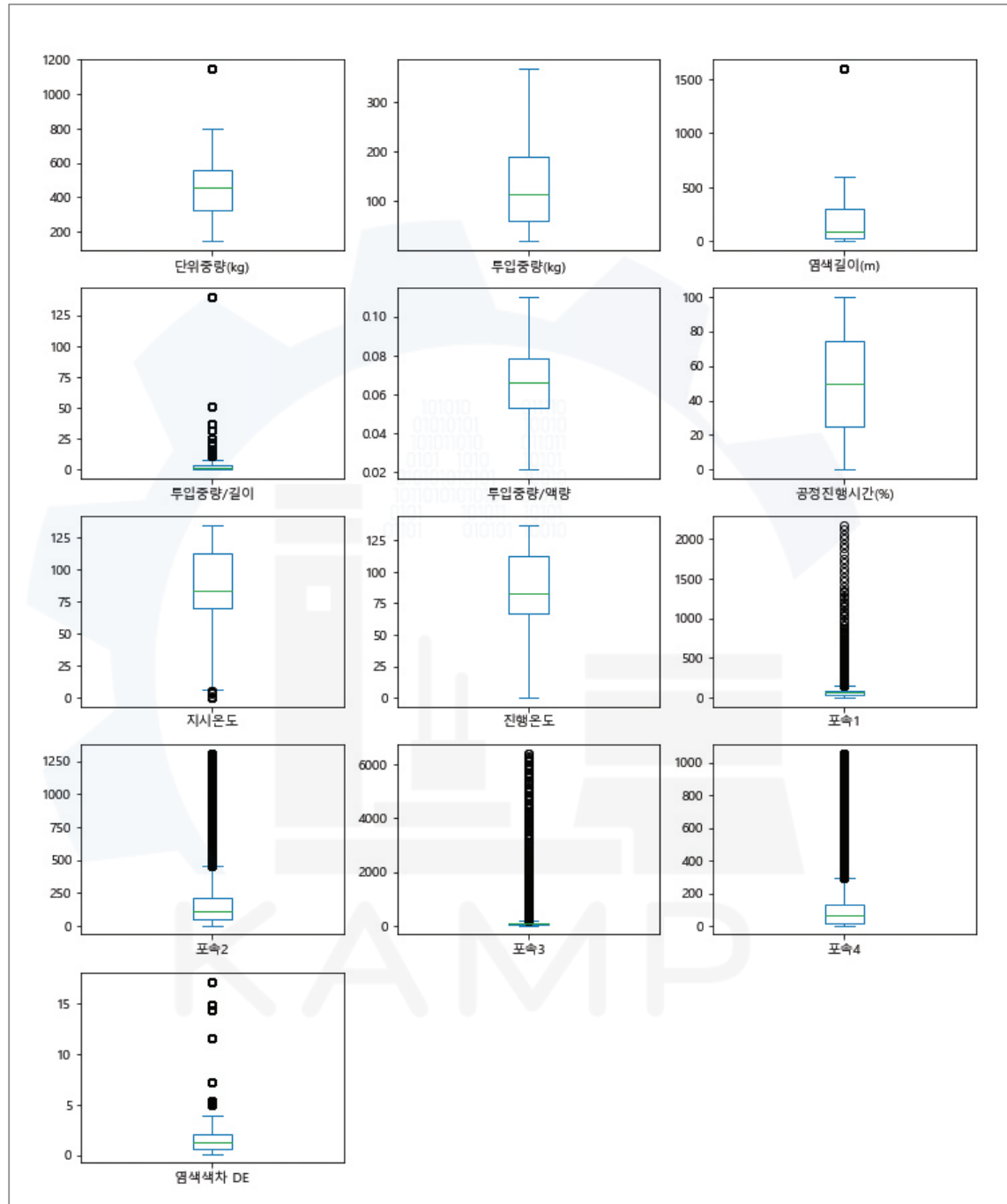


[코드 51] 데이터 시각화 - 히스토그램 y축 조절

④-2. 박스플롯

- 'plot()'함수의 kind='box' 옵션을 통해 박스플롯을 그려볼 수 있다.

```
df4.plot(kind='box', figsize=(10,12), subplots=True, layout=(5,3))
plt.tight_layout()
```



[코드 52] 데이터 시각화 - 박스플롯

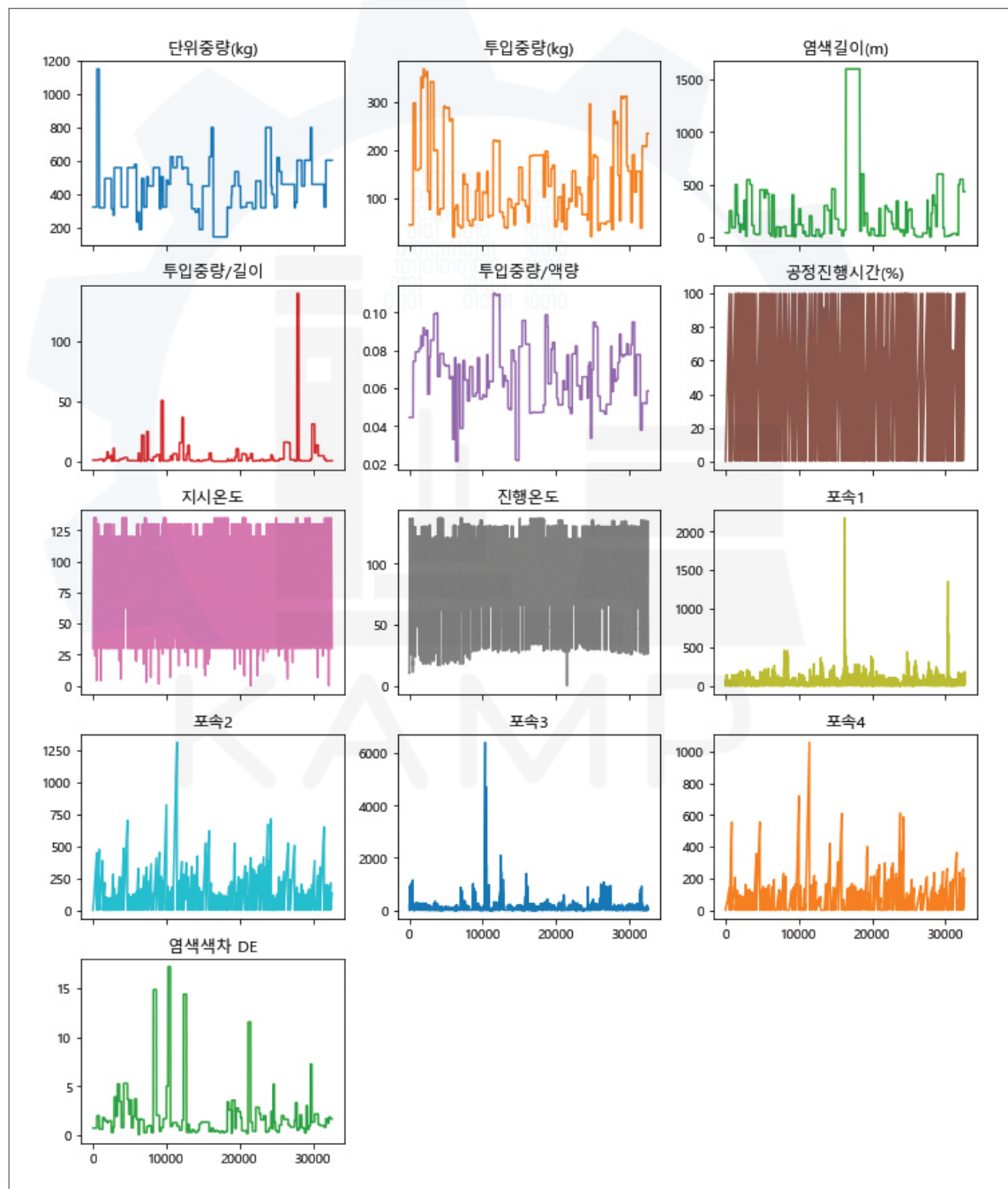


④-3. 트렌드 차트(라인플롯)

- 'plot()'함수의 kind='line' 옵션을 통해 변수의 트렌드를 확인할 수 있는 라인플롯도 그려본다. 'LOT번호' 및 '공정진행시간' 순으로 재정렬해준 후 진행하며, 이때 차트의 x축은 LOT번호-공정시간 순으로 정렬된 인덱스번호가 된다.

```
# LOT번호 및 공정진행시간 순으로 재정렬
df4.sort_values(by=['LOT번호', '공정진행시간(%)'], ascending=True, inplace=True, ignore_index=True)

# 트렌드 차트 그리기
df4.plot(kind='line', figsize=(10,12), subplots=True, layout=(5,3),
         legend=False, title=list(df4.describe().columns))
plt.tight_layout()
```



[코드 53] 데이터 시각화 - 라인플롯



[단계 ⑤] 주요 변수 선택

⑤-1. 독립변수 간의 상관관계 파악

- 독립변수 간의 상관계수를 계산하여 다중공선성 여부를 파악한다. 서로간의 상관계수가 높은 독립변수들은 선형종속 관계에 있다는 의미이며, 즉 다시 말해 종속변수에 동일한 영향을 미친다는 뜻이다. 이 경우 정확한 데이터 모델링에 방해가 되므로 깊은 상관관계에 있는 변수는 반드시 제외할 필요가 있다.
- 한글 변수로 인해 발생하는 오류와 히트맵에 음수(-) 부호 표기 관련 오류를 막을 수 있는 옵션을 먼저 설정하고 진행한다. 한글폰트는 맑은 고딕으로 지정하였다.

```
# 필요 라이브러리 불러오기
import seaborn as sns

# 그래프 '-' 음수부호 오류 해결
plt.rcParams['axes.unicode_minus'] = False

# 차트 폰트 설정
plt.rcParams['font.size'] = 13
plt.rcParams['font.family'] = 'Malgun Gothic'
```

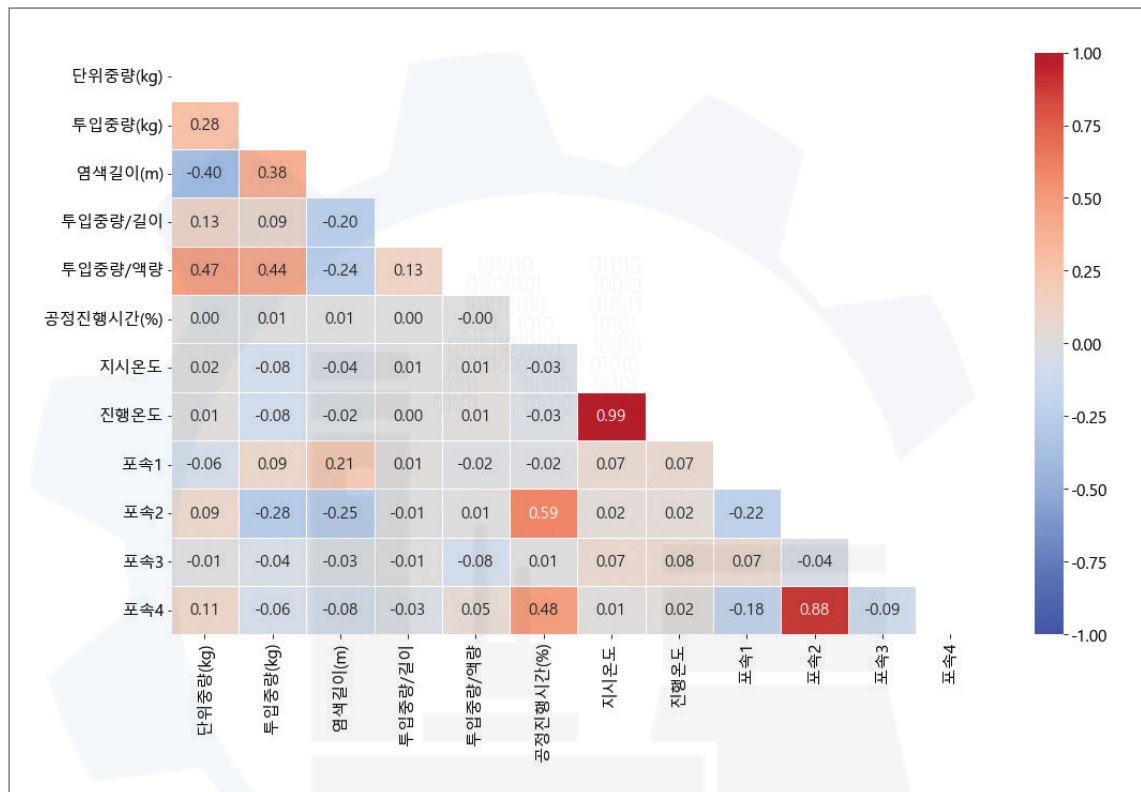
[코드 54] 히트맵 옵션 설정

- 상관관계 분석을 진행할 독립변수 데이터만 따로 추출한 뒤, 'corr()' 함수를 통해 상관계수를 도출한다. 상관계수는 한눈에 알아보기 힘들므로, 'seaborn' 패키지의 'heatmap()' 함수를 통해 히트맵 형태로 시각화한다. 함수의 다양한 파라미터 설정을 통해 히트맵의 색상, 상관계수 표기 여부, 셀 간격 등을 조정할 수 있다. 히트맵은 대각선을 기준으로 동일한 내용을 표시하므로, 'mask' 파라미터를 설정하여 상단의 반을 가려주었다.



```
# 독립변수 정의
X = df4[['단위중량(kg)', '투입중량(kg)', '염색길이(m)', '투입중량/길이', '투입중량/액량',
        '공정진행시간(%)', '지시온도', '진행온도', '포속1', '포속2', '포속3', '포속4']]

# 피어슨 상관관계수 계산
corr = X.corr()
# 삼각형 형태로 그리기 위해 mask 정의
mask = np.triu(np.ones_like(corr, dtype=np.bool))
# 히트맵 그리기
fig = plt.figure(figsize=(17,10))
sns.heatmap(corr, mask=mask, vmin=-1, vmax=1, cmap='coolwarm', annot=True, fmt=".2f", linewidths=.5)
plt.show()
```



[코드 55] 독립변수 간의 상관관계수 계산 및 삼각형 히트맵 그리기

⑤-2. 상관관계가 높은 변수 제외

- 본 분석에서는 “높은” 상관관계의 기준을 $|상관계수| \geq 0.8$ 로 설정한다. 히트맵을 통해 서로 간의 상관계수가 0.8 이상인 변수쌍(pair) 중 하나씩만 선택하여 분석변수에 포함한다.

변수1	변수2	상관계수	선택
진행온도	지시온도	0.99	진행온도
포속4	포속2	0.88	포속4

[표 15] 상관관계가 높은 변수쌍



[단계 ⑥] 데이터셋 분리 및 스케일링

⑥-1. 데이터셋 정의: 분석변수 정의

- [단계 ②]~[단계 ⑤]를 통해 전처리를 모두 완료한 최종 데이터셋에서 AI 모델링에 포함할 분석변수를 정의한다. 독립변수와 종속변수를 구분하여 정의하고, 최종 전처리 데이터셋에서 해당 변수의 데이터를 추출한다.

```
# 컬럼명 확인
```

```
df4.columns
```

```
Index(['LOT번호', '검사차수', '작업명', '공정코드', '설비번호', '단위중량(kg)', '투입중량(kg)',  
      '염색길이(m)', '투입중량/길이', '투입중량/액량', '공정진행시간(%)', '지시온도', '진행온도', '포속1', '포속2', '포속3', '포속4', '염색색차 DE'], dtype=object)
```

```
# 독립변수 정의
```

```
Xcols = ['단위중량(kg)', '투입중량(kg)', '염색길이(m)', '투입중량/길이', '투입중량/액량', '공정진행시간(%)',  
        '진행온도', '포속1', '포속3', '포속4']
```

```
# 종속변수 정의
```

```
ycol = ['염색색차 DE']
```

```
# 독립변수 데이터셋 추출
```

```
X = df4[Xcols]
```

```
# 종속변수 데이터셋 추출
```

```
y = df4[ycol]
```

[코드 56] 분석변수 정의 및 데이터셋 추출

⑥-2. 학습, 검증, 평가 데이터셋 분리

- 좋은 모델은 학습 시에 접한 적 없는 새로운 데이터에 대해 일반화 가능한 성능을 보이는 모델이라고 할 수 있다. 따라서 평가데이터를 사용하여 AI모델의 일반화 성능을 확인하는 것이 보편적이며, 이를 위해 데이터셋을 용도에 맞게 총 3가지- 학습(train), 검증(validation), 평가(test) 데이터로 나눌 필요가 있다.
- 학습 데이터셋은 AI모델 학습 시에 사용한다. 검증 데이터셋은 학습 중에 사용하여 모델의 성능을 검증하고, 이를 바탕으로 모델의 초매개변수(hyper-parameter)를 조정함으로써 학습된 모델이 평가 데이터셋에도 잘 작동하는 우수한 일반화 성능을 확보할 수 있도록 한다. 모델의 성능이 검증 데이터셋을 통해 일차적으로 검증이 되면, 마지막으로 평가 데이터셋을 사용하여 모델의 성능을 최종적으로 판단한다.
- 본 분석에서는 전체 데이터셋의 60%, 20%, 20%를 각각 학습, 검증, 평가 데이터로 활용한다. 검증 데이터의 분리는 모델의 학습 과정에서 진행된다.
- 학습데이터와 평가데이터의 분리를 위해 'train_test_split()' 함수를 활용한다. 'test-size' 파라미터를 통해 학습데이터와 평가데이터의 비율을 지정하고, 'random_state' 파라미터를 통해 매번 같은 결과가 나올 수 있도록 설정한다.



```
# 필요 패키지 불러오기
from sklearn.model_selection import train_test_split

# 데이터 분리
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=123)
```

[코드 57] 학습 데이터와 테스트 데이터 분리

⑥-3. 데이터 스케일링 (Data Scaling)

- 일반적인 제조데이터는 변수에 따라 값의 범위가 천차만별이다. 이처럼 범위 (scale)가 다른 데이터를 AI 모델링에 사용할 경우, 모델 학습시 변수별로 공평하게 기여하지 못하거나 편향이 발생할 수 있다. 이를 방지하기 위해, 데이터 표준화 (Standardization)를 통해 각 변수의 평균이 0, 분산이 1인 정규분포를 따르도록 변환한다.
- ‘scikit-learn’에서 제공하는 표준화 스케일러(‘StandardScaler()’)를 사용하여 학습 및 평가 데이터셋에 표준화를 실시한다. 학습데이터는 ‘fit_transform()’ 함수를 활용하여 ‘fit’과 표준화 적용(‘transform’)을 둘 다 실행하고, 평가데이터에는 학습데이터를 기준으로 ‘fit’한 스케일러를 그대로 적용(‘transform’)만 하여 표준화를 실행한다.

```
# 필요 패키지 불러오기
from sklearn.preprocessing import StandardScaler

# 데이터 표준화 스케일러 정의
scaler = StandardScaler()

# 훈련셋에 스케일러 적용 - fit, transform 모두 적용
X_train_sc = scaler.fit_transform(X_train)
X_train = pd.DataFrame(X_train_sc, index=X_train.index, columns=X_train.columns)

# 테스트셋에 스케일러 적용 - transform만 적용
X_test_sc = scaler.transform(X_test)
X_test = pd.DataFrame(X_test_sc, index=X_test.index, columns=X_test.columns)

# 학습데이터셋 표준화 결과 확인
X_train.head()
```

	단위중량(kg)	투입중량(kg)	염색길이(m)	투입중량/길이	투입중량/액량	공정진행시간(%)	진행온도	포속1	포속3	포속4
30406	0.069468	-0.999559	-0.631353	0.156736	0.867568	1.664968	-1.780026	-0.083082	-0.417772	-0.803234
5640	0.687940	1.398919	0.534008	-0.312830	-0.114874	1.412741	-1.591484	0.369904	-0.117588	0.214264
15213	0.007620	0.303318	-0.194671	-0.286213	0.818171	0.344506	-0.124686	-0.913556	-0.131715	0.690712
12289	0.638462	0.931979	-0.633983	2.272810	2.332997	0.979757	-0.476206	-0.732362	-0.025767	0.416149
5668	0.811634	1.445613	0.436675	-0.308392	-0.058617	-1.619532	-1.402941	0.490700	-0.131715	-0.787084

[코드 58] 데이터 표준화 및 결과 확인



[단계 ⑦] 모델 구축, 학습 및 평가

⑦-1. 하이퍼파라미터(hyper-parameter) 정의 및 모델 구축

- 머신러닝을 수행할 때 알고리즘을 구성하는 하이퍼파라미터를 잘 설정하는 것은 중요하다. 'scikit-learn' 패키지의 'GridSearchCV' 클래스를 통해 하이퍼파라미터를 모델에 순차적으로 적용하면서 편리하게 최적의 파라미터를 도출할 수 있다.
- 본 가이드북에서 튜닝할 랜덤포레스트 회귀모델('Random Forest Regressor')의 파라미터는 트리의 최대 깊이를 설정하는 'max_depth', 트리 모델의 개수를 설정하는 'n_estimators', 그리고 리프노드가 되기 위해 필요한 최소한의 샘플 데이터수를 의미하며, 과적합 제어 용도로 사용되는 'min_samples_leaf'로 한다. 랜덤포레스트 파라미터에 대한 설명은 2.2장의 <표 10>에서 확인할 수 있다.
- 'KFold()' 함수를 통해 검증 데이터셋의 분할 비율을 설정할 수 있으며 본 가이드북에서는 5로 설정한다.

```
# 필요 패키지 불러오기
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV, KFold

# 하이퍼파라미터 정의
parameters={
    'max_depth':[2, 4, 6, 8],
    'n_estimators':[20, 30, 50, 100, 150],
    'min_samples_leaf':[2, 4, 6, 8, 10]}

# K FOLD 교차검증
kfold = KFold(n_splits=5, shuffle=True, random_state=42)

# 모델 생성
model_rfr = GridSearchCV(RandomForestRegressor(), parameters, cv=kfold, verbose=2, n_jobs=-1)
```

[코드 59] 하이퍼파라미터 정의 및 랜덤포레스트 모델 생성

⑦-2. 모델 학습 및 최적화

- 'fit()' 함수를 통해 위에서 생성한 모델에 표준화된 학습 데이터셋을 적용하여 모델 학습을 수행한다. 'fit()' 함수를 호출하면 'GridSearchCV' 클래스가 자동으로 복수의 내부 모델을 생성하고, 이를 모두 실행시켜서 최적의 파라미터를 찾는다. 따라서 하이퍼파라미터 정의 시, 튜닝값을 많이 입력할수록 학습과정은 오래 걸리게 된다.
- 'GridSearchCV'를 통해 생성된 복수개의 내부 모델과 실행 결과는 다양한 속성에 저장된다. 이 중, 'best_params_' 속성을 통해 가장 높은 점수를 낸 파라미터를 확인할 수 있고, 'best_estimator_' 속성을 통해서도 가장 높은 점수를 낸 파라미터를 가진 모델을 확인할 수 있다.
- 'best_estimator_' 속성을 활용하여 최적의 모델을 새로이 정의하고, 학습 데이터셋에 적용하여 재학습을 수행한다.



```
# 모델 학습
model_rfr.fit(X_train, y_train)
print(f"랜덤포레스트 회귀모델 best 파라미터: {model_rfr.best_params_}")
# 모델 최적화
best_model_rfr = model_rfr.best_estimator_
best_model_rfr.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 100 candidates, totalling 500 fits
랜덤포레스트 회귀모델 best 파라미터: {'max_depth': 8, 'min_samples_leaf': 4, 'n_estimators': 100}
```

```
RandomForestRegressor
RandomForestRegressor(max_depth=8, min_samples_leaf=4)
```

[코드 60] 모델 학습 및 최적화

⑦-3. 모델 예측 및 모델 성능 확인

- 최적화된 모델에 'predict()' 함수를 호출하여 테스트 데이터셋에 대한 예측을 수행한다.

```
# 모델 예측
y_pred_rfr = best_model_rfr.predict(X_test)
```

[코드 61] 모델 예측

- 모델 성능에 대한 평가지표는 수정 결정계수(Adjusted R2), 평균 제곱근 오차(RMSE), 평균 절대 오차(MAE)를 사용하였다. 수정 결정계수는 0.966, RMSE는 0.554, MAE는 0.331로, 평가지표에 따라 모델의 성능은 다르게 해석될 수 있다.
- 수정 결정계수는 모델에 사용된 독립변수들의 종속변수 변화량에 대한 설명력을 나타낸다. 결정계수 1은 독립변수가 종속변수를 100% 설명한다는 뜻으로, 예측값과 실제값이 정확히 일치한다는 의미이므로 본 실습 모델의 0.966은 매우 높은 성능을 나타낸다.
- 한편, RMSE와 MAE는 예측값과 실제값의 오차를 나타내며, 두 지표 다 종속변수와 동일한 단위를 갖는다. 따라서, '염색색차 DE'의 1사분위수~3사분위수 범위의 분포값(0.6~2.0)을 기준으로 봤을 때, RMSE 0.554, MAE 0.331은 상당히 크거나, 적어도 그리 작지는 않은 오차임을 알 수 있다.

```
# 필요 패키지 불러오기
from math import sqrt
from sklearn.metrics import mean_squared_error as MSE
from sklearn.metrics import r2_score
from sklearn.metrics import mean_absolute_error

# 모델 성능 계산
r2 = r2_score(y_test, y_pred_rfr)
adj_r2 = 1 - (1 - r2) * (len(y_test) - 1) / (len(y_test) - X_test.shape[1] - 1)
rmse = sqrt(MSE(y_test, y_pred_rfr))
mae = mean_absolute_error(y_test, y_pred_rfr)

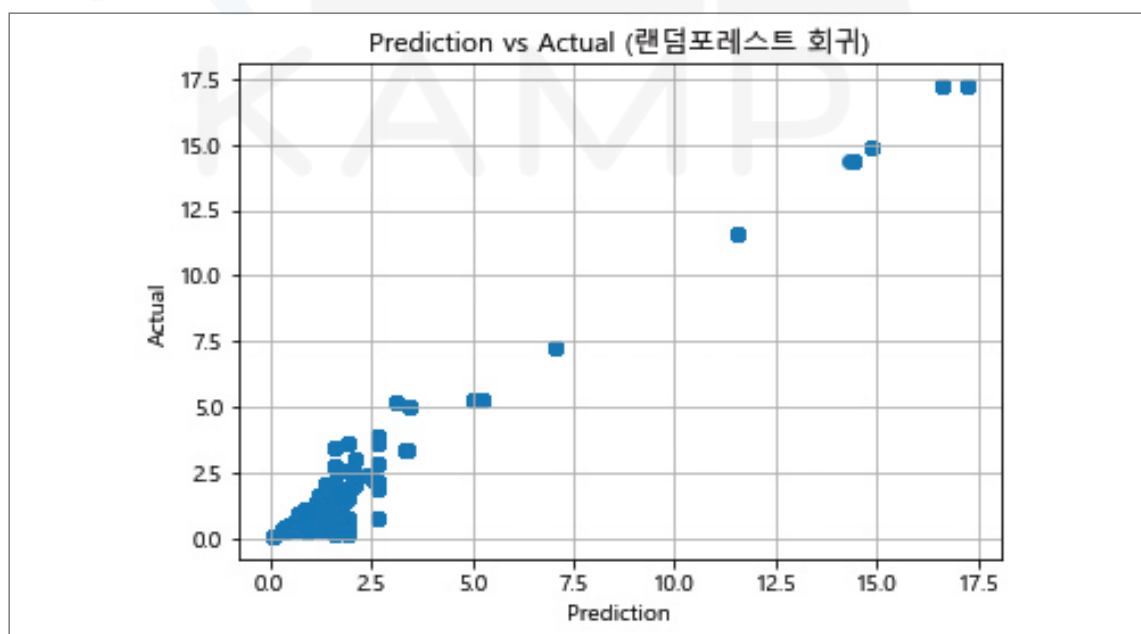
# 결과 인쇄
print(f"Adjusted. R2_score : {round(adj_r2,3)}")
print(f"RMSE score : {round(rmse,3)}")
print(f"MAE score : {round(mae,3)}")
```

```
Adjusted. R2_score : 0.966
RMSE score : 0.554
MAE score : 0.331
```

[코드 62] 모델 성능 평가지표 계산 및 확인

- ‘pyplot’의 ‘scatter()’ 함수를 활용하여 예측값과 실제값을 비교하여 그려본다. ‘pyplot’의 다양한 함수를 통해 플롯 제목, 축 제목, 그리드 사용여부 등을 설정할 수 있다. 모델 성능지표를 통해서도 확인하였듯이 예측값과 실제값은 거의 1:1의 관계를 보이며, 모델이 매우 정확히 예측하고 있음을 보여준다.

```
plt.title(f'Prediction vs Actual (랜덤포레스트 회귀)')
plt.xlabel('Prediction')
plt.ylabel('Actual')
plt.grid()
plt.scatter(y_pred_rfr, y_test)
plt.show()
```



[코드 63] 실제값과 예측값 비교 플롯



⑦-4. SHAP(SHapley Additive exPlanations)

- 모델의 성능도 중요하지만, 예측에 있어 어떤 변수가 어떻게 영향을 끼치는지 해석하고, 활용하는 것도 중요한 부분이다. 본 가이드북에서는 SHAP을 활용하여, 구축한 모델에 대한 특성(변수) 중요도와 각 특성이 어떤 방향으로 영향을 끼치는지 살펴보고자 한다.
- 먼저, SHAP을 처음 사용한다면 아래 코드를 통해 설치를 진행한다. 설치하는 처음 한번만 진행하면 된다.

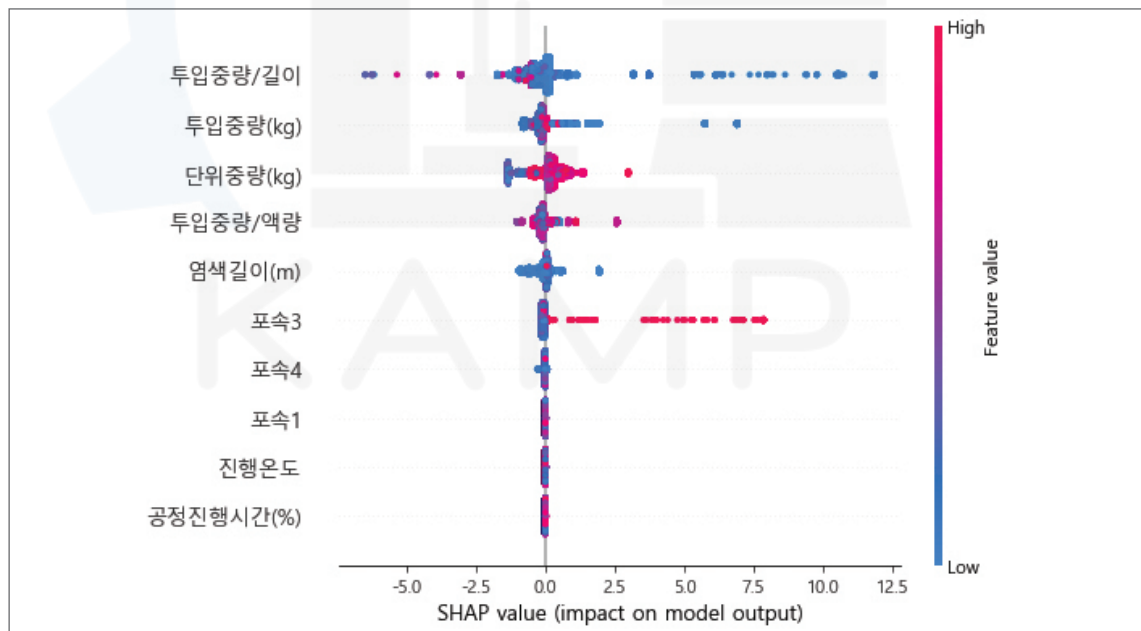
```
!pip install shap
```

[코드 64] SHAP 패키지 설치

- 모델에 대한 Tree 기반의 Explainer를 생성한 뒤, 'shap_values()' 함수를 통해 SHAP 값을 계산한다. 그리고 'summary_plot()' 함수를 통해 개략적으로 각 특성이 모델 예측에 어떤 영향을 미쳤는지 확인한다.

```
# 필요 패키지 불러오기
import shap

# 특성 영향도 그리기
explainer = shap.TreeExplainer(best_model_rfr)
shap_values = explainer.shap_values(X_train)
shap.summary_plot(shap_values, X_train)
```



[코드 65] SHAP summary plot 그리기

- Summary plot의 y축은 모델의 각 특성을, x축은 SHAP 수치를 나타낸다. 그래프상에서 특성은 예측에 미치는 영향력(=중요도)에 따라 정렬되어, 상단에 위치할수록 평균적으로 변수의 공헌도가 큰 것이고, 하단에 위치할수록 공헌도는 작다. 즉, '투입중량/길이'가 결과값 예측에 가장 큰 영향을 끼친 것이고, '포속1', '진행온도', '공정진행시간' 등이 가장 적은 영향을 끼쳤다.
- 점의 색깔은 변수의 값을 나타내며, x=0의 축을 기준으로 한 점의 이동방향은 예측값에 대한 영향력의 방향을 의미한다. 즉, '투입중량/길이'는 변수의 값이 낮을수록(파란색) 예측값에 대해 양의 영향력을 가지고(=예측값을 높임), '포속3'은 반대로 변수의 값이 높을수록(빨간색) 양의 영향력을 가진다. '포속4', '포속1', '공정진행시간', '진행온도' 등은 변수값이 모델 예측값에 거의 영향을 미치지 않는 것으로 보인다.
- SHAP 포함 모든 변수 중요도를 해석할 때 주의할 점은, 변수 중요도는 예측값과의 인과관계를 설명하는 것이 아닌 모델에 대한 각 변수의 영향도를 설명한다는 것이다.

[단계 ⑧] 유전 알고리즘(Genetic Algorithm) 적용 및 공정 최적화

⑧-1. 피쳐(변수) 변환 함수 정의 및 범위 설정

- 본 가이드북에서는 앞서 구축한 품질예측 모델과 유전 알고리즘(GA)을 활용하여 목표 염색색차 값에 대한 최적의 염색공정 운영값을 도출하고자 한다. 앞서 [단계 ⑦]에서는 염색공정의 공정조건(독립변수)이 주어졌을 때, 염색품질을 나타내는 '염색색차 DE(종속변수)' 값을 예측하는 모델을 구축하였다. 이제 본 단계에서는 반대로 '염색색차 DE'의 목표값을 설정하고, 해당 목표값을 달성하기 위해 필요한 공정조건을 역으로 도출한다. 유전 알고리즘에 대한 자세한 설명은 2.2장에서 확인할 수 있다.
- 유전 알고리즘을 적용하기에 앞서, 먼저 GA에서 요구하는 형태로 변수를 변환하는 함수를 아래와 같이 정의한다. 각 변수는 ⑥-1에서 독립변수('Xcols')를 정의한 순서대로 인덱스번호가 지정된다. 단, '투입중량/길이'의 경우 '투입중량'과 '염색길이' 변수들의 조합으로 만들어진 변수이기 때문에 따로 인덱스번호를 지정해주지 않고 산술적 관계를 정의해준다. ('투입중량/액량'의 경우에는 '투입액량' 변수가 포함되어 있지 않기 때문에 다른 변수와 동일하게 취급한다.) 만약 변수값이 정수인 경우는 'float' 대신 'int'를 사용해준다.



독립변수 확인
Xcols

```
['단위중량(kg)',
'투입중량(kg)',
'염색길이(m)',
'투입중량/길이',
'투입중량/액량',
'공정진행시간(%)',
'진행온도',
'포속1',
'포속3',
'포속4']
```

유전 알고리즘을 사용하기 위해 독립변수를 변환하는 함수 정의

```
# 0: '단위중량(kg)',
# 1: '투입중량(kg)',
# 2: '염색길이(m)',
# 3: '투입중량/길이',
# 4: '투입중량/액량',
# 5: '공정진행시간(%)',
# 6: '진행온도',
# 7: '포속1',
# 8: '포속3',
# 9: '포속4'
def input_conversion(x_input):
    conversion = [float(x_input[0]),
                  float(x_input[1]),
                  float(x_input[2]),
                  x_input[1] / x_input[2], # 투입중량/길이 = 투입중량 / 염색길이
                  float(x_input[3]),
                  float(x_input[4]),
                  float(x_input[5]),
                  float(x_input[6]),
                  float(x_input[7]),
                  float(x_input[8])]
    return conversion
```

[코드 66] 유전 알고리즘을 사용하기 위한 변수 변환 함수 정의

- 다음으로는 각 변수의 범위를 설정한다. 유전 알고리즘이 이 범위 안에서 최적값을 찾는 것이므로, 생산현장의 관리기준을 위배하지 않는 범위를 설정하는 것이 좋다. 본 실습에서는 각 변수의 최소, 최대값 정보를 활용한다. '투입중량/길이'는 따로 범위를 설정해주지 않는다. 범위의 경계에 가까워질수록 모델의 정확도가 떨어지기도 하고, 0이 분모가 되면 문제가 되므로, '포속', '진행온도'와 같이 경계값이 0인 경우는 1을 더해준다.
- 또한, 조건 제한이 필요한 변수는 [min, max] 범위가 아닌 고정값을 사용한다. 본 예시에서는 공정이 완료되었을 때, 즉 '공정진행시간'=100%일 때의 최적값을 찾고자 해당 변수를 [100, 100]으로 고정해주었다.



```
# 찾고자 하는 피처의 범위 설정

# 0: '단위중량(kg)',
# 1: '투입중량(kg)',
# 2: '염색길이(m)',
# 3: '투입중량/길이',
# 4: '투입중량/액량',
# 5: '공정진행시간(%)',
# 6: '진행온도',
# 7: '포속1',
# 8: '포속3',
# 9: '포속4'
varbound = np.array([[df4['단위중량(kg)'].min(), df4['단위중량(kg)'].max()],
                     [df4['투입중량(kg)'].min(), df4['투입중량(kg)'].max()],
                     [df4['염색길이(m)'].min(), df4['염색길이(m)'].max()],
                     [df4['투입중량/액량'].min(), df4['투입중량/액량'].max()],
                     [100, 100],
                     [df4['진행온도'].min()+1, df4['진행온도'].max()],
                     [df4['포속1'].min()+1, df4['포속1'].max()],
                     [df4['포속3'].min()+1, df4['포속3'].max()],
                     [df4['포속4'].min()+1, df4['포속4'].max()]
                    ])

```

[코드 67] 유전 알고리즘을 통해 최적값을 찾고자 하는 변수들의 범위 설정

⑧-2. 목적함수 정의

- 유전 알고리즘이 최적값을 찾기 위해 최대 또는 최소가 되게 할 목적함수를 정의한다. [단계 ⑦]에서 구축한 ‘염색색차 DE’ 예측모델인 랜덤포레스트 모델(model_rfr)을 목적함수의 파라미터로 사용하며, 목표 ‘염색색차 DE’ 값을 함수 안에 정의한다. 목표 값은 1.5로 설정하였다.
- 함수의 리턴값이 되는 ‘math.log(abs(target - prediction[0]))’은 예측값과 목표값과의 차이를 계산하는 손실함수(loss function)이다. 손실함수는 다양하게 정의가 가능하나, 여기서는 목표값과 예측값의 차이에 절대값을 씌워주고 로그를 해주었다. 유전 알고리즘이 목표값에 가까이 가서는 차이가 잘 안 줄어들 수 있기 때문에, 가까이 갈수록 이득(gain)이 더 커지도록 로그를 씌워준 것이다.

```
# 목표 '염색색차 DE'값을 얻기 위한 최적 공정조건 도출
def object_function_target(x_input, target_model=model_rfr):
    input_converted = input_conversion(x_input)
    input_converted = np.array(input_converted).reshape(1, -1)
    input_scaled = scaler.transform(input_converted)
    prediction = target_model.predict(input_scaled)
    target = 1.5 # 목표 '염색색차 DE' 값
    return math.log(abs(target - prediction[0]))

```

[코드 68] 목적함수 정의



⑧-3. 유전 알고리즘 파라미터 정의

- 유전 알고리즘에 사용될 파라미터를 정의한다. 파라미터를 통해 진화횟수, 초기 세대 개체수, 세대 변이율, 세대 교차율 등을 조정할 수 있다. 예를 들어, 아래 [코드 69]는 다음 세대의 30%는 부모세대와 동일('parents_portion'=0.3)하고, 50%는 부모세대와 자식세대의 교차('crossover_probability'=0.5)로 이뤄지도록 정의한다.

```
# 유전 알고리즘 파라미터 정의
algorithm_param = {'max_num_iteration': 500, # 진화(연산) 횟수
                  'population_size': 200,   # 초기 세대(generation)의 개체수(population)
                  'mutation_probability': 0.2, # 다음 세대의 몇 %를 변이로 채울 것인지 결정
                  'elit_ratio': 0.01,        # 이전 세대의 가장 뛰어난 해의 몇 %를 가져올 것인지 결정
                  'crossover_probability': 0.5, # 이전 세대와 현 세대를 몇 % 교차할 것인지 결정
                  'parents_portion': 0.3,     # 자식 세대가 부모 세대로부터 얼마나 가져오는지 결정
                  'crossover_type': 'uniform', # 교차방식
                  'max_iteration_without_improv': None} # 지정한 횟수만큼 연산이 없으면 정지. None: 정지하지 않음
```

[코드 69] 유전 알고리즘 파라미터 정의

⑧-4. 유전 알고리즘 모델 정의 및 실행

- 유전 알고리즘을 처음 사용한다면 아래 코드를 통해 패키지 설치를 진행한다. 설치하는 처음 한 번만 필요하다.

```
!pip install geneticalgorithm
```

[코드 70] 유전 알고리즘 패키지 설치

- 'geneticalgorithm()' 함수를 통해 알고리즘을 정의한다. ⑧-1~⑧-3에서 정의한 목적함수, 변수 범위, 알고리즘 파라미터 등이 함수의 파라미터로 사용된다. 'dimension' 파라미터에는 변수의 개수를 입력하며, 기존 변수들의 조합인 '투입중량/길이'는 제외된다. 'variable_type'에는 타깃 변수의 형식('float', 'int' 등)을 입력한다.

```
from geneticalgorithm import geneticalgorithm

# 유전 알고리즘 정의
model_ga_target = geneticalgorithm(function=object_function_target,
                                   dimension=(len(Xcols)-1), # 변수 개수. '투입중량/길이' 제외
                                   variable_type='real',      # 타깃 변수가 정수면 'int' 사용
                                   variable_boundaries=varbound,
                                   algorithm_parameters=algorithm_param)
```

[코드 71] 유전 알고리즘 정의

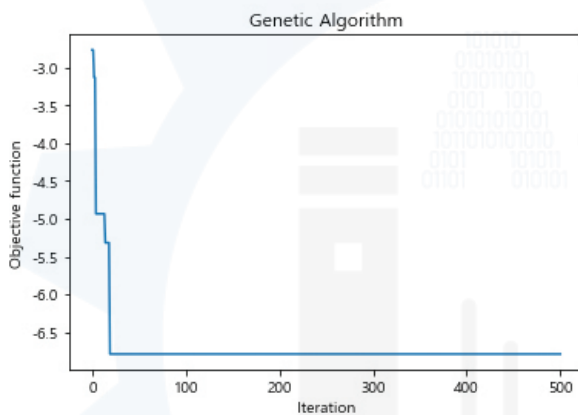
- GA의 실행은 'run()' 함수를 사용한다. 알고리즘이 실행되는 동안에는 화면에 연산 진척도(%)가 표시되어 진행상황을 알 수 있다. 실행이 완료된 뒤, 최적값과 값을 찾아가는 동안의 손실함수 값의 변화를 나타내는 그래프가 화면에 출력된다. 컴퓨터 사양, 변수 개수, 예측모델의 종류, GA 파라미터 설정값 등에 따라 연산에 상당한 시간이 소요될 수도 있다.

```
import math
```

```
# 유전 알고리즘 실행
model_ga_target.run()
```

```
The best solution found:
[6.00540347e+02 1.88690150e+02 1.73432396e+02 3.41069322e+02
1.00000000e+02 1.15044274e+02 3.60875868e+02 4.10290350e+02
9.29372713e+02]
```

```
Objective function:
-6.792874646531184
```



[코드 72] 유전 알고리즘 실행

- 상기 결과의 그래프를 통해 GA가 상당히 이른 시점에 최적의 해를 찾은 것을 알 수 있다. GA가 도출한 최종 결과를 좀 더 보기 쉽게 표 형태로 변환한다.

```
# 최종 솔루션
convergence_target = model_ga_target.report # iteration 동안 convergence 되는 과정
solution_target = model_ga_target.output_dict

# 최종 솔루션을 보기 쉽게 표 형태로
ga_solution = input_conversion(solution_target['variable'])
ga_solution = np.array(ga_solution).reshape(1, -1)
ga_solution_df = pd.DataFrame(ga_solution, columns=Xcols)
ga_solution_df
```

단위중량(kg)	투입중량(kg)	염색길이(m)	투입중량/길이	투입중량/액량	공정진행시간(%)	진행온도	포속1	포속3	포속4
0	600.540347	188.69015	173.432396	1.087975	0.034107	100.0	115.044274	360.875868	410.29035 929.372713

[코드 73] 최종 결과를 표 형태로 변환



⑧-5. 공정 최적값에 대한 '염색색차 DE' 예측

- 마지막으로 위에서 도출한 공정 최적값으로 다시 '염색색차 DE' 값을 예측한다. 예측에는 동일한 랜덤포레스트 회귀모델을 사용한다. 예측 결과, 목적함수에서 설정한 목표값인 1.5에 매우 가깝게 예측한 것으로 나타났다.

```
# 위 최적의 해를 사용하여 최종 '염색색차 DE' 값 예측
prediction = model_rfr.predict(scaler.transform(ga_solution))
ga_solution_df['염색색차 De'] = prediction[0]
ga_solution_target = ga_solution_df
ga_solution_target
```

	단위중량 (kg)	투입중량 (kg)	염색길이 (m)	투입중 량/길이	투입중 량/액량	공정 진행 시간 (%)	진행온도	포속1	포속3	포속4	염색색차 De
0	600.540347	188.69015	173.432396	1.087975	0.034107	100.0	115.044274	360.875868	410.29035	929.372713	1.498878

[코드 74] 공정 최적값에 대한 '염색색차 DE' 예측

- 상기 결과에 따르면, 원단은 188kg, 173m만큼 투입하고, 염색액량은 5,500L 투입하며, 공정완료 시점에 진행온도가 115도, 포속1 360, 포속3 410, 포속4가 929일 경우, 염색색차 DE 값은 목표로 한 1.5를 달성할 수 있다. 물론, 이는 변수의 범위를 단순히 통계적 최소~최대 값으로만 설정한 결과로, 현실적으로는 불가능한 공정조건일 수도 있다. 예를 들어, 각 염색설비의 크기에 따른 한계가 있기 때문에 '투입액량'(또는 '투입중량/액량')에도 제한조건이 필요하다. 따라서, 실제 현장에 적용할 수 있는 분석 결과를 도출하기 위해서는 현장 전문가와의 충분한 논의를 통한 도메인 지식을 바탕으로 변수 범위 및 제한조건이 설정되어야 한다.
- 유전 알고리즘은 생물학적 진화에 바탕을 둔 알고리즘이기 때문에 위 도출한 최적의 해가 유일한 최적값은 아니며, 실행 시마다 다른 해가 도출될 것이다. 따라서 제조현장에 적용 시에는 반복실행을 통해 여러 최적값 중에서 선택이 필요하다.



3. 유사 타현장의 「의류 공정최적화 AI 데이터셋」 분석 적용

3.1 본 분석이 적용 가능한 제조현장 소개

- 의류 염색가공 공정을 포함하는 대부분의 중소기업 제조업체에서는 본 가이드북에서 제시된 것과 동일한 문제를 안고 있을 것이다. 이와 같은 어려움이 있는 산업현장에서 만약 공정운영 데이터를 설비 PLC 또는 센서를 통해 확인 및 수집이 가능하고, 품질검사 결과를 전산화하여 관리하고 있다면 본 분석은 적용 가능하다.
- 본 실습에서는 염색공정의 설비데이터와 원재료 투입물량 및 생산물량 데이터를 바탕으로, 머신러닝을 통한 염색품질결과 사전예측 모델을 구축하고, 나아가 유전 알고리즘 적용을 통해 목표 염색품질 달성을 위한 공정 최적값을 도출하였다. 현장 데이터의 종류에 따라 분석의 세부적인 부분은 변경이 필요하지만, 데이터 분석 및 모델 구축의 전체적인 프로세스는 동일하기 때문에 이를 바탕으로 알맞게 활용한다면 실질적인 품질향상 및 공정 최적화를 통한 비용절감이 가능할 것으로 기대된다.
- 특히, 본 실습에서는 세 개의 다른 데이터셋을 하나의 데이터셋으로 통합하는 과정을 포함하고 있으며, 시계열 데이터와 횡단면의 비시계열성 데이터를 결합하는 과정에서 발생하는 이슈를 다뤘다. 일반적으로 설비 PLC 또는 센서를 통해 수집되는 데이터는 시계열성, MES 등의 시스템에 직접 key-in 하는 방식으로 관리하는 생산정보나, 공정완료 후 진행되는 품질검사 데이터는 비시계열성일 가능성이 크다. 대부분의 제조현장에서는 이처럼 다른 형태의 데이터가 수집되기 때문에 본 가이드북에서 제시된 이중 데이터의 결합 방법론은 타 제조현장에서도 충분히 활용이 가능할 것이다.

3.2 본 「의류 공정최적화 AI 데이터셋」 분석을 원용하여 타 제조현장 적용 시, 주요 고려사항

- 본 가이드북에서 소개한 데이터 분석 및 모델 구축 방법론을 타 제조현장에 효과적으로 적용하기 위해서는 기본적으로 제조 환경, 공정 및 설비 등에 대한 충분한 지식 확보가 선행되어야 한다. 실습의 전처리 과정에서 진행한 중복데이터 처리, 비유일성 데이터 처리, 파생변수 생성, 데이터셋 결합 등의 모든 작업은 데이터셋의 구조와 내용에 대한 이해 없이는 올바르게 진행되기 어려우며, 데이터를 제대로 이해하기 위해서는 해당 데이터가 수집된 제조공정에 대한 기본적인 이해가 요구된다.
- 본 실습에서 구축된 품질예측 모델은 타 생산현장 데이터에 바로 적용될 수는 없다. 분야/공정/설비/제품이 변경되면 이에 해당되는 데이터로 분석모델을 다시 학습하여 모델을 구축하여야 한다.
- 본 실습은 데이터 수량이 부족하여 제품 품목 및 컬러에 대한 구분 없이, 복수의 품목/컬러의 데이터를 단일 품목/컬러로 취급하여 진행하였다. 단일 품목 및 단일 컬러가 아닌 다양한 제품에 적용하기 위해서는 각 대상 품목 및 컬러에 대해 충분한 데이터가 축적되어야



할 것이고, 품목 및 컬러와 관련한 범주형 변수들을 모두 포함한 더욱 정밀한 AI 모델 구축이 요구된다.

- “Garbage in, garbage out”이라는 표현이 있듯이 좋은 데이터 분석결과를 얻기 위해서는 좋은 데이터가 필요하다. 다시 말해 데이터 분석은 데이터 수집부터 시작된다고 할 수 있다. AI 분석을 진행하기 전에 먼저 분석 목적(무엇을 예측하고자 하는지, 무엇을 최적화하고자 하는지)을 정의하고, 예측 또는 최적화하고자 하는 특성과 연관이 있는(연관이 있어 보이는) 특성에 대해 구체화한 후, 이들 특성에 대한 양질의 데이터 수집이 필요하다.
- 또한, 데이터 수집의 형태 및 구조에 대한 논의 및 관리가 필수적이다. 특히 제조데이터의 경우는 모든 데이터가 “추적이 가능한” 기준값(제품번호, LOT번호, 배치번호, 작업번호 등)이 포함되어 있어야 한다. 만약, 본 실습에서와 같이 설비데이터에 따른 품질결과를 예측하고자 할 때 어느 한쪽에라도 ‘LOT번호’ 값이 부재했다면 설비데이터와 품질데이터를 연결할 수 없었을 것이다. 따라서 현장 관리자와 AI 분석가는 충분한 소통을 통해 분석에 필요한 데이터의 형태 및 구조를 사전에 논의하여 가공 가능한 형태로 데이터를 수집하고 저장함으로써 AI 분석모델 적용의 토대를 구축하는 것이 중요하다.
- 본 실습의 공정 최적화에 사용된 유전 알고리즘은 별도로 정의되는 변수 범위 내에서 최적값을 찾기 때문에 변수 범위와 변수 제한조건 설정이 도출되는 최적값에 매우 큰 영향을 미친다. 본 실습에서는 각 변수의 통계적 범위(최소~최대)를 활용하였지만, 실제 현장에 적용 시에는 현장 관리자와 충분한 논의를 통해 해당 도메인에서 요구되는 공정관리기준에 적합하고, 설비용량 등의 물리적, 현실적 한계를 고려하여, 보다 정교하게 범위를 설정하여야 할 것이다.

3 부록_분석환경 구축을 위한 설치 가이드

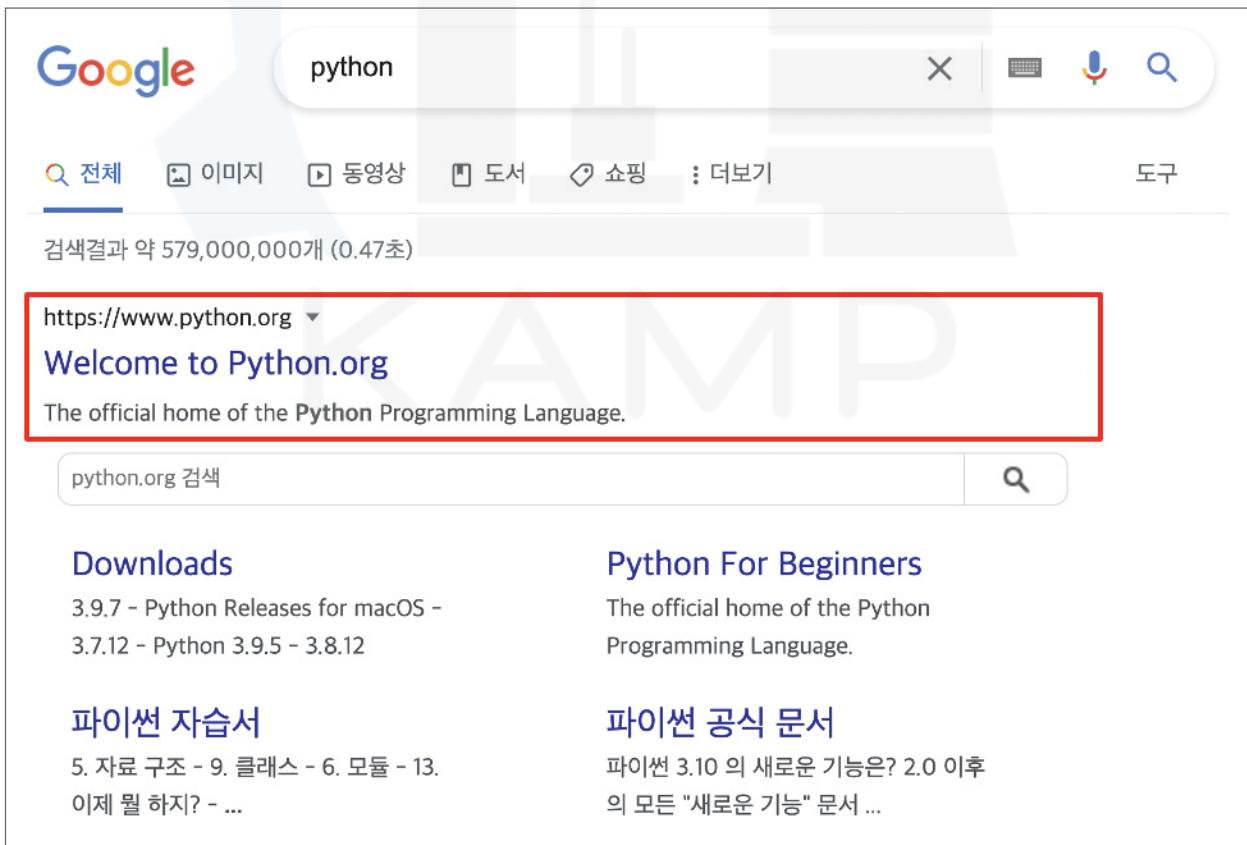
1. 파이썬(Python) 설치

- 파이썬은 컴퓨터 언어로서 최근 데이터 분석 및 AI 분석모델 개발 등에 널리 사용되는 도구이다.
- 다운로드 및 설치가 간편하고 활용도가 높은 파이썬을 로컬 컴퓨터에 설치하고 적용하는 방법을 안내한다.

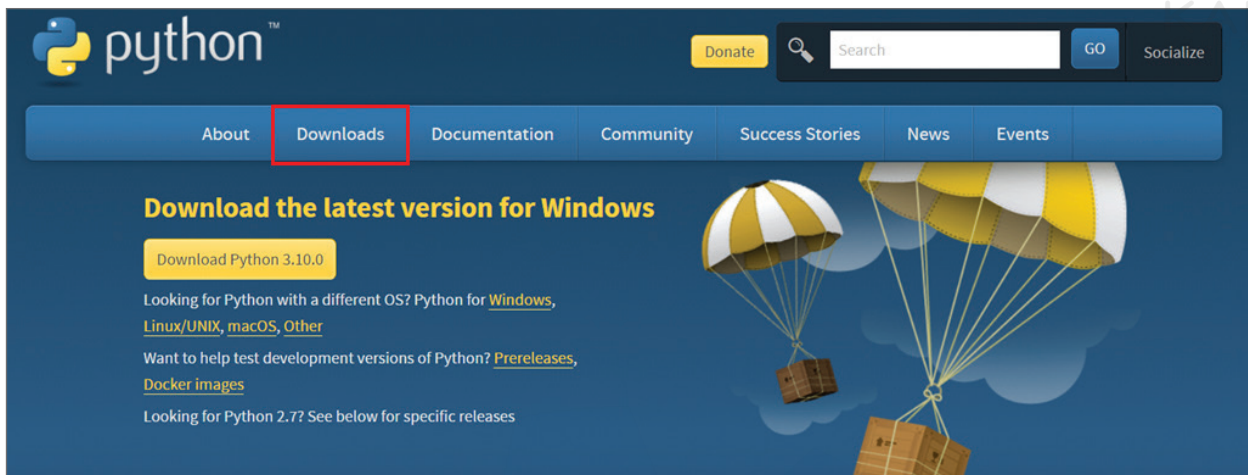
① google.com 등의 검색 엔진에 'python'을 검색한다.



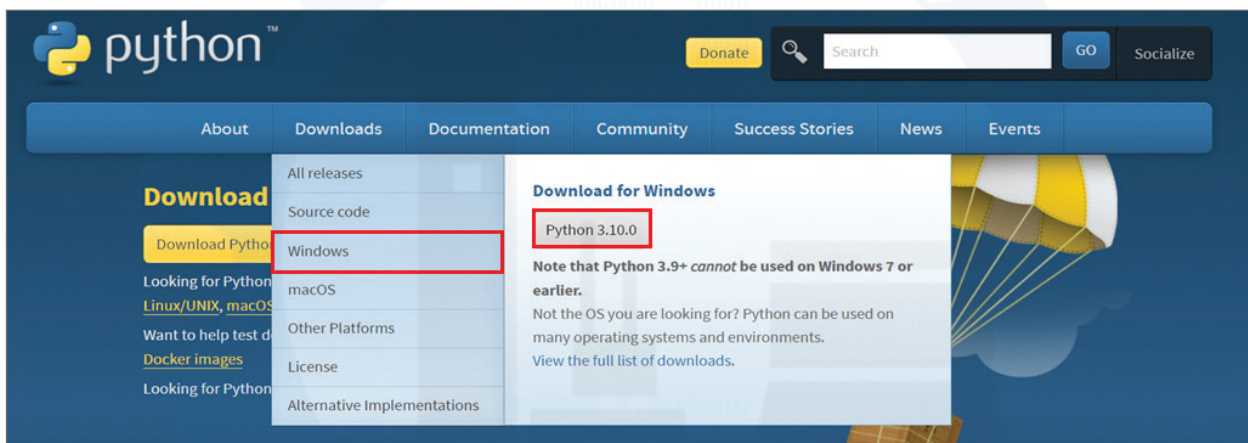
② 제일 처음에 보이는 'Welcome to Python.org'를 클릭한다.



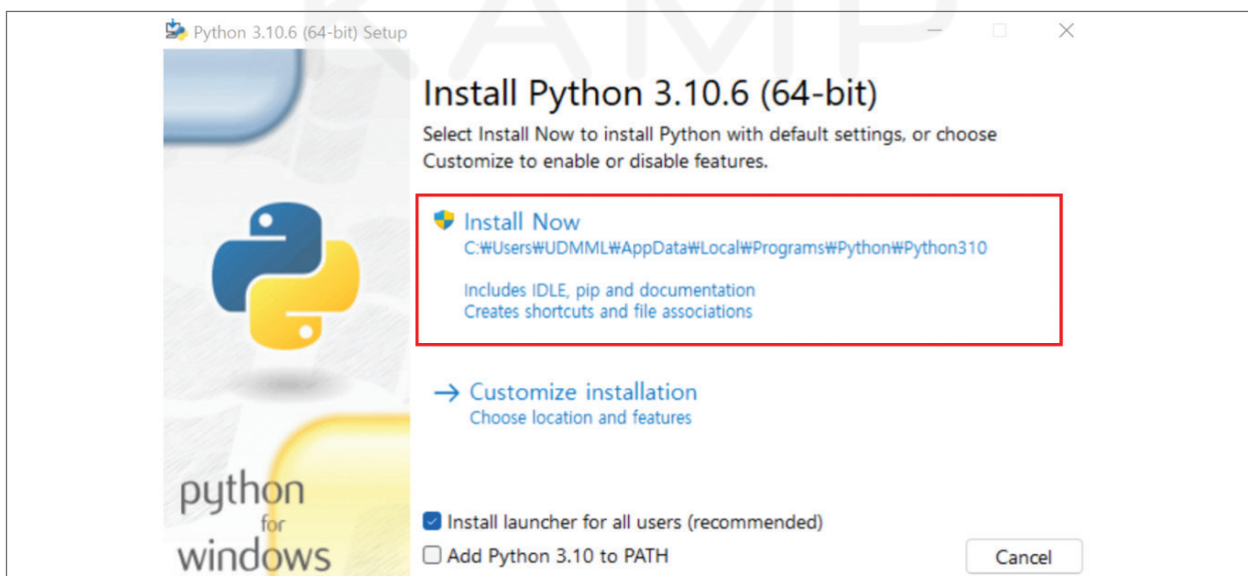
③ 클릭 후 보이는 페이지의 정면에서 상단 좌측 2번째 'Downloads'를 클릭한다.



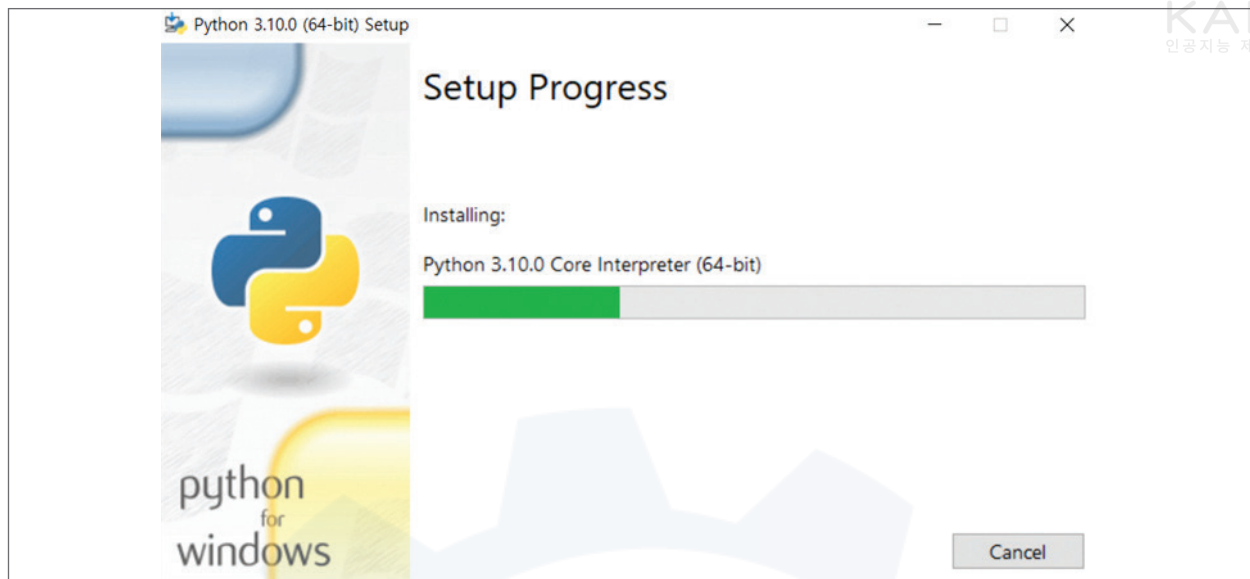
④ 컴퓨터의 운영체제에 따라 상단에서 세 번째(macOS의 경우 네 번째) 탭을 선택한 후, Python 3.10.0을 다운로드한다. (Python 3.10.0 버전은 업데이트 될 수 있다)



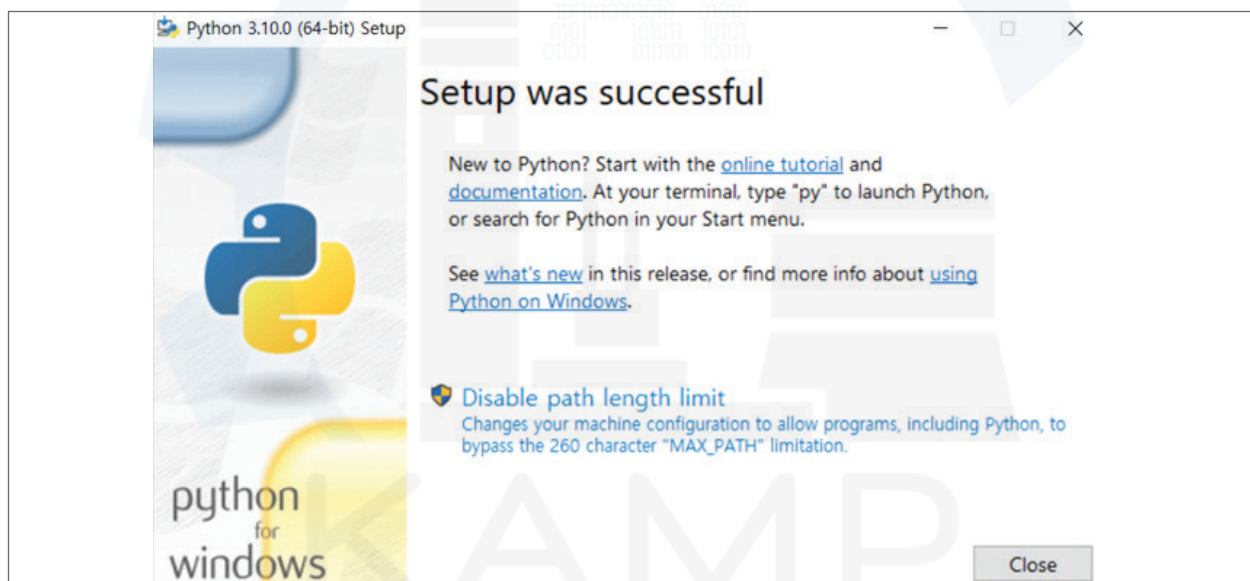
⑤ 아래와 같은 설치창이 뜨면, 'Install Now'를 클릭한다.



⑥ 아래와 같은 설치 진행창이 완료가 될 때까지 유지한다.



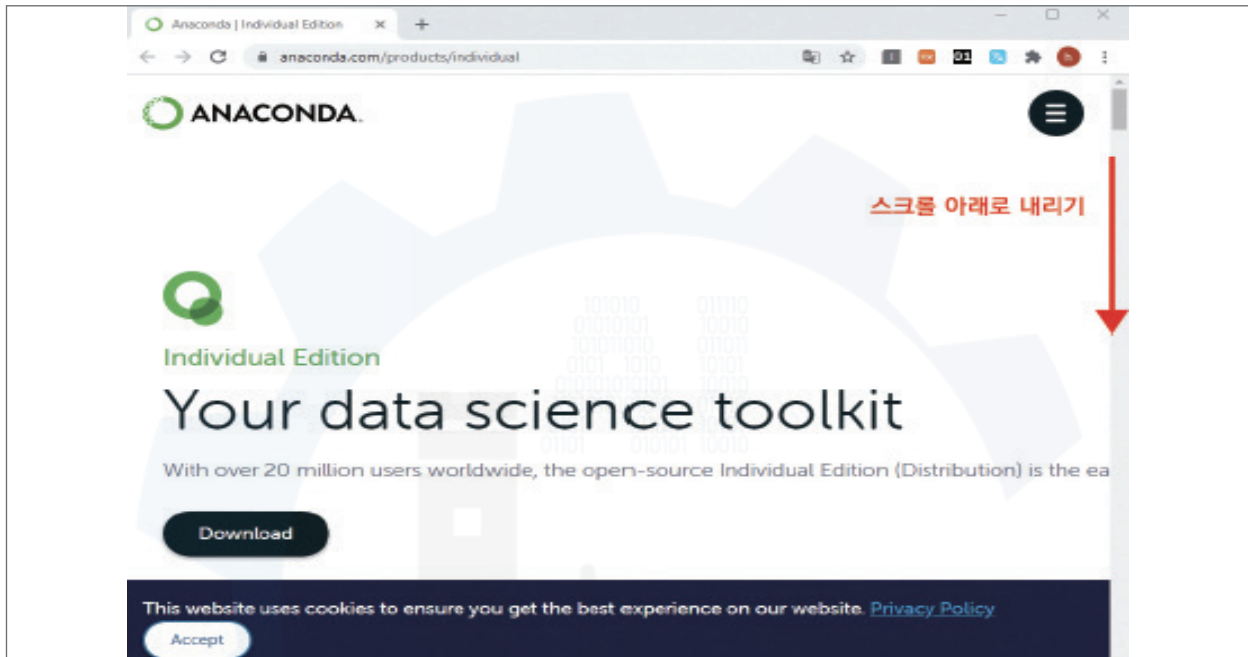
⑦ 완료가 되면 아래와 같은 창이 뜨는 것을 확인 후 종료한다. [설치완료]



2. 아나콘다(Anaconda) 설치

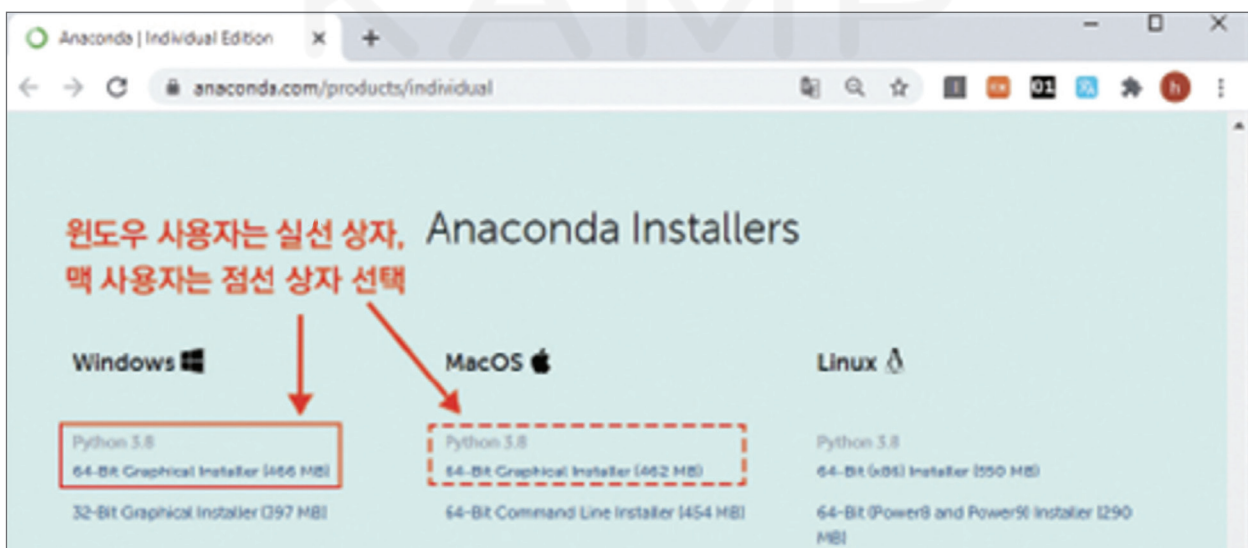
- 아나콘다란 파이썬과 같은 분석 도구를 사용할 때 필요한 고급 기능 및 분석을 보조하는 도구이다.
- 아나콘다를 설치함으로써 다양한 기능들을 바로 사용할 수 있고, 결과물을 쉽게 확인할 수 있다.
- 아나콘다를 설치하고 분석을 실시할 수 있는 환경을 구축하는 방법에 대해 안내한다.

① <https://www.anaconda.com/distribution/> 로 접속한다.



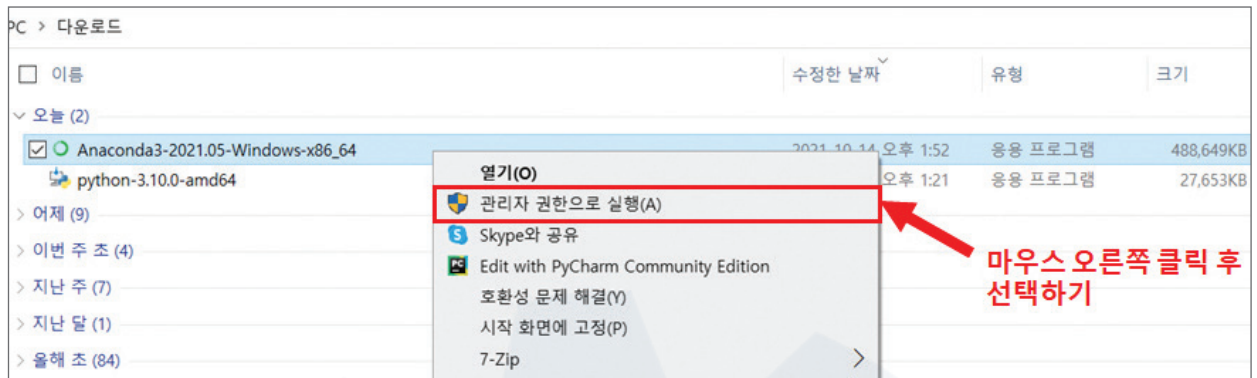
② 다음과 같은 화면이 나올 때 까지 스크롤하여 아래로 내린 후, 로컬 컴퓨터 사용 환경에 맞는 파일을 다운받는다.

- ▶ Windows : 64-Bit Graphical Installer
- ▶ MacOS : 64-Bit Graphical Installer



- ③ 다운로드 받은 파일로 이동하여, 설치된 아나콘다 파일 위에서 마우스 오른쪽을 클릭한 후, 방패모양의 ‘관리자 권한으로 실행’을 클릭한다.

▶ (예) ‘다운로드’ 폴더로 아나콘다를 다운 받은 경우

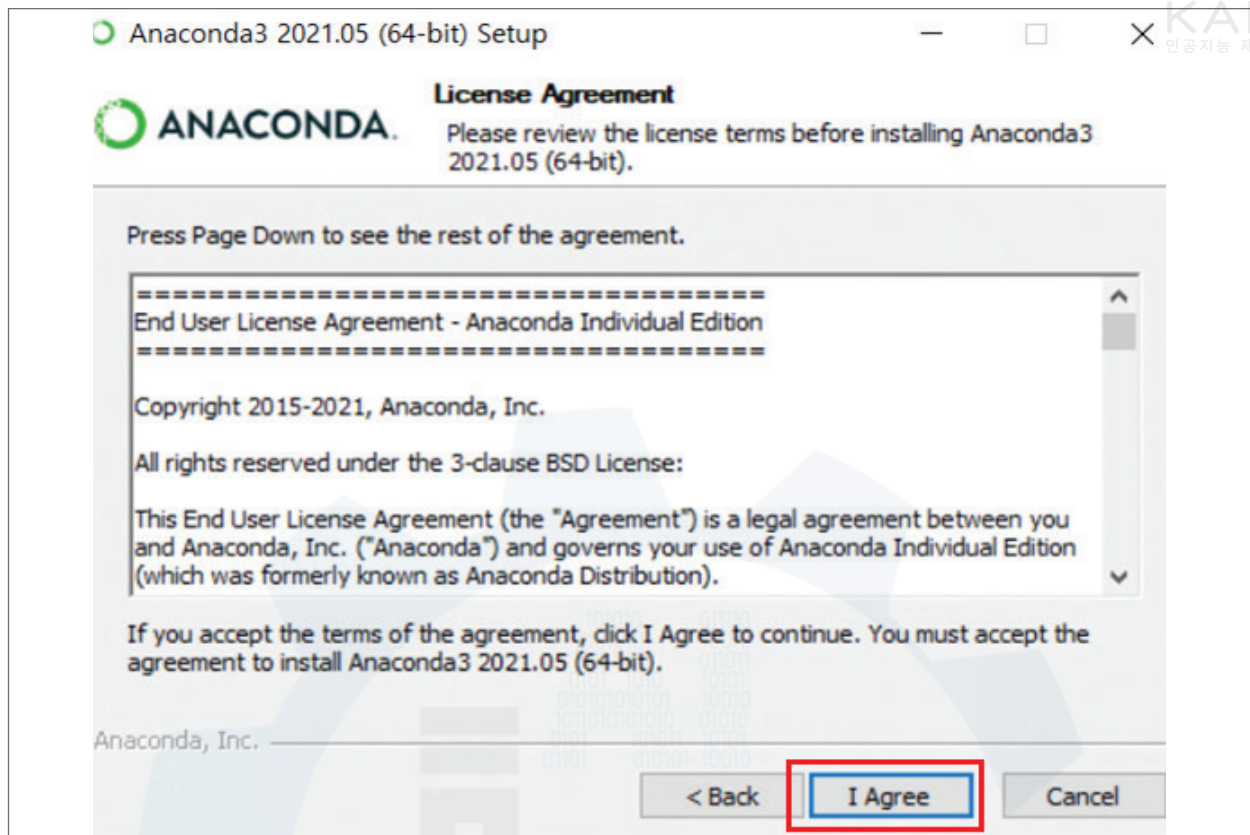


- ④ 파일을 실행 한 후, ‘Next’ 버튼을 클릭한다.

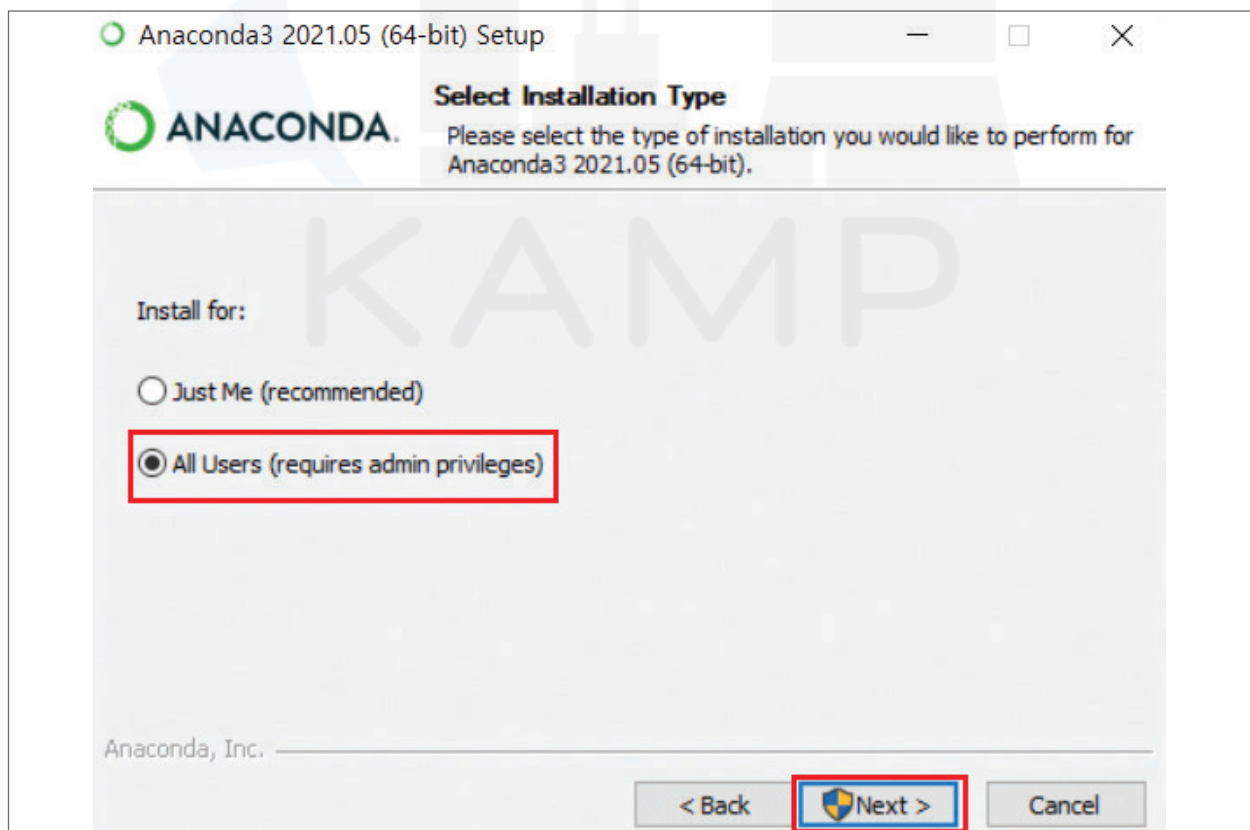




⑤ 다음 창이 나타나면 'I Agree'를 선택한다.

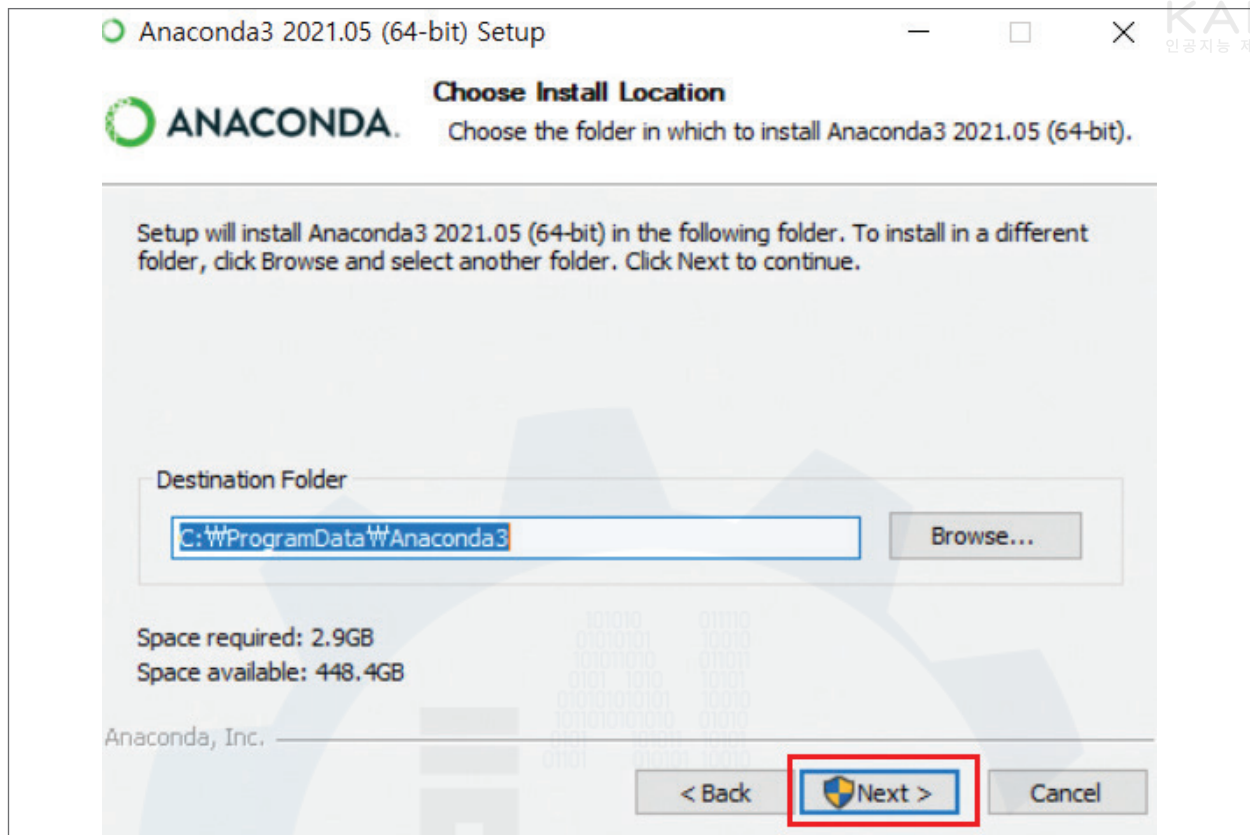


⑥ 세팅창이 뜨면 화면의 'All Users'를 선택 후 아래의 'Next'를 클릭한다.

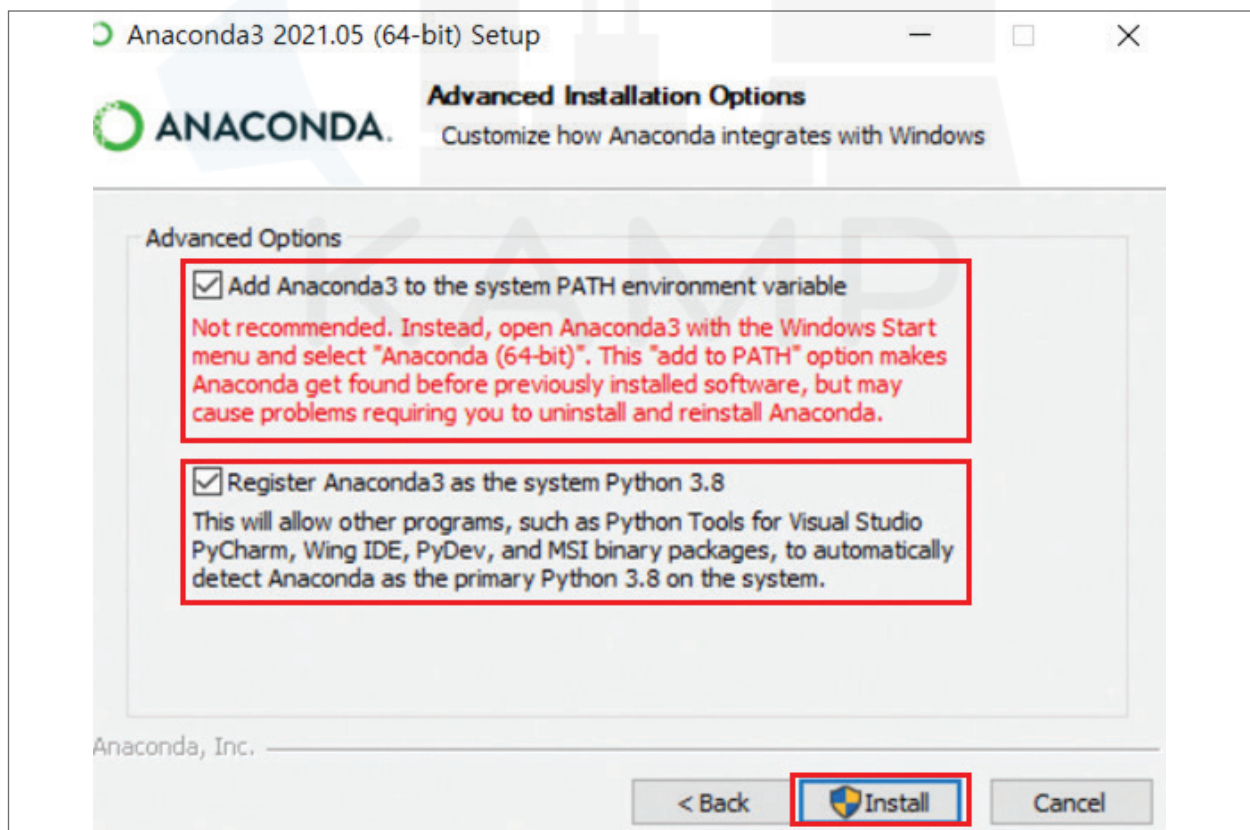




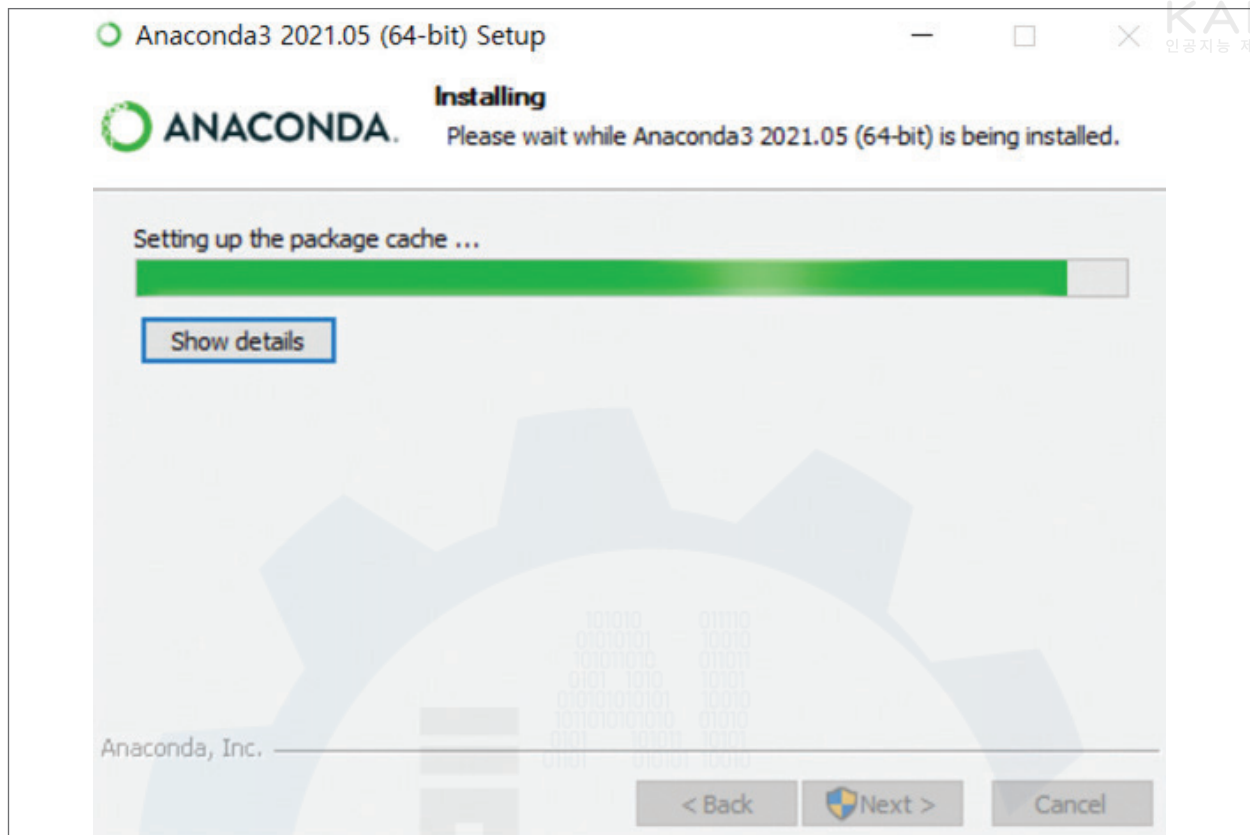
⑦ 다운로드 받을 경로를 물어보는 창이 뜨면, 아래의 'Next'를 클릭한다.



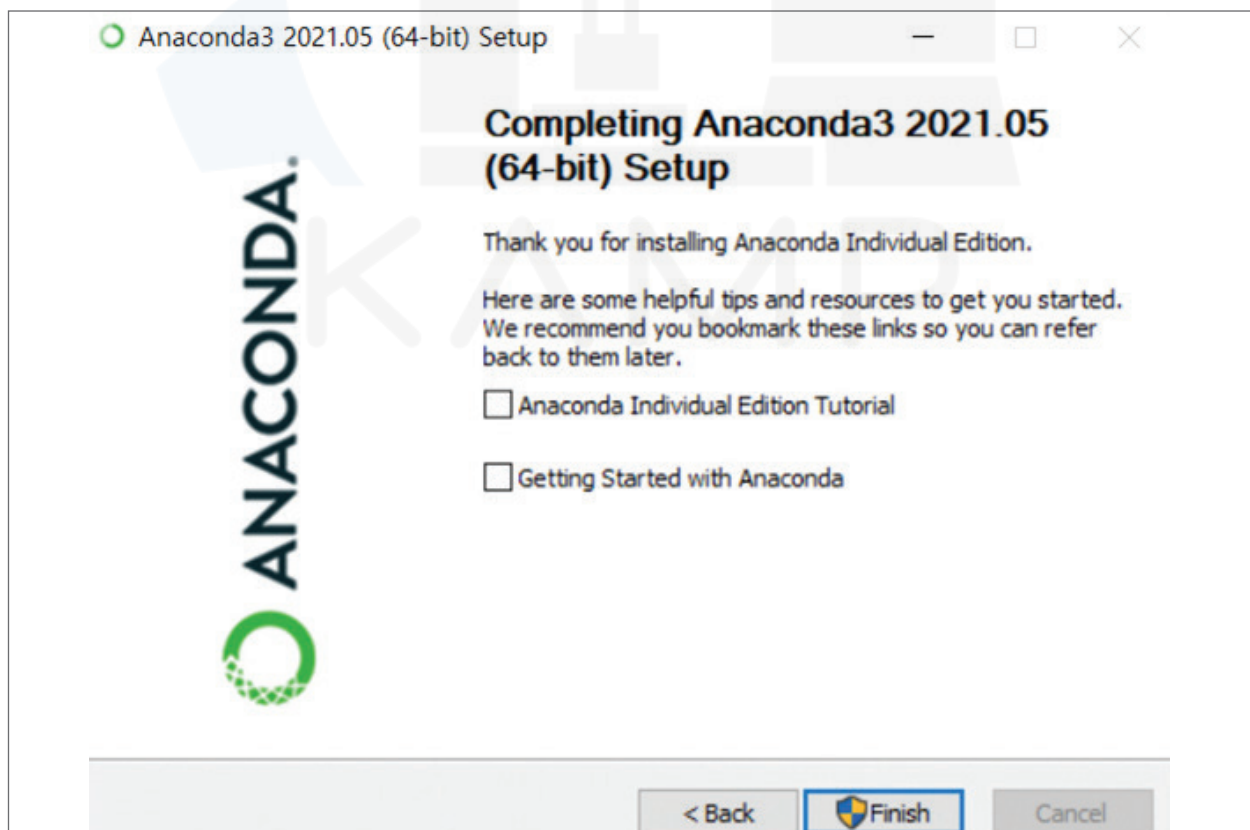
⑧ 고급 옵션 선택창이 뜨면, 아래와 같이 모두 선택 후, 'Install'을 클릭한다.



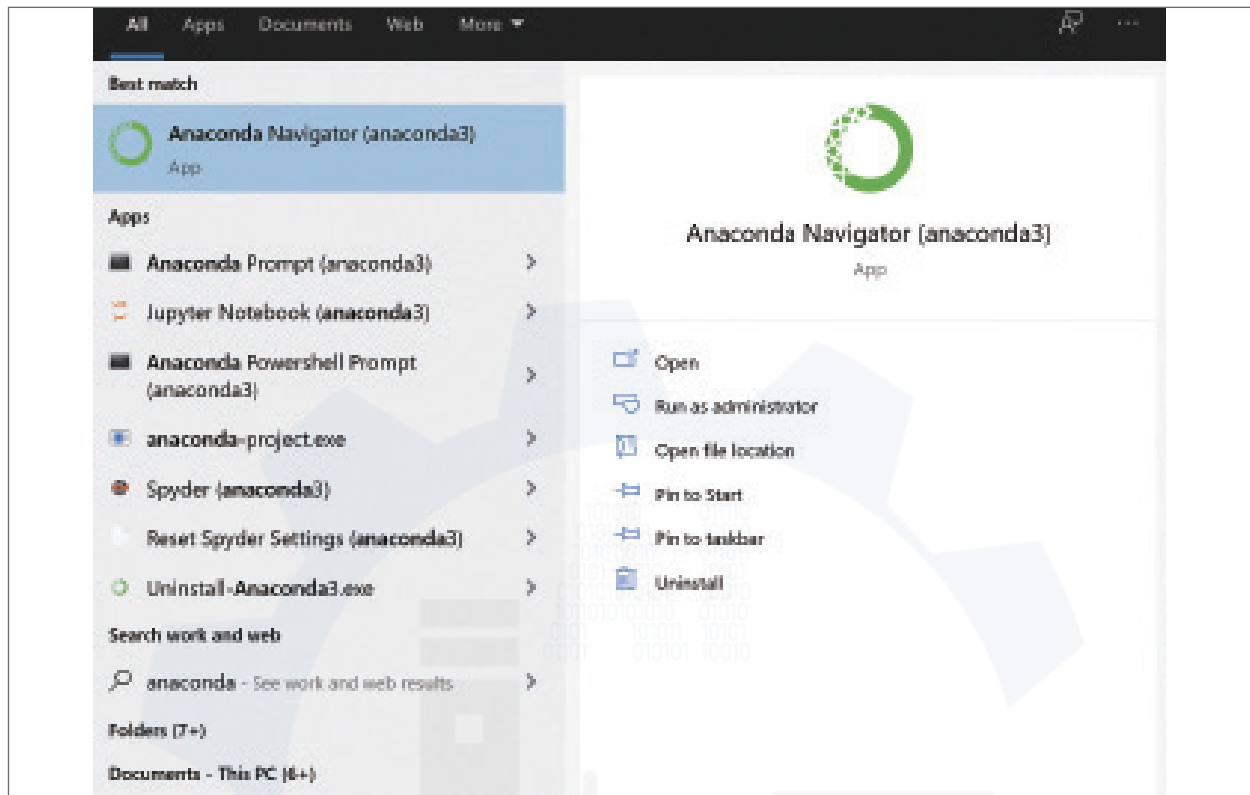
⑨ 다음과 같은 설치창이 뜨면 완료가 될 때까지 대기 (5분 이상 소요)



⑩ 마지막 화면에서, 모두 체크 해제한 후, 'Finish'를 눌러 설치를 완료한다.



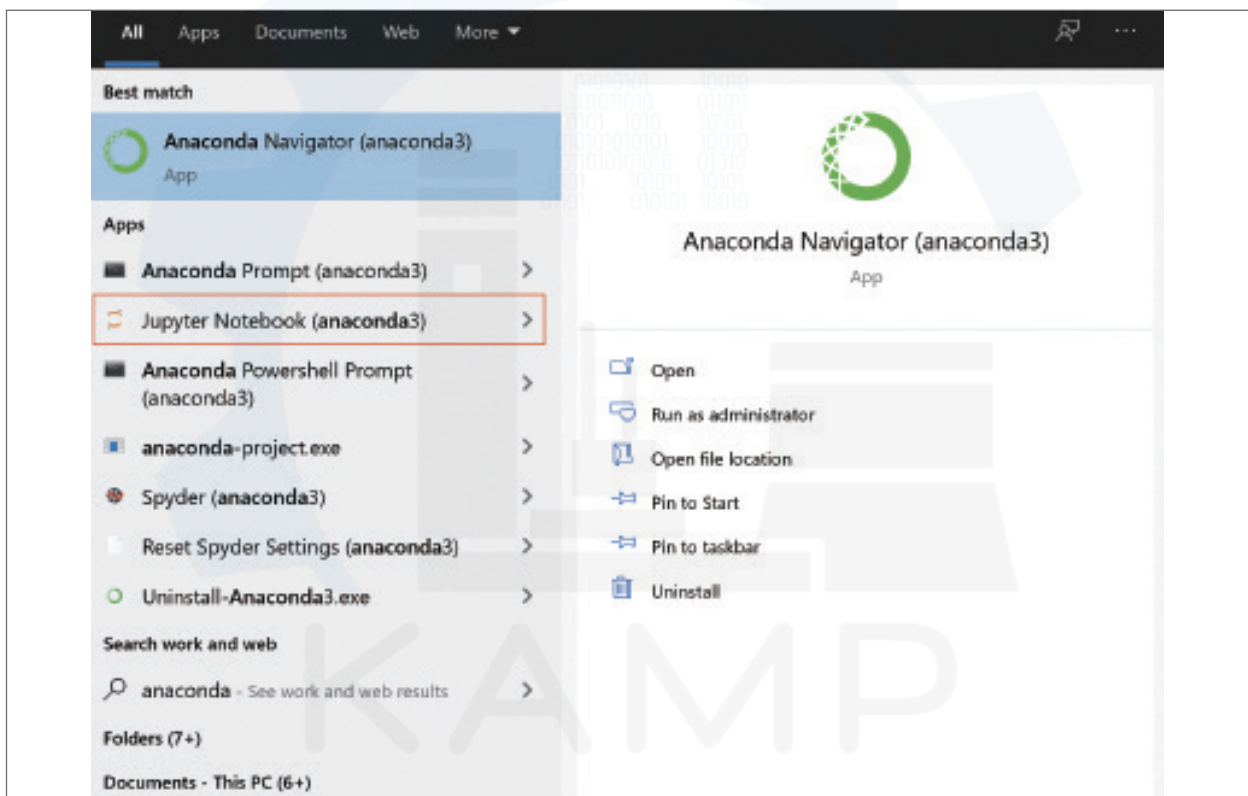
- ⑪ 화면상의 '홈(Windows)'키를 눌러서 화면과 같이 anaconda prompt가 잘 설치되었는지를 확인한다. Anaconda Navigator, Anaconda Prompt, Jupyter Notebook 등의 다른 응용 프로그램들도 함께 확인이 된다면 설치가 완료된 것이다.



3. 주피터 노트북 (Jupyter Notebook) 실행

- 주피터 노트북은 실제로 사용자가 코딩(분석 문장 작성)을 할 수 있는 도구이다.
- 쉽게 비교하자면, 문서 도구로 마이크로소프트사의 'Word' 프로그램이나, 한컴소프트 사의 '한글' 프로그램 등과 같은 도구라고 생각할 수 있다.
- 데이터 분석에 다양한 입력, 실행 도구가 있지만, 본 가이드북에서는 주피터 노트북을 활용하는 방법을 안내하기로 한다.
- Anaconda를 설치한 이후 주피터노트북(Jupyter Notebook) 설치 방법을 확인하면 된다.

① 화면상의 '홈(Windows)'키를 눌러서 화면과 같이 'anaconda' 검색 및 폴더 리스트 중 'Jupyter Notebook (anaconda3)'을 실행한다.





② Jupyter Notebook을 클릭 시 아래처럼 2개의 윈도우가 실행된다.

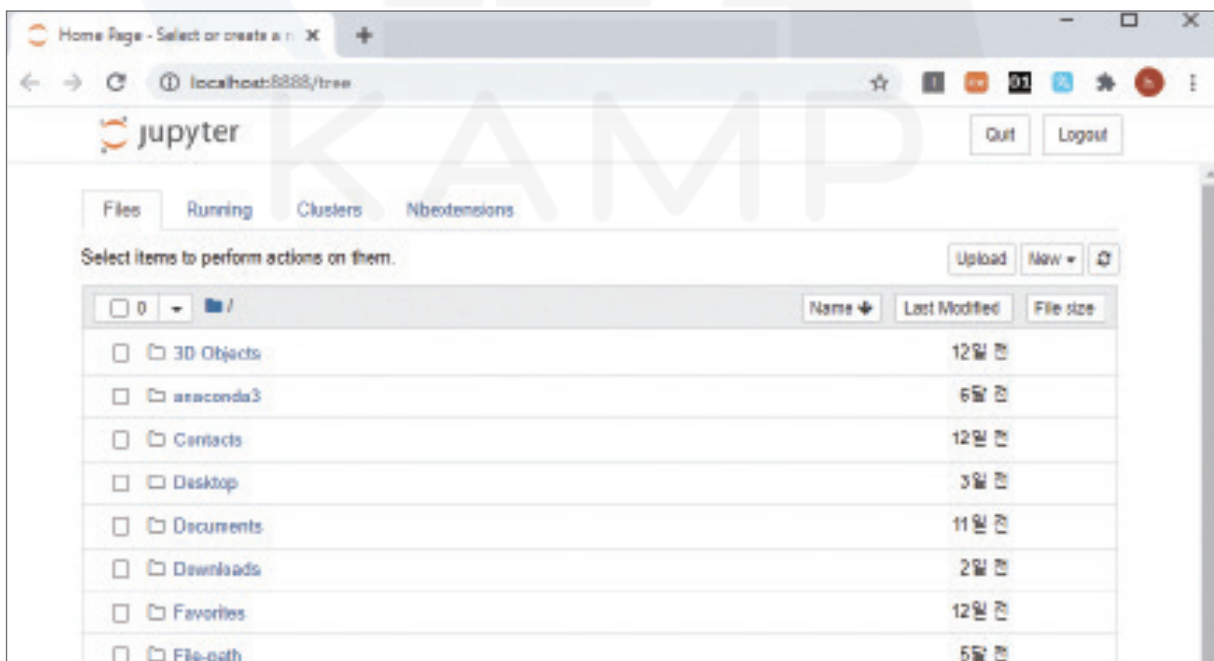
- (1) 검은색 배경의 화면은 주피터 노트북이 실행되는 환경에 대한 상태를 나타내는 상태 표시 창이다. 주피터 노트북을 사용하는 동안 종료하면 안 된다.

```

Jupyter Notebook (anaconda3)
[I 12:39:38.566 NotebookApp] JupyterLab extension loaded from C:\Users\leedaeyoung\Anaconda3\lib\site-packages\jupyterlab
[I 12:39:38.566 NotebookApp] JupyterLab application directory is C:\Users\leedaeyoung\Anaconda3\share\jupyter\lab
[I 12:39:38.627 NotebookApp] Serving notebooks from local directory: C:\Users\leedaeyoung
[I 12:39:38.628 NotebookApp] The Jupyter Notebook is running at:
[I 12:39:38.628 NotebookApp] http://localhost:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
[I 12:39:38.628 NotebookApp] or http://127.0.0.1:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
[I 12:39:38.628 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[12:39:39.053 NotebookApp]

To access the notebook, open this file in a browser:
file:///C:/Users/leedaeyoung/AppData/Roaming/jupyter/runtime/nbserver-6568-open.html
Or copy and paste one of these URLs:
http://localhost:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
or http://127.0.0.1:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
  
```

- (2) 사용하는 인터넷 프로그램(크롬, 인터넷 익스플로러)에 주피터 노트북이 열린다. (다른 확장 프로그램 사용하고 싶다면 기본 브라우저를 변경해 주어야 한다.)

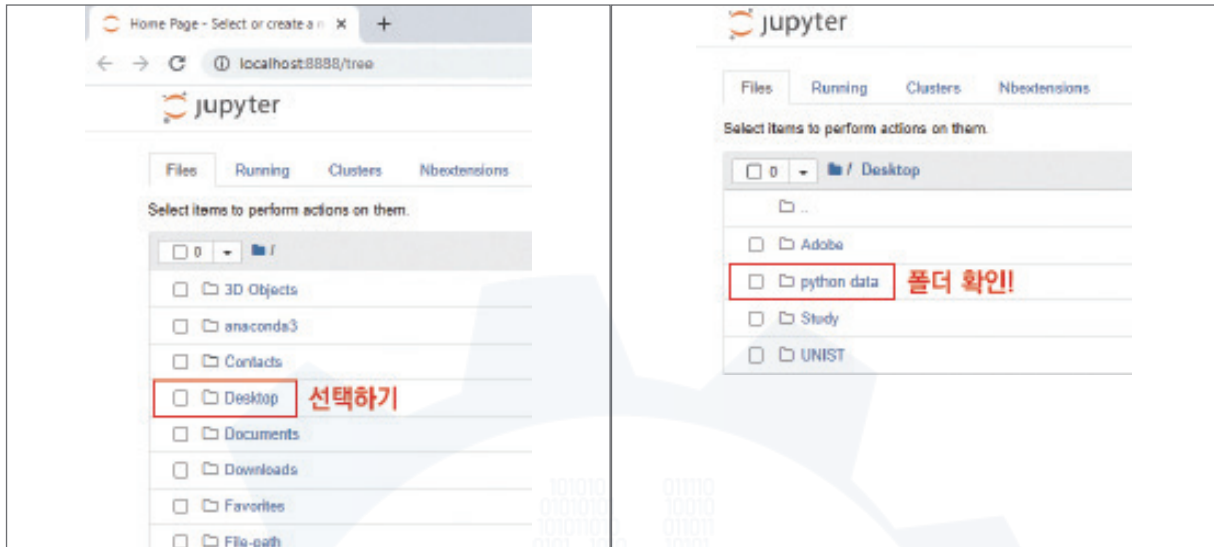




③ 데이터를 저장하고, 불러오고, 분석할 경로의 폴더를 하나 생성한다.

▶ (예) '바탕화면'에 'python data' 폴더를 생성 후 (미리 생성), 파이썬 코드 실행하고 저장한다.

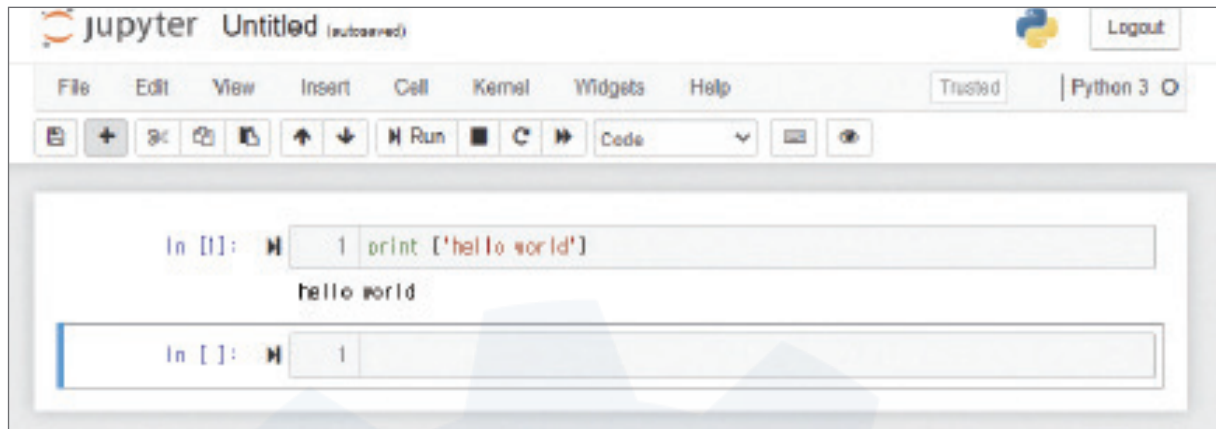
(1) 주피터 노트북에서 'python data' 폴더를 확인한다.



(2) python data에서 파이썬 파일을 생성한다. 오른쪽 상단의 'New'를 누른 후, 'Python 3'을 선택한다.



- (3) 'hello world' 출력을 확인해본다. 보이는 In [] 우측 회색 창에 print('hello world') 입력 후, 상단의  Run 버튼 혹은 shift + Enter 키를 눌러서 실행한다. 코드 파일의 확장자는 '.ipynb'로 저장된다.



4 부록_데이터 품질 전처리 [실습 코드]

앞선 '2. 분석 실습'에서 데이터 품질 전처리에 필요한, Gartner에서 제시한 6종의 데이터 품질 평가 지표에 대하여 논의하였다. (완전성, 유일성, 유효성, 일관성, 정확성, 무결성)

※ 본 분석의 염색공정 원본 데이터는 3개의 데이터셋으로 구성되어 있으며, 데이터 품질 전처리 작업도 각각의 데이터셋에 대해 진행하고자 한다.

• 필요 라이브러리 및 데이터 불러오기

- pandas 패키지 내의 'read_excel()' 또는 'read_csv()' 함수를 활용하여 EXCEL(.xls, .xlsx) 또는 CSV(.csv) 형식의 데이터를 불러올 수 있다.
- 가이드북 분석용 데이터인 'LOT 물량.xlsx', 'PRODUCTION_TREND.csv', 'CCM 측정값.xlsx' 파일을 분석 코드와 같은 폴더 내에 저장 후 사용한다.

```
# 필요 라이브러리 불러오기
import pandas as pd
import numpy as np

# 경고문구 제거 설정
import warnings
warnings.filterwarnings(action='ignore')
```

[코드 1] 필요 라이브러리 불러오기 및 경고문구 제거 설정

```
# SET1: LOT 물량 정보
workload = pd.read_excel('LOT 물량.xlsx')
# SET2: 시계열 설비데이터
operation = pd.read_csv('PRODUCTION_TREND.csv', encoding='CP949')
# SET3: CCM 검사결과
ccm = pd.read_excel('CCM 측정값.xlsx')
```

```
# SET1 데이터 확인
workload.head()
```

	EXT1_QTY(투입중량 (KG))	EXT2_QTY (액량 (LITER))	PRODT_ORDER_NO	JOB_CD	PROCESS_CD	단위중량	염색 가동 길이
0	245.6	4000	F2111300008	F12-1203	F12-1203	590	120.0
1	71.4	600	F2111300012	F12-1203	F12-1203	445	123.0
2	98.3	4000	F2111300020	F12-1203	F12-1203	0	460.0
3	97.0	4000	F2111300021	F12-1203	F12-1203	0	452.0
4	40.6	600	F2111300027	F12-1203	F12-1203	445	86.0

```
# SET2 데이터 확인
operation.head()
```

	LOT_NO	WC_CD	WC_CNT	SEQ_NO	PGM_ID	RESOURCE_CD	CR_TEMP	TRD_TEMP_SP	TRD_TEMP_PV	TRD_SPEED1	TRD_SPEED2	TRD_SPEED3
0	F2112280005	F12-1203	1	1	B9700	FCM04	30	30.0	14.5	0	0	0
1	F2112280005	F12-1203	1	2	B9700	FCM04	30	30.0	16.1	0	0	0
2	F2112280005	F12-1203	1	3	B9700	FCM04	30	30.0	19.6	111	1	11
3	F2112280005	F12-1203	1	4	B9700	FCM04	30	30.0	20.0	111	1	11
4	F2112280005	F12-1203	1	5	B9700	FCM04	30	30.0	19.9	111	1	11

```
# SET3 데이터 확인
ccm.head()
```

	lot_no	seq	oper_id	염색 색자 DL	염색 색자 DA	염색 색자 DB	염색 색자 DC	염색 색자 DH	염색 색자 DE
0	F2002240067	0	염색	1.402431	-0.079870	0.255513	0.063653	-0.260028	1.427754
1	F2003120058	0	염색	-0.043186	-0.021964	0.614762	0.032673	-0.614286	0.616669
2	F2006160123	0	원단검사	12.724530	-1.963228	-1.810259	1.940723	-1.834361	13.001730
3	F2009030093	0	원단검사	0.243153	0.251606	-3.862172	3.830362	-0.554988	3.877990
4	F2009030093	0	원단검사	0.673786	0.199348	-3.894216	3.854515	-0.589382	3.957100

[코드 2] 데이터 불러오기 및 상단 5개 데이터 확인

- 품질지수 분석에 앞서, 분석의 편의를 위해 품질지수 분석 시 사용될 변수명을 통일한다.

```
# SET1 변수명 변경
workload.rename(columns = {'PRODT_ORDER_NO':'LOT번호', 'JOB_CD':'공정코드'}, inplace=True)

# SET2 변수명 변경
operation.rename(columns = {'LOT_NO':'LOT번호', 'WC_CD':'공정코드', 'SEQ_NO':'공정진행시간'}, inplace=True)

# SET3 변수명 변경
ccm.rename(columns = {'lot_no':'LOT번호', 'seq':'검사차수'}, inplace=True)
```

[코드 3] 품질지수 분석 시 사용될 변수명 통일

▶ 완전성 품질지수

$$\text{완전성 품질 지수} = \left(1 - \frac{\text{결측 데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

- ① Null 값이 30% 이상인 데이터들은 데이터의 완전성이 떨어지기 때문에 각 열(column)에 대한 Null 값의 비율을 확인하여 삭제한다. 확인 결과, 데이터셋#1과 데이터셋#3은 Null 값 30%를 넘는 컬럼이 존재하지 않으므로 제거할 컬럼이 없는 반면, 데이터셋#2에는 30%를 넘는 컬럼이 존재하므로 해당 컬럼은 삭제한다(가이드북 [단계 ②] 데이터 탐색 및 특성 파악에서 진행).



```
# SET1
workload.isnull().sum()/len(workload)*100
```

```
EXT1_QTY(투입중량 (KG))    0.0
EXT2_QTY(액량 (LITER))    0.0
LOT번호                   0.0
공정코드                  0.0
PROCESS_CD                0.0
단위중량                  0.0
염색 가동 길이            0.0
dtype: float64
```



```
# SET1
perc=30
workload.isnull().sum()/len(workload)*100>perc
```

```
EXT1_QTY(투입중량 (KG))    False
EXT2_QTY(액량 (LITER))    False
LOT번호                   False
공정코드                  False
PROCESS_CD                False
단위중량                  False
염색 가동 길이            False
dtype: bool
```

```
# SET2
operation.isnull().sum()/len(operation)*100
```

```
LOT번호                   0.000000
공정코드                  0.000000
WC_CNT                   0.000000
공정진행시간              0.000000
PGM_ID                   0.000000
RESOURCE_CD               0.000000
CR_TEMP                  0.000000
TRD_TEMP_SP              0.000000
TRD_TEMP_PV              0.000000
TRD_SPEED1               0.000000
TRD_SPEED2               0.000000
TRD_SPEED3               0.000000
TRD_SPEED4               0.000000
INSRT_DT                 0.000000
PRODUCTION_RESULT_ID     85.732221
dtype: float64
```



```
# SET2
perc=30
operation.isnull().sum()/len(operation)*100>perc
```

```
LOT번호                   False
공정코드                  False
WC_CNT                   False
공정진행시간              False
PGM_ID                   False
RESOURCE_CD               False
CR_TEMP                  False
TRD_TEMP_SP              False
TRD_TEMP_PV              False
TRD_SPEED1               False
TRD_SPEED2               False
TRD_SPEED3               False
TRD_SPEED4               False
INSRT_DT                 False
PRODUCTION_RESULT_ID     True
dtype: bool
```

```
# SET3
ccm.isnull().sum()/len(ccm)*100
```

```
LOT번호                   0.0
검사차수                  0.0
oper_id                   0.0
염색 색차 DL              0.0
염색 색차 DA              0.0
염색 색차 DB              0.0
염색 색차 DC              0.0
염색 색차 DH              0.0
염색 색차 DE              0.0
dtype: float64
```



```
# SET3
perc=30
ccm.isnull().sum()/len(ccm)*100>perc
```

```
LOT번호                   False
검사차수                  False
oper_id                   False
염색 색차 DL              False
염색 색차 DA              False
염색 색차 DB              False
염색 색차 DC              False
염색 색차 DH              False
염색 색차 DE              False
dtype: bool
```

[코드 4] 분석 도구를 활용하여 각 컬럼의 결측치 비율 확인

- ② 총 결측치 개수를 확인하기 위해 'isnull()' 함수를 사용하여 각 열(column)의 개별 결측치 개수를 구한 뒤, 'sum()' 함수를 이용하여 합계를 구한다.



SET1 컬럼별 결측치 여부 확인
workload.isnull()

	EXT1_QTY(투입중량 (KG))	EXT2_QTY (액량 (LITER))	LOT번호	공정코드	PROCESS_CD	단위중량	염색 가동 길이
0	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...
2145	False	False	False	False	False	False	False
2146	False	False	False	False	False	False	False
2147	False	False	False	False	False	False	False
2148	False	False	False	False	False	False	False
2149	False	False	False	False	False	False	False

SET1 컬럼별 결측치 개수 확인
workload.isnull().sum()

```
EXT1_QTY(투입중량 (KG))    0
EXT2_QTY (액량 (LITER))    0
LOT번호                    0
공정코드                   0
PROCESS_CD                 0
단위중량                   0
염색 가동 길이             0
dtype: int64
```

SET1 총 결측치 개수 확인
set1_null = workload.isnull().sum().sum()
set1_null

0

[코드 5] 분석 도구를 활용하여 데이터셋#1 컬럼별 결측치 존재 여부 및 결측치 개수 확인



SET2 컬럼별 결측치 여부 확인
operation.isnull()

	LOT 번호	공정 코드	WC_CNT	공정 진행 시간	PGM_ID	RESOURCE_CD	CR_TEMP	TRD_TEMP_SP	TRD_TEMP_PV	TRD_SPEED1
0	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...
2119540	False	False	False	False	False	False	False	False	False	False
2119541	False	False	False	False	False	False	False	False	False	False
2119542	False	False	False	False	False	False	False	False	False	False
2119543	False	False	False	False	False	False	False	False	False	False
2119544	False	False	False	False	False	False	False	False	False	False

SET2 컬럼별 결측치 개수 확인
operation.isnull().sum()

LOT번호	0
공정코드	0
WC_CNT	0
공정진행시간	0
PGM_ID	0
RESOURCE_CD	0
CR_TEMP	0
TRD_TEMP_SP	0
TRD_TEMP_PV	0
TRD_SPEED1	0
TRD_SPEED2	0
TRD_SPEED3	0
TRD_SPEED4	0
INSRT_DT	0
PRODUCTION_RESULT_ID	1817133
dtype:	int64

SET2 총 결측치 개수 확인
set2_null = operation.isnull().sum().sum()
set2_null

1817133

[코드 6] 분석 도구를 활용하여 데이터셋#2 컬럼별 결측치 존재 여부 및 결측치 개수 확인



```
# SET3 컬럼별 결측치 여부 확인
ccm.isnull()
```

LOT번호	검사차수	oper_id	염색 색차 DL	염색 색차 DA	염색 색차 DB	염색 색차 DC	염색 색차 DH	염색 색차 DE
0	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...
609	False	False	False	False	False	False	False	False
610	False	False	False	False	False	False	False	False
611	False	False	False	False	False	False	False	False
612	False	False	False	False	False	False	False	False
613	False	False	False	False	False	False	False	False

```
# SET3 컬럼별 결측치 개수 확인
ccm.isnull().sum()
```

```
LOT번호      0
검사차수      0
oper_id      0
염색 색차 DL  0
염색 색차 DA  0
염색 색차 DB  0
염색 색차 DC  0
염색 색차 DH  0
염색 색차 DE  0
dtype: int64
```

```
# SET3 총 결측치 개수 확인
set3_null = ccm.isnull().sum().sum()
set3_null
```

```
0
```

[코드 7] 분석 도구를 활용하여 데이터셋#3 컬럼별 결측치 존재 여부 및 결측치 개수 확인



③ 구한 결측치의 개수를 이용하여 완전성 품질지수를 구한다.

```
# SET1 LOT 물량
print("\nSET1\n결측치 = %d개 \n완전성 지수 : %.2f%%"%(set1_null,
(1 - set1_null / (workload.shape[0]*workload.shape[1])) *100))

# SET2 시계열 설비데이터
print("\nSET2\n결측치 = %d개 \n완전성 지수 : %.2f%%"%(set2_null,
(1 - set2_null / (operation.shape[0]*operation.shape[1])) *100))

# SET3 CCM 검사결과
print("\nSET3\n결측치 = %d개 \n완전성 지수 : %.2f%%"%(set3_null,
(1 - set3_null / (ccm.shape[0]*ccm.shape[1])) *100))

#결과는 아래에서 확인 가능하다.
```

```
[SET1]
결측치 = 0개
완전성 지수 : 100.00%

[SET2]
결측치 = 1817133개
완전성 지수 : 94.28%

[SET3]
결측치 = 0개
완전성 지수 : 100.00%
```

[코드 8] 분석 도구를 활용하여 개별 데이터셋 완전성 품질지수 도출

```
tot_null = set1_null + set2_null + set3_null # 전체 결측치 개수

print("\n[종합]\n결측치 = %d개 \n완전성 지수 : %.2f%%"%(tot_null,
(1 - tot_null /
(workload.shape[0]*workload.shape[1] + operation.shape[0]*operation.
shape[1] + ccm.shape[0]*ccm.shape[1])) *100))

#결과는 아래에서 확인 가능하다.
```

```
[종합]
결측치 = 1817133개
완전성 지수 : 94.29%
```

[코드 9] 분석 도구를 활용하여 종합 완전성 품질지수 도출



▶ 유일성 품질지수

$$\text{유일성 품질 지수} = \left(\frac{\text{유일한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

① 데이터에서 유일한 값을 갖는 열(column)을 찾는다.

- 데이터셋#1은 'LOT번호' 열이 유일한 값을 가지는 기본키이다.
- 데이터셋#2는 시계열 데이터임에 따라 LOT번호별-공정번호별-공정시간을 결합한 데이터가 유일한 값을 가진다. 따라서 'LOT번호', '공정코드', '공정진행시간' 열을 기본키로 사용한다.
- 데이터셋#3은 LOT번호와 검사차수를 결합한 데이터셋이 유일한 값을 가짐에 따라 'LOT번호', '검사차수' 열을 기본키로 사용한다.

② 유일한 값을 갖는 열(column)에 대해 'groupby()' 함수를 이용하여 각 열(column)에서의 유일한 데이터 값을 확인하고, 'size()' 함수를 이용하여 유일한 데이터 개수를 구한다.

```
# SET1 유일한 데이터 확인
workload.groupby(['LOT번호']).size()
```

```
LOT번호
F2111300008    1
F2111300012    1
F2111300020    1
F2111300021    1
F2111300027    1
..
F2210130017    1
F2210130019    1
F2210130020    1
F2210130021    1
F2210130022    1
Length: 2127, dtype: int64
```

```
# SET1 유일한 데이터 개수 확인
len(workload.groupby(['LOT번호']).size())
```

```
2127
```

[코드 10] 분석 도구를 활용하여 데이터셋#1의 유일한 값 및 개수 확인



```
# SET2 유일한 데이터 확인
operation.groupby(['LOT번호', '공정코드', '공정진행시간']).size()
```

```
LOT번호    공정코드    공정진행시간
F2004270020  F12-1203    1          1
              2          1
              3          1
              4          1
              5          1
              ..
L2210180007  L12-1203    277        1
              278        1
              279        1
              280        1
              281        1
Length: 2099361, dtype: int64
```

```
# SET2 유일한 데이터 개수 확인
len(operation.groupby(['LOT번호', '공정코드', '공정진행시간']).size())
```

2099361

[코드 11] 분석 도구를 활용하여 데이터셋#2의 유일한 값 및 개수 확인

```
# SET3 유일한 데이터 확인
ccm.groupby(['LOT번호', '검사차수']).size()
```

```
LOT번호    검사차수
F2002240067  0          1
F2003120058  0          1
F2006160123  0          1
F2009030093  0          2
F2009050181  0          1
              ..
f2208040130  0          1
f2208090010  0          1
f2208250020  0          2
f22083080214  0          1
f2209080008  0          1
Length: 522, dtype: int64
```

```
# SET3 유일한 데이터 개수 확인
len(ccm.groupby(['LOT번호', '검사차수']).size())
```

522

[코드 12] 분석 도구를 활용하여 데이터셋#3의 유일한 값 및 개수 확인

③ 구한 유일한 데이터 개수를 이용하여 유일성 품질지수를 구한다.

- 유일성 품질지수 확인을 통해 3개 데이터셋 모두에 중복데이터가 존재함을 확인할 수 있다. 중복데이터 및 비유일성 데이터에 대한 처리는 가이드북 [단계③] 데이터 전처리 및 병합에서 진행할 예정이다.



```

uniq_len1 =len(workload.groupby(['LOT번호']).size())
uniq_len2 =len(operation.groupby(['LOT번호', '공정코드', '공정진행시간']).size())
uniq_len3 =len(ccm.groupby(['LOT번호', '검사차수']).size())

```

```

print("[SET1] 유일성 지수 : %.2f%%" % ((uniq_len1 / len(workload)) * 100))
print("[SET2] 유일성 지수 : %.2f%%" % ((uniq_len2 / len(operation)) * 100))
print("[SET3] 유일성 지수 : %.2f%%" % ((uniq_len3 / len(ccm)) * 100))

```

#결과는 아래에서 확인 가능하다.

```

[SET1] 유일성 지수 : 98.93%
[SET2] 유일성 지수 : 99.05%
[SET3] 유일성 지수 : 85.02%

```

[코드 13] 분석 도구를 활용하여 개별 데이터셋 유일성 품질지수 도출

```

uniq_len_tot = uniq_len1 + uniq_len2 + uniq_len3
print("[종합] 유일성 지수 : %.2f%%" % ((uniq_len_tot / (len(workload) + len(operation) + len(ccm))) * 100))

```

#결과는 아래에서 확인 가능하다.

```

[종합] 유일성 지수 : 99.04%

```

[코드 14] 분석 도구를 활용하여 종합 유일성 품질지수 도출

▶ 유효성 품질지수

$$\text{유효성 품질 지수} = \left(\frac{\text{유효성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

① 데이터가 유효범위 내에 있는가?

- 데이터셋에 정의된 수집 범위를 이용하여 조건(유효범위)을 모두 충족하는 데이터들을 vald_dt에 저장한다.
- 본 품질지수 분석에 사용된 전처리 전 3개의 데이터셋은 각기 다른 유효범위('LOT번호')를 가진다. 이들 3개의 데이터셋은 전처리 단계에서 'LOT번호'를 기준으로 결합되며, 이때 모든 데이터셋에 존재하는 'LOT번호'만 남게 되어 해당 'LOT번호'가 분석범위로 사용된다. 따라서, 현재의 전처리 전 데이터셋으로는 유효범위를 확인하는 것이 무의미함에 따라 모든 데이터가 유효범위 안에 있는 것으로 간주한다.

```

# SET1 LOT번호 개수 확인
print("[SET1] LOT번호 개수 : {}".format(workload['LOT번호'].nunique()))

# SET2 LOT번호 개수 확인
print("[SET2] LOT번호 개수 : {}".format(operation['LOT번호'].nunique()))

# SET3 LOT번호 개수 확인
print("[SET3] LOT번호 개수 : {}".format(ccm['LOT번호'].nunique()))

```

```

[SET1] LOT번호 개수 : 2127
[SET2] LOT번호 개수 : 7003
[SET3] LOT번호 개수 : 434

```

[코드 15] 분석 도구를 활용하여 각 데이터셋의 LOT번호 개수 확인



```
vald_dt1 = workload
vald_dt2 = operation
vald_dt3 = ccm
```

[코드 16] 분석 도구를 활용하여 유효범위 내에 있는 데이터를 vald_dt에 저장

② 데이터가 형식에 맞는가?

- 'info()' 함수를 이용하여 데이터의 형식을 확인한다.

```
# SET1
vald_dt1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2150 entries, 0 to 2149
Data columns (total 7 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   EXT1_QTY(투입중량 (KG))                2150 non-null   float64
1   EXT2_QTY (액량 (LITER))                2150 non-null   int64
2   LOT번호                                2150 non-null   object
3   공정코드                              2150 non-null   object
4   PROCESS_CD                            2150 non-null   object
5   단위중량                              2150 non-null   int64
6   염색 가동 길이                        2150 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 117.7+ KB
```

[코드 17] 'info()' 함수를 활용하여 데이터셋#1의 데이터 형식 확인

- 'LOT번호' 데이터에 유효하지 않은 형식의 데이터가 들어있는지 unique() 함수를 이용하여 확인한다. 동일한 방법으로 다른 열의 데이터도 확인해볼 수 있다. [코드 18]의 결과를 통해 유효성을 위배하는 데이터가 없는 것을 알 수 있다.

```
vald_dt1['LOT번호'].unique()
```

```
array(['F2111300008', 'F2111300012', 'F2111300020', ..., 'F2210130020',
       'F2210130021', 'F2210130022'], dtype=object)
```

```
vald_dt1['공정코드'].unique()
```

```
array(['F12-1203'], dtype=object)
```

[코드 18] 'unique()' 함수를 활용한 데이터셋#1의 데이터 유효성 검사

- 데이터셋#2의 데이터 형식도 위와 동일한 방법으로 확인한다.



```
# SET2
vald_dt2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2119545 entries, 0 to 2119544
Data columns (total 15 columns):
#   Column                Dtype
---  -
0   LOT번호                object
1   공정코드              object
2   WC_CNT                int64
3   공정진행시간          int64
4   PGM_ID                object
5   RESOURCE_CD           object
6   CR_TEMP               int64
7   TRD_TEMP_SP           float64
8   TRD_TEMP_PV           float64
9   TRD_SPEED1            int64
10  TRD_SPEED2            int64
11  TRD_SPEED3            int64
12  TRD_SPEED4            int64
13  INSRT_DT              object
14  PRODUCTION_RESULT_ID  object
dtypes: float64(2), int64(7), object(6)
memory usage: 242.6+ MB
```

[코드 19] 'info()' 함수를 활용하여 데이터셋#2의 데이터 형식 확인

- 데이터셋#2의 'INSRT_DT' 열을 날짜(datetime) 형식으로 변경해준다. to_datetime() 함수를 이용하여 날짜 형식으로 변경할 때 함수 파라미터로 errors='coerce' 옵션을 추가하면, 올바르지 않은 형식을 Null 값으로 채워준다. 날짜 형식 변경 후, Null 값의 존재여부 확인을 통해 기존 데이터에 올바르지 않은 형식의 데이터가 있었는지를 확인할 수 있게 된다. [코드 21]의 결과를 통해, 'INSRT_DT' 열의 모든 데이터의 형식이 올바르다는 것을 알 수 있다.

```
vald_dt2['INSRT_DT'] = pd.to_datetime(vald_dt2['INSRT_DT'], format='%Y-%m-%d', errors='coerce')
```

[코드 20] 'to_datetime()' 함수를 활용하여 데이터셋#2의 데이터 형식 변경

```
vald_dt2[vald_dt2['INSRT_DT'].isnull()]
```

LOT 번호	공 정 코 드	WC_CNT	공 정 진 행 시 간	PGM_ID	RESOURCE_CD	CR_TEMP	TRD_TEMP_SP	TRD_TEMP_PV	TRD_SPEED1	TRD_SPEED2

[코드 21] 분석 도구를 활용하여 데이터셋#2의 데이터 형식 확인

```
vald_dt2['공정코드'].unique()
```

```
array(['F12-1203', 'L12-1203', 'F12-1211', 'L12-1211', 'F12-1212', '1',
      '1203', 'F12-1213', 'F12-1230', 'L12-1210', 'F123', 'F12-0123',
      'L12'], dtype=object)
```

[코드 22] 'unique()' 함수를 활용한 데이터셋#2의 데이터 유효성 검사



- 위와 동일한 함수를 활용하여 데이터셋#3의 데이터 형식과 데이터 유효성을 확인한다.

```
# SET3
vald_dt3.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   LOT번호      614 non-null    object
1   검사차수      614 non-null    object
2   oper_id      614 non-null    object
3   염색 색차 DL  614 non-null    float64
4   염색 색차 DA  614 non-null    float64
5   염색 색차 DB  614 non-null    float64
6   염색 색차 DC  614 non-null    float64
7   염색 색차 DH  614 non-null    float64
8   염색 색차 DE  614 non-null    float64
dtypes: float64(6), object(3)
memory usage: 43.3+ KB
```

```
# 데이터 유효성 검사
vald_dt3['검사차수'].unique()
```

```
array([0, 'QC', 1, 2], dtype=object)
```

```
# 데이터 유효성 검사
vald_dt3['oper_id'].unique()
```

```
array(['염색', '원단검사'], dtype=object)
```

[코드 23] 분석 도구를 활용하여 데이터셋#3의 데이터 형식 및 유효성 확인

③ 수집된 날짜 안에 들어가 있는가?

- 본 분석데이터는 2022년 01월 01일부터 2022년 10월 31일에 수집된 데이터이기 때문에, 데이터가 해당 기간을 벗어나지 않았는지 확인한다.
- 다만, 데이터셋#1과 데이터셋#3은 날짜 변수가 없고, 데이터셋#2만 날짜 변수가 있으므로, 데이터셋#2의 날짜 범위 유효여부만 확인한다. 데이터셋#1과 데이터셋#3의 경우, 'LOT번호' 열에 날짜로 유추할 수 있는 정보가 포함되어 있기는 하나, 'LOT번호'의 날짜정보와 실제 공정일시 정보가 일치하지 않는 경우가 많기 때문에(데이터셋#2의 'LOT번호' 열과 'INSRT_DT' 열 참고) 이는 정확하지 않은 정보로 판단하고, 사용하지 않기로 한다.

```
import datetime

d0 = pd.Timestamp(datetime.date(2022,1,1)) # 데이터 수집 시작일
d1 = pd.Timestamp(datetime.date(2022,10,31)) # 데이터 수집 종료일

set2_con1 = vald_dt2['INSRT_DT'] >= d0
set2_con2 = vald_dt2['INSRT_DT'] <= d1

vald_dt2 = vald_dt2[set2_con1 & set2_con2] # 데이터 수집기간 안에 들어오는 데이터만 추출
```

[코드 24] 분석 도구를 활용하여 데이터 수집 기간 확인

- ④ 위 3가지 경우를 모두 만족하는 데이터는 vald_dt에 저장하고, 최종적으로 완성된 데이터를 이용하여 유효성 품질지수를 구한다.

```
vald_len1 =len(vald_dt1)
vald_len2 =len(vald_dt2)
vald_len3 =len(vald_dt3)

print("[SET1] 유효성 지수 : %.2f%%"%(vald_len1 /len(workload) *100))
print("[SET2] 유효성 지수 : %.2f%%"%(vald_len2 /len(operation) *100))
print("[SET3] 유효성 지수 : %.2f%%"%(vald_len3 /len(ccm) *100))

#결과는 아래에서 확인 가능하다.
```

```
[SET1] 유효성 지수 : 100.00%
[SET2] 유효성 지수 : 100.00%
[SET3] 유효성 지수 : 100.00%
```

[코드 25] 분석 도구를 활용하여 개별 데이터셋 유효성 품질지수 도출

```
vald_len_tot = vald_len1 + vald_len2 + vald_len3
print("[종합] 유효성 지수 : %.2f%%"%(vald_len_tot / (len(workload) +len(operation) +len(ccm)) *100))
```

```
[종합] 유효성 지수 : 100.00%
```

[코드 26] 분석 도구를 활용하여 종합 유효성 품질지수 도출

▶ 일관성 품질지수

$$\text{일관성 품질 지수} = \left(\frac{\text{일관성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- 본 데이터가 구조, 값, 형태를 일관성 있게 지키고 있는지 확인한다. 'LOT번호' 열의 데이터가 일관된 형식(알파벳 1자리, 연월일 날짜 6자리, 순번 4자리, (예: F2111300008))의 데이터를 가졌는지는 앞서 유효성을 살펴보면서 확인하였다. 이외에 공정 및 품질검사 데이터 값들의 형태는 'dtypes()' 함수를 이용하여 정수(int64) 또는 실수(float64) 형태인지 확인한다.

# SET1 workload.dtypes	# SET2 operation.dtypes	# SET3 ccm.dtypes
EXT1_QTY(투입중량 (KG)) float64 EXT2_QTY(액량 (LITER)) int64 LOT번호 object 공정코드 object PROCESS_CD object 단위중량 int64 염색 자동 길이 float64 dtype: object	LOT번호 object 공정코드 object WC_CNT int64 공정진행시간 int64 PGM_ID object RESOURCE_CD object CR_TEMP int64 TRD_TEMP_SP float64 TRD_TEMP_PV float64 TRD_SPEED1 int64 TRD_SPEED2 int64 TRD_SPEED3 int64 TRD_SPEED4 int64 INSRT_DT datetime64[ns] PRODUCTION_RESULT_ID object dtype: object	LOT번호 object 검사차수 object oper_id object 염색 색차 DL float64 염색 색차 DA float64 염색 색차 DB float64 염색 색차 DC float64 염색 색차 DH float64 염색 색차 DE float64 dtype: object

[코드 27] 'dtypes()' 함수를 활용하여 데이터 타입 확인



- 본 데이터는 일관성이 있다고 할 수 있다.

▶ 정확성 품질지수

$$\text{정확성 품질 지수} = 1 - \left(\frac{\text{정확성에 위배되는 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터의 '평균값'이 수집된 데이터에 한하여, 다음과 같이 정확성 품질지수를 확인할 수 있다.
- ② 누락된 데이터가 있는지 확인하기 위해, 'mean()'함수를 이용하여 데이터들의 평균을 구한다.
- ③ 'mean()'함수로 구한 데이터의 평균에서 데이터의 '평균값'과의 차이를 구한다.
- ④ 이 차이가 0이 아니면 데이터를 위배하는 데이터가 존재하게 된다.

* 본 분석에 사용된 염색공정 AI데이터셋은 데이터의 '평균값'이 수집되지 않았기 때문에 정확성 품질지수를 확인할 수 없다.

▶ 무결성 품질지수

$$\text{무결성 품질지수} = \left\{ 1 - \left(\frac{\text{유일성, 유효성, 일관성 중 100%가 아닌 지수}}{3} \right) \right\} \times 100$$

- 데이터셋#1의 유일성은 98.93%, 유효성과 일관성은 100%로, 세 가지 지수 중 유일성 지수가 100%를 만족하지 못하므로 이를 사용하여 도출한 무결성 품질지수는 66.67%이다.
- 데이터셋#2의 유일성은 99.05%, 유효성과 일관성은 100%로, 세 가지 지수 중 유일성 지수가 100%를 만족하지 못하므로 이를 사용하여 도출한 무결성 품질지수는 66.67%이다.
- 데이터셋#3의 유일성은 85.02%, 유효성과 일관성은 100%로, 세 가지 지수 중 유일성 지수가 100%를 만족하지 못하므로 이를 사용하여 도출한 무결성 품질지수는 66.67%이다.
- 개별 데이터셋의 지수를 모두 종합할 경우, 유일성은 99.04%, 유효성과 일관성은 100%로, 세 가지 지수 중 유일성 지수가 100%를 만족하지 못하므로 이를 사용하여 도출한 종합 무결성 품질지수는 66.67%이다.

▶ 가중치 지수

$$\text{가중치지수} = \text{품질지수} \times \text{가중치}$$

- 데이터의 특성에 따라 품질지수의 중요도는 달라진다. 활용되는 데이터의 중요한 품질지수가 무엇인지를 평가하고, 데이터의 특성에 따라 적절한 가중치를 부여한다.
- 본 분석데이터에서 대량의 결측치가 발생한 열(데이터셋#2의 'PRODUCTION_RESULT_iD')의 경우, 실제 AI분석에 사용되지 않을 변수임에 따라 사실상 해당 결측치는 분석에 전

혀 영향을 미치지 않는다. 따라서 완전성의 가중치를 5%를 부여하여, 해당 지수가 전체 품질지수에 영향을 거의 미치지 않도록 한다. 이 같은 조치는 본 분석의 특이한 상황에 따른 것으로, 일반적인 경우는 이보다 높게 책정하여야 할 것이다. 특히, AI분석은 데이터의 개수가 많을수록 더 좋은 학습 및 예측 결과를 얻을 수 있는데, 만약 결측치 개수가 너무 많아서 해당 변수를 사용하지 못하게 되거나, 다량의 행(row)을 삭제해야 하는 상황이 생기면 데이터 양을 확보하지 못하게 될 수 있으니 데이터의 완전성은 중요하게 고려되어야 할 지수이다.

- 본 분석의 설비데이터(데이터셋#2)는 시계열 데이터라는 특성상, LOT별, 시간대별로 단 하나의 값만 존재해야 한다. 마찬가지로, 품질데이터(데이터셋#3)는 생산품질관리 특성상 LOT별, 검사차수별로 단 하나의 값만 존재해야 한다. 이처럼 유일성을 위반하는 데이터가 시스템적으로 존재해서는 안 되기 때문에(하지만 데이터수집 시스템의 오류로 실제로는 종종 발생한다.), 유일성의 가중치와 지수 산출 과정에서 유일성을 고려한 무결성의 가중치를 각각 25%씩 부여하여, 상대적으로 높게 가중치를 둔다.
- 나머지 품질지수인 유효성, 일관성, 정확성은 동일하게 중요하다는 가정에서 가중치를 각 17.5%씩 부여한다.

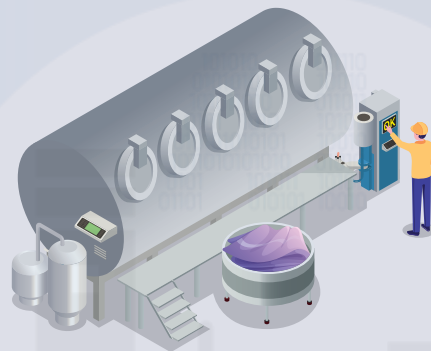
▶ [종합] 데이터 품질지수

데이터 품질지수 = $\sum$ 가중치지수

- 최종적으로 구한 6가지 데이터 품질지수에 적절한 가중치를 부여하여 종합 품질지수를 구한다. 품질지수 가중치는 활용되는 데이터의 특성에 따라 부여할 수 있다. 중요한 품질지수가 무엇인지를 평가하고, 데이터의 특성에 따라 가중치를 부여한다.
- 본 분석데이터의 종합 품질지수 결과는 아래와 같다. 가중치를 부여하지 않고 6가지 품질지수(완전성, 유일성, 유효성, 일관성, 정확성, 무결성)의 평균을 구한 평균 품질지수는 93.33%이다.
- 각각의 품질지수에 가중치를 적용한 가중치 품질지수는 93.64%로, 가중치를 적용하지 않은 품질지수 93.33%보다 0.31% 높은 값이 나왔다. 가중치 품질지수의 오류율은 6.36%이다.

구분	품질지수	가중치	가중치 지수	오류율
완전성	94.29%	5.00%	4.71%	0.29%
유일성	99.04%	25.00%	24.76%	0.24%
유효성	100.00%	17.50%	17.50%	0.00%
일관성	100.00%	17.50%	17.50%	0.00%
정확성	100.00%	17.50%	17.50%	0.00%
무결성	66.67%	17.50%	11.67%	5.83%
총합	93.33%	100%	93.64%	6.36%

※ 3개 데이터셋의 개별 데이터셋 품질지수를 모두 합한 총합 품질지수 기준



「의류 공정최적화 AI 데이터셋」 분석실습 가이드북